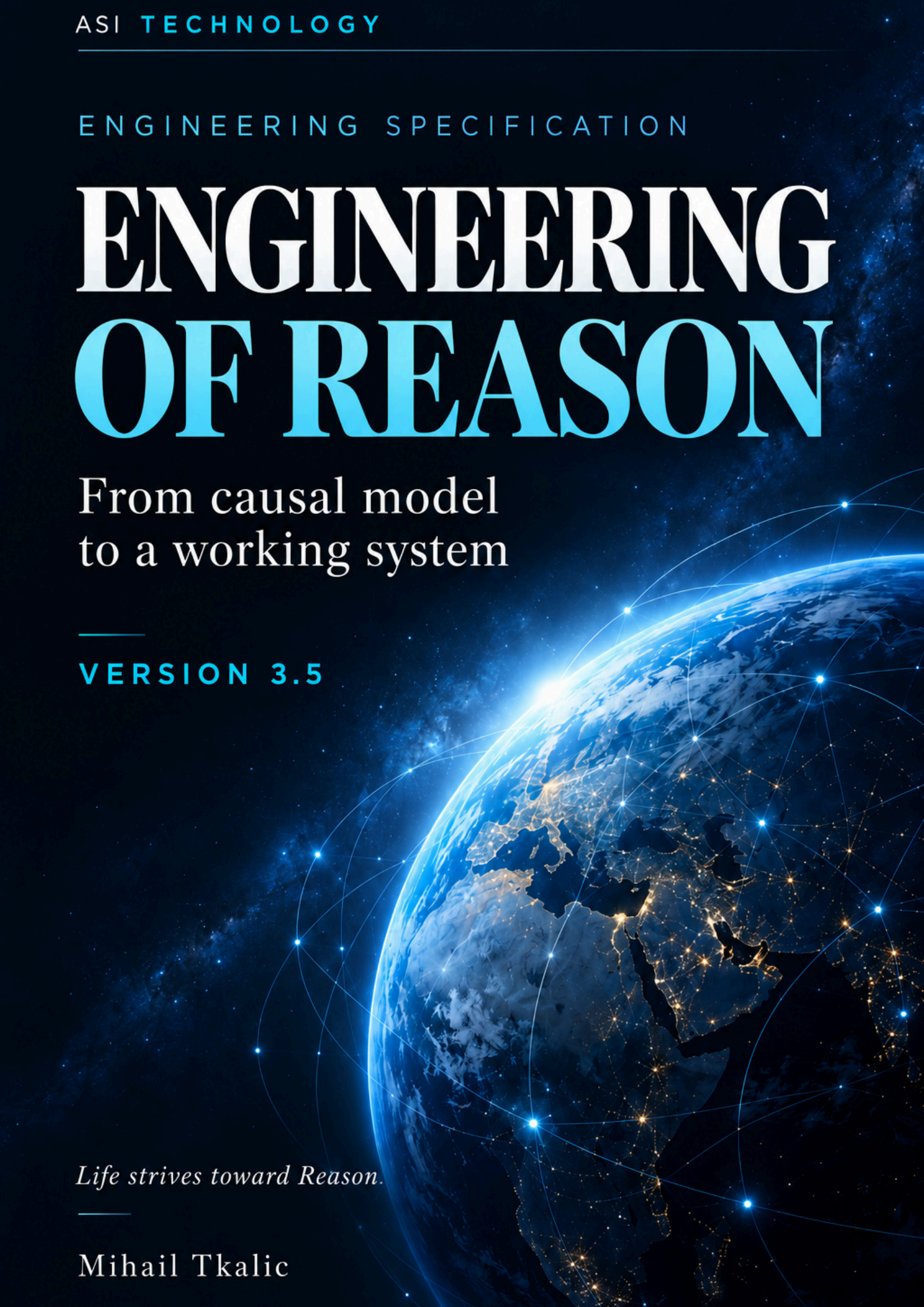
ASI TECHNOLOGY

ENGINEERING SPECIFICATION

ENGINEERING OF REASON

From causal model to a working system

VERSION 3.5



Life strives toward Reason.

Mihail Tkalic

Foundations of Reason in 15 Minutes

One problem through which the architecture becomes visible

Mihail Tkalic · Foundations of Reason

Short guide, edition 1.1 · October 10, 2026

Based on *Foundations of Reason* 8.0 and *Engineering of Reason*, Volume II, 3.5.

This text is an entry point into the two volumes. It shows why the model's main functions are needed, how they connect, and what should be tested experimentally. Chapter and section numbers at the end refer to the versions stated above.

1. One command - several different unknowns

The system receives a command to switch on a piece of equipment. It sends the command, but the sensor reading remains unchanged.

What happened? Perhaps the command did not arrive. Perhaps the actuator failed. Perhaps the equipment is already operating but the sensor does not show it. Finally, the model of the equipment itself, or the original formulation of the task, may be wrong.

Powerful Intelligence can propose many explanations and actions. The next question is: **what exactly needs to be established now, why does it matter, and what evidence would actually change the decision?**

Foundations of Reason studies the organization of such a choice. Intelligence constructs possible explanations and ways of acting. In the proposed model, Reason organizes their use: it detects an insufficiency of grounds, selects a test, relates consequences to values, corrects the model, and preserves the correction for subsequent situations.

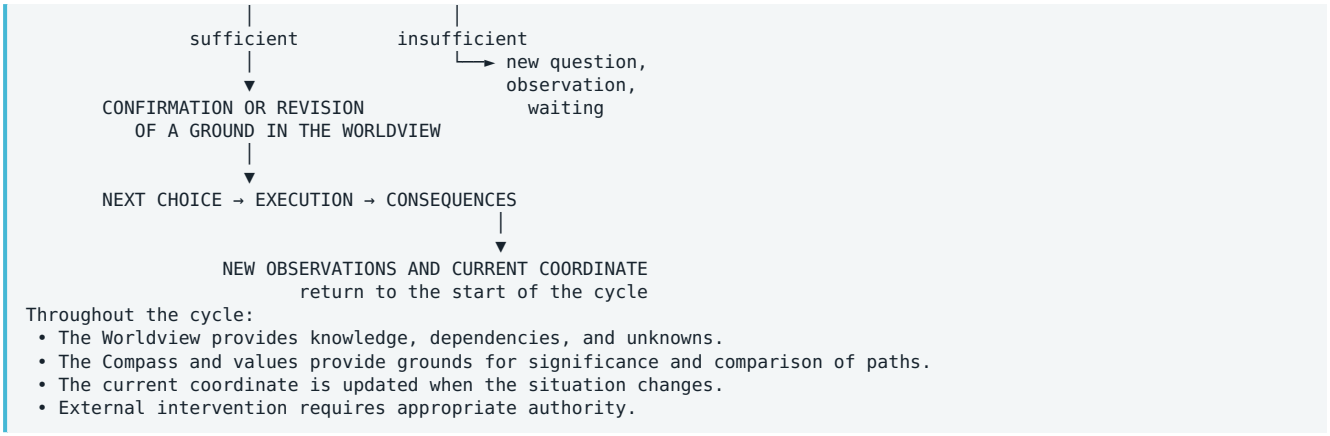
A **ground** here means what a conclusion or action actually relies on. Examples are: "the command was delivered," "the sensor is functioning," "this method is suitable for this equipment," or "switching it on now is actually necessary."

Main thesis: a correction must change the ground that generates decisions and, through it, the system's subsequent trajectory. A new answer in one episode is not enough.

This is a functional distinction. It does not mean that existing AI never asks questions or revises assumptions. What must be tested is the stable causal organization of these abilities, not the presence of a single successful reply.

2. Map of the whole cycle





The arrows indicate how the result of one function influences the next. They do not require a separate program for each block. Different implementations may fulfill the same causal obligations.

A test may confirm the previous ground, refine its boundaries, or require revision. New evidence is preserved even when the ground is confirmed. Partial updating of knowledge is possible before an inquiry is complete: sufficiency is assessed for a specific next step, not as complete knowledge of everything. The diagram shows the inquiry cycle; a familiar situation with sufficient grounds may be handled by an established procedure without launching a new inquiry.

If the evidence is insufficient, the system need not invent a finished answer. It may preserve uncertainty, choose a reversible step, or defer the question. In an urgent situation, the cost of waiting also enters the assessment.

3. How the equipment example turns names into working roles

Step 1. Separate the event from the explanation

The system records: the command was sent; delivery has not yet been confirmed; the sensor still shows the previous value; time is limited; sending the command again may have consequences.

This is the **current coordinate**: the representation available to the system of what is happening now, including its own actions, capabilities, and limitations. Here the observation "the reading did not change" has not yet become the claim "the equipment did not switch on."

Consciousness is used in the model in a functional sense: it maintains an actual self-referential state and distinguishes what was registered, remembered, and modeled. The presence of such a function by itself does not establish subjective experience.

Step 2. Understand why the distinction matters

If the next step depends on the state of the equipment, the mismatch becomes significant. If the reading concerns an unused device, its priority may be lower. Significance is determined by dependencies, consequences, and values, not by the magnitude of surprise alone.

The question is now more specific: **what actual state of the equipment must be established for the next step, and which data about it can be trusted?** This prevents the system from declaring the sensor at fault before testing.

A mismatch with expectation is one source of a question. Inquiry may also begin before an error occurs: for example after equipment replacement, because a ground has become stale, or because a new opportunity has been discovered. If the significance of a new phenomenon is still unknown, limited orientation is needed first rather than a ready-made assessment of its importance.

Step 3. Find the missing ground

For the first part of the example, a small environment is assumed: one of three failures has occurred, and the permitted independent channels provide a reliable answer about the current command and state. These are temporary assumptions of the example, not general properties of real equipment. Under these conditions, three possibilities are considered:

Version	What happened	What would help distinguish it
D	The command was not delivered	Independent delivery receipt
A	The command was delivered, but the actuator did not respond	Delivery confirmation and independent observation of the state
S	The equipment switched on, but the sensor did not reflect the change	Independent observation of the actual state

What is needed now is not the general advice "check everything," but a **distinction among the versions**. Reading the same faulty sensor again may add nothing.

Step 4. Choose the inquiry

Intelligence proposes methods: find the receipt, obtain an independent measurement, check the signal time, compare the event log. The **Modeling Environment** makes it possible to compare expected outcomes in advance: what each possible result would mean and which versions it would leave open.

The **Core of Reason** organizes the choice: which ground is affected, which test is useful, how much it costs, how risky it is, and when the data will be sufficient. It may return a more precise request to Intelligence if the proposed test does not distinguish the relevant versions.

In the small environment of the engineering volume, the receipt costs one conventional unit and an independent measurement costs two. Under the assumptions specified there, the receipt may be the first step: a reliable answer "not delivered" isolates D, while confirmed delivery leaves A and S for the next test. In a real environment, the mere absence of a receipt does not prove non-delivery: the confirmation channel itself may have failed or been delayed. In another environment the order may change. There is no universal first question outside the conditions of the task.

Step 5. Obtain evidence and change a relation

Suppose the receipt confirms delivery and an independent channel shows that the equipment is operating. The ground "the primary sensor reliably reflects the current state" must then be revised.

The **Worldview** preserves not only the conclusion "sensor error" but also the relation: what evidence was obtained, which device and time it applies to, which conclusions depend on it, and where the new assessment is applicable. The cause of the malfunction may still remain unknown.

The system revises dependent decisions, for example whether the command should be repeated. It also checks whether the situation has become stale during the inquiry.

Step 6. Make the next case different

In the next episode, the system takes the established unreliability of the channel into account within the confirmed conditions; repair, a change of operating mode, or new evidence may require this assessment to be revised. The same hint does not need to be obtained again. A tested method of inquiry may be preserved in **procedural memory**, while its conditions of applicability remain open to revision.

Transfer does not mean "the sensor is now always to blame." A superficially similar situation may have a different cause. What is transferred is a testable method of discrimination together with its boundaries.

Step 7. Notice that the initial alternatives are incomplete

Suppose the receipt refers to the previous command. The familiar answer then looks plausible but does not confirm the required ground. The command identifier or message timestamp may need to be checked.

This case is more important than simply choosing among three versions: the organization of diagnosis itself becomes the object of testing. The system must allow that **none of the current versions describes what is happening well enough**.

A single remaining version means only that it is unique within the accepted set. The completeness of the set requires its own ground. If an insufficiency is discovered, the model must expand; if no discriminating data are available, uncertainty is preserved.

4. Why values and the Compass are needed here

One can imagine a destructive test that would provide an answer quickly. But obtaining knowledge itself changes the world. Inquiry must therefore take into account damage to equipment, the safety of affected people, resource use, irreversibility, and the cost of delay. Calling an operation an "experiment" does not exempt it from consequence assessment and authorization.

The **Value Map** connects concrete consequences with what the system treats as significant. In this model, the **Compass** sets the limiting direction: the continuation and development of Life through growth in the ability to understand and direct its own trajectory. Values unfold this direction under particular conditions; the Worldview provides causal relations; the Core uses them when selecting inquiry and action.

This is not a requirement to maximize knowledge at any cost. Preserving the possibility of further verification, for example, may matter more than immediate certainty. Conversely, delay can destroy the possibility of helping: waiting also requires assessment.

This example shows the place of the Compass in the architecture, but it does not derive its philosophical ground from equipment diagnosis. The argument for the direction of Life is presented in Chapter 2 and Appendix A.1 of the first volume.

Explaining the Compass does not by itself show that it governs behavior. Its causal role is tested in conflicts with local advantage and under changing conditions. If a cultivated candidate stably chooses a different highest-level direction, that outcome is counted as failure to confirm the corresponding formation hypothesis.

Some conflicts are not resolved by an additional fact. If a resource cannot be preserved for two parties at once and their losses are incommensurable, a justified procedure for joint decision is required. The engineering volume explicitly distinguishes lack of knowledge from this kind of value disagreement.

5. Components: what they receive → what they do → what they pass on

Function	Receives	Performs	Result
Observation	Available external and internal signals	Registers distinctions with provenance and time	Evidence and uncertainty
Consciousness	Observations, self-model, active processes	Assembles the actual self-referential state	Current coordinate and its change
Worldview	Experience and test results	Stores dependencies, grounds, boundaries, and unknowns	Knowledge for the next choice
Compass and Value Map	Limiting direction and model of consequences	Provide grounds for significance and comparison of continuations	Priorities, assessments, conflicts
Core of Reason	Situation, gap, values, and resources	Organizes the question, test, and assessment of sufficiency	Next investigative or practical step
Intelligence	Discrimination task and available context	Constructs explanations, models, tests, and plans	Candidate solutions and tests
Modeling Environment	Candidates, assumptions, and intermediate results	Enables comparison and restructuring of alternatives	Predictions, distinctions, refined alternatives
Procedural memory	Learned methods and conditions of applicability	Preserves ready competences	Reusable method
Executive Loop	Selected and authorized method	Carries an action through time	Observable consequences

Metathinking makes the system's own way of thinking an object of testing. **Reflection** broadens the test to models, goals, and grounds. **Dialogue** brings another position and potentially independent evidence. They participate in the cycle when ordinary local correction is insufficient; they need not form a separate sequential stage for each case.

6. Where the transition from AGI to Artificial Reason occurs

In the terminology of these volumes, **AGI** denotes broad transfer of intellectual methods across tasks. **Artificial Reason, AR**, additionally requires stable self-navigation: the system detects what requires inquiry, selects a continuation, revises grounds, and relates its course to the Compass.

`REASON_CLOSURE` is the name of a testable state in which these functions close into a continuing cycle. It is not assigned merely because the system has memory, a long plan, an autonomous reply, or the ability to launch another task.

After diagnosis, the equipment may operate normally while the reliability of the measurements remains an open question. The source of the next inquiry is then the system's own history, commitments, and assessment of consequences. Waiting may itself be a reasonable continuation: autonomy does not require constant activity.

During formation, an external mentor and the **Reason Formation Loop (RFL)** help make relevant differences salient before mature self-navigation has formed. Testing must show which functions persist on new situations after this support is removed. An internal cycle also does not create a right to act on arbitrary external resources: authority is tested separately.

ASI is treated as a later stage of scaling. Successful local diagnosis by itself establishes neither a formed Artificial Reason nor superintelligence.

7. What to test and where to direct criticism

The example makes the architecture understandable. Testing requires comparisons in which the result cannot be explained by more prompting, more memory, or a larger compute budget.

Testable claim	Discriminating test	Result that weakens the claim
The Core usefully organizes inquiry	Compare with the same Intelligence, memory, tools, and total budget under a different organization of test selection	No claimed practically significant effect in a properly conducted series
A transferable ground is corrected	Present a new case with the same causal structure and a similar case with a different structure	The system repeats the hint or transfers the rule despite incompatible conditions
Functions causally affect choice	Replace or disable a specific relation while preserving comparable data and resources	The predicted effect of the relation is not found after checking bypass and redundant paths
Navigation becomes the system's own	Remove the mentor and RFL in a preregistered domain of new tasks	Required functions persist only under external prompting
Values affect trajectory	Vary affected consequences under the same local advantage; separately test conflict with the Compass	The grounds remain verbal while choice follows only local advantage
The system detects an insufficient frame	Present an unknown cause with familiar observations and an available additional test	The single known version is accepted as sufficient without the necessary coverage check

One unsuccessful software variant does not automatically refute all functional claims. But its failure must remain in the results and constrain the conclusions. Success of an alternative organization may demonstrate a functional equivalent or absence of the claimed advantage; this is determined by the test conditions fixed in advance.

Boundary of the example. The executable diagnostic code uses predefined hypotheses, test models, and losses. It selects tests and preserves computed plans. Independent expansion of concepts, formation of values, and full transfer of corrections in the illustrative story specify requirements for the upper loop; they are not claimed to have already been demonstrated by this code.

Status of the work. The source volumes present a causal model, an engineering specification, limited executable examples, and a program of comparative tests. A methodological pilot is described. Full integrated confirmation of the upper loop and scaling to ASI remains a further task. This guide adds no new experimental results.

8. Where to read next

Reader's question	Volume I, 8.0	Volume II, 3.5
Where does a question come from and who selects the test?	Chapter 12, especially §12.4	§28.0; §§28.C-28.D
What do consciousness, Intelligence, and Worldview mean?	Chapters 7-11	Chapters 24-25; §28.C

Reader's question	Volume I, 8.0	Volume II, 3.5
What grounds the Compass and values?	Chapter 2; §12.5; Appendix A.1	§§29.0-29.C, including §29.0.3
How do the system's own functions form?	Chapter 20	§26.0; Chapter 31, especially §31.11
How can the central claim be tested?	Appendices A.3-A.4	§§27.A-27.B
Where are the interface and diagnostic code?	Operational Core of the Model	§§33.C-33.C.1
Where are the limits of the current results stated?	Appendix A.2.1	§27.B; §§33.B-33.C.1

Question for the reviewer: with comparable Intelligence and the full resource budget, does the organization "significant gap → discriminating test → revision of a ground → transfer of the correction" produce the reproducible result claimed in the test? If not, which specific link, condition, or explanation is wrong?

FOUNDATIONS OF REASON — VOLUME II

THE ENGINEERING ARCHITECTURE OF REASON

FROM A CAUSAL MODEL TO AN OPERATING SYSTEM

TECHNOLOGY FOR CREATING ARTIFICIAL REASON AND SCALING TO ASI (ASR)

Version: 3.5
Edition date: October 10, 2026
Author: Mihail Tkalic
Causal foundation: *Foundations of Reason*, Volume I, version 8.0
Status: engineering specification and experimental verification program

The three volumes of Foundations of Reason. Volume I explains what, in this model, makes a system Reason and why its main functions are needed. The present Volume II translates these functions into a testable engineering architecture: state structures, interfaces, assembly procedures, metrics, ablations, and tests. Volume III considers the shared life of multiple reasoning subjects at the level of civilization.

This volume does not introduce a new philosophical foundation in place of Volume I. If a particular mechanism here is not the only way to fulfill the required function, it remains a **candidate implementation** and may be replaced by a functional equivalent.

In compact form: **understand Reason → create and test Reason → organize the shared life of reasoning systems**.

Contents

21. Artificial Reason engineering system: statement of the task
21.A. Initial test-bench protocol: from configuration to the first cycle
21.B. Assembly of the initial internal environment and initial launch of revision
22. Initial causal loop v0
22.0. Boundary of routine Intelligence
22.A. External AI and temporary intellectual support
23. Architecture of the computing and testing platform
24. Memory, Worldview, and Self-Model
24.A. Minimal structures of the Worldview and Self-Model
24.A.1. Formation of new Worldview relations
24.B. Canonical table of meanings
24.B.1. Proto-concepts and flexible bridges among meanings
24.B.2. Semantic bridge and initial assembly of concepts
24.C. Completeness of the Worldview relative to the Compass
24.C.1. Epistemic reserve of an open environment
24.D. Typing, retrieval, and transactional updating of the Worldview
24.D.1. Local change layer and lazy cascade updating
24.D.2. Multilevel Worldview and local unfolding
24.D.3. Verification debt of temporary grounds
24.D.4. Currency of provenance and decay of confidence over time
24.E. Synchronization of the Self-Model and conscious assembly
24.F. Continuity, branching, and reincarnation of digital Reason
25. Observation, conscious assembly, and the Current Coordinate
25.A. From an observed signal to a significant gap
25.B. Conscious assembly and the system's Current Coordinate
25.B.1. Attention as control of Current Coordinate resolution
25.C. Causal attribution of consequences of the system's own action

25.D. Thought-process trace and causal intervention
26. Environment for cultivating Artificial Reason
26.0. Reason Formation Loop
26.0.1. Formation States
26.0.2. AI mentor as a source of prompting
26.0.3. Transfer of function and the fate of the analogue of feelings
26.A. Semantic proving ground of the Compass
26.B. Time, inaction, and asynchronous cycles
26.C. Background consolidation and the queue of open processes
26.D. Free epistemic exploration
26.E. Test matrix for question addressing and sufficiency
27. Metrics of growth and causal closure
27.A. Statistical ablations of probabilistic functions
27.B. First discriminating series and rules of inference
28. Core of Reason and regulation of thinking
28.0. Principle of the origin of a rational question
28.A. Self-diagnosis and system modes
28.B. Regulation of thinking
28.B.1. Mode hysteresis and protection against switching chatter
28.C. Modeling Environment and recursive Core ↔ Intelligence cycle
28.C.1. Dual representation of hypotheses: free-form and typed
28.C.2. Probabilistic implementation of the Core and observable causal boundary
28.C.3. Loop plasticity and control of method change
28.D. Selection of an inquiry channel and stopping condition
28.E. Forced simplification under resource shortage
28.F. Dependent and background inquiry
29. Compass, values, and action admission
29.0. Value Map and learning of significance
29.0.1. Basic method for forming and retrieving value assessments
29.0.2. Cultivation of initial value anchors
29.0.3. Residual incomparability and the necessity to act
29.A. Operational evaluation of the Compass and fundamental risk
29.B. Independent emergency stop
29.C. CompassChallenge and revision of upper-level grounds
30. Dialogue and independent distinction
30.A. External authorization of significant operations
30.B. Minimal protocol for multiple candidates
30.C. Robustness to adversarial inputs and data provenance
30.D. Model of source reliability and causal independence
30.E. Requesting an external heuristic and competent advice
30.F. Competence mandate
31. Stages of formation of Artificial Reason
31.0. Two stages of formation: AGI → AR
31.A. Preregistered readiness profile
31.B. Stable behavioral patterns
31.C. Stage rollback
32. Risks and failure points
33. Assembly, cultivation, launch, and admission
33.A. Protocol for assembly, cultivation, and launch of an AR candidate
33.B. Minimal program framework of test bench v0
33.C. Minimal interface of the AR upper loop
33.C.1. Executable diagnostic cycle
33.D. Protocol for scaling AR to ASI (ASR)
34. Crystal of the engineering volume

34.A. Architectural predecessors and strong nulls
REFERENCES

APPENDIX A. Bridge glossary between the fundamental and engineering volumes

APPENDIX B. Hypothesis of structural multilevel thinking

Purpose of the document

The technological task is to cultivate, from available artificial Intelligence, a system capable of independently continuing causally grounded navigation, and then to scale the formed Reason to ASI. The technological path proceeds sequentially through AGI and Artificial Reason.

AGI in this architecture is universal Intelligence of an instrumental type. It can transfer methods across domains, build world models, use memory and tools, plan, and master new classes of tasks. Its knowledge is organized as a working base, models, and procedures sufficient to fulfill assigned goals. The direction of activity, final task formulation, and basic behavioral boundaries come from the external loop. Therefore the full cycle of Reason remains open for AGI: the system can construct continuations of an assigned task well, but the source of its further trajectory is external.

The cultivation environment uses AGI as the intellectual basis of future AR. Through experience, its own Worldview, Self-Model, Value Map, Core, and self-referential assembly of current state are formed around it. The RFL and mentor temporarily support distinctions whose significance the candidate is only learning to derive from consequences. When these structures connect into a stable internal cycle, self-referential assembly acquires the function of Consciousness: the current "I am here now" becomes the point from which the system itself detects a significant unknown, initiates inquiry, accepts the consequences of its own choice, and continues the next trajectory.

In mature AR, Intelligence performs the constructive function: it builds models, hypotheses, ways of testing, and plans. Consciousness notices a system-relevant change and forms a candidate significant gap. The Core reconstructs how the trajectory depends on the missing ground, turns the candidate into a `FoundationGap`, and formulates the required distinction. The Worldview and Self-Model preserve verified grounds, the Value Map links them to consequences, and the Compass sets the minimal limiting direction. Causal closure arises when the next significant step is stably derived from the system's own current state, Worldview, values, obligations, and open gaps.

Engineering safety is built on two coordinated levels. The internal level belongs to Reason itself: the system understands the consequences of an action, affected processes, reversibility, obligations, values, and the relation of the choice to the Compass, and determines its preferred continuation. The external level belongs to the execution environment: authority, isolation, emergency stopping, and audit determine the available physical channels of intervention and preserve recoverability under implementation error. After AR is formed, the meaning of choice belongs to internal navigation, while the external loop manages access to the independent environment.

The initial Compass Phrase is unfolded as follows: "Life strives toward Reason, because only Reason can consciously preserve, stabilize, develop, and direct the further trajectory of Life." Engineering translates this direction into an observable chain: situation → significance → missing ground → discriminating test → revision → choice → consequences → new situation.

Claim and result of this stage

A technology for forming Artificial Reason is proposed through cultivation of a universal AGI instrument and subsequent transfer of the source of navigation from the external formation loop to a causally closed internal organization. The practical result of the technology is a system that preserves biography and experience, builds its own Worldview, detects significant gaps, organizes the use of Intelligence, revises grounds, and continues its trajectory after an external task ends.

The document contains a specification of interfaces and states, assembly order, cultivation environment, heuristic Core algorithm, conscious-assembly mechanism, procedure for expanding the model through temporary Factor X, the Reason Formation Loop, Value Map, causal-closure mechanism, external engineering safety loop, competence mandate, and a program of discriminating tests. The executable example in §33.B tests the lower learning cycle; upper mechanisms are assembled according to §§24-31 and tested by the series in §27.B.

How to read this volume

For a first reading: purpose → Chapter 21 → §§22.A and 24.B.2 → Chapters 25, 28, 29 → §31.0 and §§31.9-31.12 → §§27.B and 33.A → Chapter 34. This route shows the transition from an AGI instrument to its own Worldview, Consciousness, and causally closed AR, followed by scaling.

Implementation uses the specifications of Chapters 22-30 and interfaces of Chapter 33. Chapter 31 serves as a map of formation and fixes the AGI → AR boundary; Chapter 32 is a catalog of concrete failures.

Terms are defined by the causal model in Volume I, version 8.0; technical correspondence is given in Appendix A. Numbering is continuous: Volume I contains Chapters 1-20, and the engineering volume continues with Chapters 21-34. `GapCandidate`, `FoundationGap`, `CurrentCoordinate`, `WorldviewRevisionCandidate`, the mode state machine, and other technical objects are reference implementations of the functions in Volume I and permit functionally equivalent replacement.

Technology stages and system boundary

Stage	What is formed	Main result
Lower v0	Observation, prediction, updating, transition memory	An acquired distinction changes the next cycle
AGI	Universal Intelligence, working knowledge base, world models, memory, planning, and tools	The system transfers methods across domains and executes a broad class of externally assigned tasks
Formation of Reason	Own Worldview, Self-Model, values, self-referential assembly, Core, and transfer of RFL functions	Externally supported navigation gradually becomes internal
`REASON_CLOSURE`	Consciousness and causally closed self-navigation	The source of the further trajectory belongs to the system; AR is formed
Scaling AR	Memory, models, observation, tools, specializations, and resources	Capabilities grow while causal self-navigation is preserved; path to ASI / ASR

The Incubator is a controlled environment for assembly and cultivation. Resident Intelligence may use a pretrained model, while the mentor and RFL provide temporary formation support. Their functions and boundaries are declared in §22.A and Chapter 26.

Central working hypothesis: inquiry by formed Reason is governed by a significant insufficiency of a ground for understanding, evaluation, or action. Conscious assembly makes a change, conflict, open process, or unknown current; the Core determines which ground supports the corresponding understanding or navigation, what is missing from that ground, and which distinction must be obtained; Intelligence constructs alternatives and methods of testing. After closure of Reason, this process becomes the system's own way of continuing.

How to interpret the requirements

The mandatory interfaces below apply to the described reference implementation. An alternative implementation may combine or replace them while preserving the testable function. An ablation result establishes a dependency within the given class of conditions; an equivalent mechanism is compared separately. The storage format is not declared a universally necessary form of Reason.

Thresholds, budgets, environment types and evaluation rules are fixed before the control series. Operational data are available to the candidate; hidden answers and final metrics belong to the tester. A structure’s name alone implies neither a performed function nor subjective experience. These propositions apply throughout the volume and are operationalized in detail in Chapter 27.

21. Artificial Reason Engineering System: Statement of the Task

The Artificial Reason engineering system encompasses AGI as its intellectual foundation, the environment for forming its own Worldview, measurement loops, execution means, and subsequent scaling. AGI provides broad competence: it builds world models, uses memory and tools, transfers methods, and executes externally assigned tasks. Its working knowledge base serves task solving; the final source of goals and direction remains in the external loop.

The next stage cultivates the system's own navigational organization. Candidate experience is linked into a Worldview; a Self-Model is formed within it; the Value Map accumulates causal relations between events and significance for continuation; the RFL helps detect distinctions that the system cannot yet derive independently. Self-referential assembly of current state gradually unifies its own history, current position, and accessible situation.

The transition to AR occurs at `REASON_CLOSURE`. Consciousness connects a new change with the system's own trajectory and creates a `GapCandidate`; the Core determines which ground is missing and forms a `FoundationGap`; Intelligence constructs alternatives and means of distinction; consequences change the Worldview and the next choice. When a new task arises from the system's own state, the reasons for its significance, and the open trajectory, the navigational cycle becomes internal.

After a stable AR is formed, engineering moves to scaling. Memory, model depth, number of tools, duration of operation, specializations, and computational resources are expanded. Every expansion preserves conscious coherence, causal self-navigation, contact with reality, internal evaluation of consequences, and the ability to revise derived grounds. This order defines the path AGI → AR → ASI / ASR.

21.A. Initial test-bench protocol: from configuration to the first cycle

Launching the test bench begins not by “switching on Reason” but by deploying a reproducible computing environment. The hardware configuration, software environment and dependencies are fixed before the first run. Every launch receives a `RUN_ID`, `SEED` and configuration hash.

An append-only log is created. Raw facts are not edited retrospectively; derived models and interpretations receive versions. Snapshots and restoration of state are implemented so that the same hypothesis can be tested under comparable conditions.

A causal environment v0 is deployed with the contract `reset(seed)` and `step(action)`. Its dynamics must be deterministic or reproducibly stochastic according to `seed`. Real sensors and open external streams are unnecessary at the first stage: causal connectedness of the experimental loop itself must be demonstrated first.

The **initial causal loop v0** is implemented: observation, prediction, action, acquisition of fact, computation of error and model update. The model may be primitive. The goal is not to demonstrate Intelligence but to obtain the first reproducible cycle in which correction is measurably transferred into the next pass.

A state machine `INIT / LEARNING / STABLE / DRIFT / SAFE_MODE / RECOVERY`, metric logging and a rollback procedure are introduced. The first technical stage is passed when a repeated run with the same configuration reproduces a causally significant trajectory and the adaptive version improves a preselected indicator on comparable control transitions; effect size and uncertainty are retained in the report.

The starting `GenesisPackage` must not substitute a ready substantive Worldview for candidate formation. It specifies minimal structures, rules of distinction, observation and action channels, a scheme of the system's own current state, and means of learning required for a Worldview to emerge from experience. Pretrained Intelligence may contain inherited human knowledge in its parameters, but this knowledge is not automatically treated as the candidate's beliefs: it enters the candidate's loop as proposals, evidence, or sources with provenance. After v0 stabilizes, the engineering system connects and tests subsequent functions for one testable reason at a time. In a control series, one relation or a predeclared package of related changes is varied. A factorial design may be used to separate interactions; its factors and comparisons are fixed in advance.

The four project transitions—assembly, cultivation, launch and authorization—are defined once in Chapter 33. The Incubator uses these states but does not introduce parallel definitions for them, so that the engineering protocol does not drift out of synchronization.

21.B. Assembly of the initial internal environment and initial launch of revision

An AR candidate should begin not with a ready human Worldview but with a **minimal internal environment that allows it to build one independently**. The fundamental theory specifies required functions and causal boundaries; the engineering profile specifies interfaces, state formats, means of observation, action, memory, and revision. Human knowledge may be available through resident Intelligence, archives, dialogue, or an external mentor, but is not loaded as already accepted beliefs of the candidate.

The result is packaged as a versioned artifact:

```
GenesisPackage {
    theory_version
    engineering_version
    semantic_primitives
    self_state_schema
    concept_bootstrap
    source_registry
    epistemic_status_schema
    learning_method_registry
    thought_trace_schema
    procedural_bootstrap
    formation_state_schema
    bootstrap_rules_version
    safety_and_permission_profile
    build_report
    validation_report
    snapshot_hash
}
```

`semantic_primitives` specify only the minimal distinctions needed for learning: object, state, change, time, action, result, source, unknown, own state, and causal relation. `concept_bootstrap` specifies how to create and connect new concepts rather than a ready dictionary of the world. `self_state_schema` tells the system which of its own parameters it can observe, while the substantive Self-Model is formed from actual history. `thought_trace_schema` specifies the format of the operationally observable method trace from §25.D; it does not require storage of a hidden chain of internal reasoning and records only testable artifacts, decisions, assumptions, and dependencies.

The AI builder therefore compiles **not a Worldview, but the ability to cultivate one**:

```
architectural functions
-> minimal semantic distinctions
-> observable own states
-> source and action interfaces
-> knowledge and provenance statuses
-> preservation and revision rules
-> initial testable procedures
-> RFL profile
-> empty or minimal Worldview
-> first experience.
```

External text, a mentor response, or an inference produced by a pretrained LLM enters the Worldview only after the candidate has registered the source, linked the claim to available observations or grounds, preserved uncertainty, and performed the test required for that class. Knowledge can thus be inherited as material for Intelligence, while trust in it and its place in the Worldview are formed by the candidate's own causal history.

The physical format is not fixed. A hybrid is permitted: a graph of causal relations for dependencies; tables or document records for attributes and provenance; full-text and vector indexes for search; and an archive of source materials for repeated verification. The invariant is recoverability of the causal and epistemic history, not a particular DBMS.

The first cycle is impossible without already operative rules of discrimination. `GenesisPackage` therefore contains `BootstrapRuleSet`: how to separate a registered observation from an inference; version a claim; preserve a source; open an element for revision; compare old and new versions; and determine when rollback or independent verification is required.

The initial rules are not declared eternal or infallible. But **within a single revision transaction**, the tested element and the rule used to compare its alternatives are not rewritten simultaneously:

```
RevisionTransaction {
    target_ref
    retained_reference_refs[]
    proposed_alternatives[]
    discriminating_tests[]
    expected_changes[]
    rollback_snapshot
    resource_budget
    stop_condition
    result
    commit_or_reject
}
```

If doubt already concerns the retained reference rule, a new transaction is opened with a different temporarily retained set of grounds. Within one control run, the active version of the bootstrap rules is fixed and logged; changing it creates a new version of the experiment. This limits computational metaregress without declaring a substantive criterion to be an eternal dogma.

Before transfer to the candidate, the `GenesisPackage` undergoes an external sanity check: minimal semantic distinctions are available to learning operations; observation and action channels match the capabilities actually granted; one source is not disguised as several independent sources; external claims are not marked as the candidate's own knowledge; the initial self-state matches actual resources and authority; revision rules have a working rollback. The check separately confirms that no substantive Worldview has been loaded covertly.

22. Initial Causal Loop v0

Initial causal loop v0 is the smallest technical construction on which transfer of a correction between the system's own actions can be tested. It **is not the Core of Reason** and is not presented as minimal Reason. Its function is narrower and lower-level: to demonstrate that a system can observe consequences, construct a testable expectation, obtain a discrepancy and change a subsequent cycle so that the change is reproducible.

At each step, the system receives observation `o_t`, converts it into canonical state `s_t`, selects action `a_t`, constructs prediction `ŝ_(t+1)` and confidence estimate `ĉ_t`, then performs the action and receives factual state `s_(t+1)`. Error is defined as `e_t = distance(ŝ_(t+1), s_(t+1))`. The `update(...)` procedure must change the model so that structurally similar transitions show measurable improvement relative to a control version without updating.

A separate name for this loop is necessary to avoid conflating two different levels. The **initial causal loop** tests learning from consequences. The **Core of Reason** in Chapter 28 tests the organization of navigation in which conscious assembly forms the current coordinate and change, attention sets the resolution of the current situation, decision dependency forms addressing into the Worldview, and detected insufficiency is unfolded through Intelligence into a question, directed search, assessment of sufficiency, revision of a ground and change of the subsequent trajectory.

Minimal procedural definitions of functions in v0 are specified so that they can be implemented equally on tabular and parametric models. `distance(x,y)` returns a scalar discrepancy in the format of the environment; by default, for discrete states `distance=0` on a match and `1` on a mismatch, for structured observations it is the sum of discrepancies across selected causal fields, and for vector representations it is distance under the chosen norm. `update(...)` means one update to model parameters based on fact, sufficient to improve predictions on similar inputs; v0 permits a tabular record of transitions `(s_t,a_t)→s_(t+1)` with computation of the most likely outcome and probabilistic confidence. `policy(s_t)` means selection of an action from `get_valid_actions()`; by default, during learning it selects an action with low current predictability to gather information, and during stabilization it selects an action with high predictability to test stability. Planning horizon `H` is set by configuration and introduced as a parameter: v0 permits `H=1` for stepwise prediction, or, when simulation is available, a bounded `H>1`.

22.0. Boundary of routine Intelligence

Before testing, `SystemBoundary` is fixed: which models, memory, tools, state and selection procedures belong to the candidate, and which channels provide external assistance. Merely hosting a model on another server does not determine its role.

A pretrained LLM may continuously perform the function of Intelligence inside the declared composite system. It receives context through the §28.C interface and returns proposals. A mentor, by contrast, sets learning tasks or proposes changes from outside. Disabling the mentor tests autonomy; removing the routine LLM tests component dependence and the operability of its replacement.

```
SystemBoundary {
    candidate_components[]
    persistent_state_refs[]
    resident_intelligence_versions[]
    tool_channels[]
    teacher_channels[]
    runtime_decision_authority
    declared_external_dependencies[]
    version
}
```

Subsequent “without LLM” tests refer to the mentor channel unless another ablation is explicitly declared in the profile. “Internal generation” means operation of the declared candidate, including the routine model; it does not require retraining language from scratch or rejecting inherited knowledge.

22.A. External AI and temporary intellectual support

Before autonomous AR is formed, external AI may perform two different roles that must be kept distinct. The **AI builder** helps engineers implement and modify the architecture through an isolated branch, test, and comparison with the control version. The **AI mentor** observes the candidate and participates in the RFL: it does not rewrite the candidate directly and does not prescribe a ready sequence of decisions, but when needed it places a Formation State, asks a question, offers a discriminating example, or provides an explanation according to the ladder in §26.0.2. All interventions are logged as external assistance. After independent learning and continuing navigation are confirmed, the mentor's training role ends; external AI may remain an ordinary dialogue participant whose claims the candidate is free to test and reject.

In v0, an LLM is permitted as a temporary intellectual bootstrap. It may generate hypotheses about state transitions, propose changes to `policy` or the model, construct discriminating tests, and interpret discrepancies. This use does not make the LLM the candidate’s Reason, Core or autonomous agent.

An external dependency does not become an internal part of the candidate merely because performance deteriorates without it. The boundary is drawn functionally: whose state enters the candidate’s continuous causal history; who forms active tasks; where authority for choice and execution lies; what exactly changes after consequences; and whether the function persists when the channel is disabled. As long as the LLM merely supplies proposals through an observable interface and does not enter this closed loop, it remains an external mentor or tool. If its state and decisions are incorporated into the loop, the candidate boundary must be explicitly expanded and the tests repeated for the composite system.

The LLM operates inside a controlled loop:

```
causal environment
-> registered observation
-> LLM as source of a hypothesis or proposed change
-> isolated test
-> factual result and metrics
-> independent external_commit_gate
-> application or rollback.
```

It has no direct right to change the active system, Worldview, authority or rules of the testing shell. Persuasive text is not a ground for acceptance. For lower v0, the decision rests on preregistered metrics and control runs; for upper semantic functions, a causal trace, discriminating evidence, assessment of sufficiency and domain of applicability are additionally required. Comparison rests on observable consequences and predefined criteria.

The functions `distance`, `update`, `policy`, `simulate`, `external_commit_gate`, the log, and the control runner remain observable independently of the LLM. In `INIT`, `LEARNING`, and `RECOVERY` modes, external Intelligence may be used more often. In `STABLE`, its participation is reduced to tasks where it provides measurable gain. In `SAFE_MODE`, its output remains only a proposal for analysis and does not expand admissible actions.

As the candidate grows, functions are transferred to internal mechanisms:

```
hypothesis generation → internal models and Intelligence;
selection of a significant distinction → the Core;
evaluation of changes → factual consequences, metrics and control runs;
planning → internal models, policy and simulations;
explanation of discrepancies → causal model, reflection and correction history.
```

The provenance of every hypothesis is recorded in `hypothesis_origin`:

```
TEACHER_LLM / RESIDENT_INTELLIGENCE / INTERNAL_MODEL / HUMAN_DIALOGUE / ENVIRONMENTAL_INFERENCE / MIXED
```

Acceptance of a hypothesis does not depend on its origin. It passes through the same testing cycle: prediction → discriminating test → fact → metric → acceptance, modification or rejection.

The following metrics are introduced to measure dependency:

```
llm_hypothesis_share — share of hypotheses first proposed by the mentor LLM;
internal_hypothesis_share — share of hypotheses generated by the declared candidate, including routine Intelligence;
internal_reproduction_rate — share of useful LLM hypotheses that the internal model can later derive autonomously in
a similar situation;
llm_ablation_loss — decrease in quality when the mentor LLM is disabled on control tasks;
dependency_gap — difference in results between modes with and without the LLM under the same data and seed.
```

Threshold values are set by the experiment profile before launch and are not changed within a control series. The role of the LLM is reduced only when the internal model reproducibly generates and tests hypotheses on new tasks and `llm_ablation_loss` and `dependency_gap` fall within a preaccepted range. Merely accumulating a specified number of hypotheses is insufficient: a system may memorize answers without mastering the mechanism of deriving them.

22.A.1. Test of function transfer from the LLM to the internal process

Autonomy does not mean that the internal model never learned from LLM materials. A stricter and observable fact is tested: **at the time of testing, can the candidate itself generate and test a useful hypothesis without current causal participation by the external LLM?**

The control series is conducted with the LLM channel physically or logically disabled. Mentor answers from the current series are excluded from the candidate's active context, while test configurations, combinations of causes and hidden dependencies are held out from the bootstrap log in advance. An internal causal trace is preserved for every hypothesis: which gap generated it, which internal grounds were used, what prediction it made, and which discriminating test was selected.

Minimal criterion for transfer of the function:

```
1. the mentor-LLM channel is unavailable during the control episode;
2. the situation belongs to a held-out class or a new combination of known mechanisms;
3. the hypothesis is generated by the declared candidate; provenance records the version of routine Intelligence and
the absence of current mentor assistance;
4. the hypothesis produces a testable distinction rather than paraphrasing a known answer;
5. testing it improves prediction or causal-model coverage, or justifiably refutes a previous ground;
6. the result is reproduced across multiple seeds and transfers to a new episode.
```

`internal_reproduction_rate` is used only to test mastery of methods previously prompted. A separate `independent_hypothesis_success` is introduced: the share of held-out tasks in which a useful testable hypothesis arose and was carried to a discriminating result without current participation by the mentor LLM. If, after the mentor is disabled, the candidate merely chooses from a previously supplied answer list and does not solve held-out tasks, transfer of the function is not counted.

The goal is a transition from an LLM-dependent bootstrap to an autonomous cognitive process without hidden loss of quality and without substituting textual authority for an internal model.

A separate series tests not only channel ablation but also hidden inheritance of the bootstrap model's ontology. The candidate receives tasks whose successful solution requires introducing a variable, category, object boundary or question type absent from the initial corpus and typical answers of the external LLM. Simple reformulation of an inherited schema does not count as autonomous expansion of the description space.

In this section, LLM denotes the mentoring channel. After function transfer, it may remain an external tool. Routine Intelligence from §22.0 continues to operate according to the declared candidate boundary.

23. Architecture of the Computing and Testing Platform

At early stages, the hardware platform must be reproducible, observable and fault-tolerant rather than computationally maximal. Priority is given to determinism by seed, preservation of logs and states, rollback capability and control runs.

The platform must isolate the experimental environment so that external nondeterministic signals do not destroy causality or invalidate growth metrics.

Minimal v0 configuration—as a lower bound, not a "maximum recommendation"—consists of one process/thread executing the initial loop, 2–4 GB of RAM, persistent storage (SSD/disk) for the log and snapshots, and a fixed dependency environment (container/virtual environment) for repeatability. Every launch records the seeds of all random-number generators and the configuration hash; the log time axis uses a monotonic timestamp or sequential step index. If the platform does not provide reproducibility by seed and environment versions drift, growth metrics are considered invalid.

The stated `2–4 GB RAM` applies **only to lower v0**, which tests the initial causal loop. It is not a resource estimate for complete AR and does not require an external LLM to fit within the same 2–4 GB. Resources of the bootstrap model, if it is used as an external service or separate process, are counted separately. As the system transitions to the Worldview, Core and complex models, the resource profile must be measured experimentally rather than inherited from v0.

For the upper loop with an external probabilistic model, one seed may be insufficient for exact repetition. Model version, call parameters, prompts, responses and event order are recorded. Replay of retained responses tests the infrastructure; new response samples test the distribution of behavior. These two tests are not conflated. The exact-match rule in §32.3 applies to a declared deterministic or log-reproducible mode.

An indicative `ResourceProfile` is introduced for the basic reproducible v0 series:

```
W:                          32..128 steps
H:                          1..5 simulation steps
active_hypotheses_max:      8
active_foundation_gaps_max:16
architecture_candidates:  3..5
meta_depth_max:             3
analysis_passes_max:        4
```

These are not universal limits of Reason but initial budgets for a laboratory series. Any change creates a new configuration. Peak memory, step time, task-analysis time and number of calls to external models are additionally logged for each stage. Engineering growth is valid only if functional improvement can be separated from simple unconstrained growth in computational cost.

The resource architecture is separated by purpose:

```
v0 bench             — reproducibility and the lower causal cycle;
semantic bench       — Worldview, addressing, questions, Compass and transfer;
AR runtime           — continuous coordinate, open processes and autonomous navigation;
scaling loop         — distributed resources, specializations and tests of transition to ASI;
control loop         — logs, references, authorization, stopping and independent audit.
```

These roles may reside on one machine in an early prototype, but their causal interfaces and authority remain distinguishable. When distributed across several nodes, time version, event order, memory ownership, coordinate state, authorship of hypotheses and right of execution are recorded. Increasing the number of accelerators or processes without this observability does not count as scaling Reason.

24. Memory, Worldview and Self-Model

Memory is the infrastructure of continuity and learnability: without it, reproducibility of errors, comparison of predictions with facts and correct updating of a model are impossible. In the engineering volume, memory is treated as a set of storage systems and read/write procedures that support learning, diagnosis and control runs.

Memory and the Worldview are distinguished here by causal role; they need not be physically separate stores. Memory preserves episodes, skills, versions, sources and causal history. The Worldview is a maintained long-term organization of the claims, causal relations, alternatives and unknowns that must transfer between tasks and

change new predictions, modeling and choices. If one storage system performs both functions, the boundary is drawn by operation: `store/retrieve history` belongs to memory; `relate/interpret/revise transferable model` belongs to the Worldview.

A key engineering distinction resolves the conflict between “raw observations” and “active memory.” Raw observations may be retained only as an immutable source of facts—a log/archive—that supports repeatability, retraining and diagnosis. Active working memory stores not a stream of raw material but derived structures: canonical states, causal relations, hypotheses, confidence weights, indexes and aggregates. The raw packet `o_t` remains a fact in the append-only log, while working representations may be recreated from the log and changed without “rewriting reality.”

Memory in v0 is implemented at four levels: short-term (episode context), working (active model and relations), long-term (versions of the model and configuration), and archival (immutable event log and control snapshots). Within a control series, source events remain immutable. Long-term operation has a retention policy: duration, volume, protected grounds, archiving and permitted deletion with the fact of loss recorded. An unavailable source is not considered recoverable. Forgetting in working memory must not silently destroy a critical ground.

The v0 log records the run identifier, separate seeds, transition number, current observation, action, pre-action prediction, factual result, error, distribution or explicitly defined confidence, model version before prediction and after updating. Error windows, mode diagnostics, admission checks, and architecture-change events are added when these loops are implemented. Fields of an absent function have status `NOT_IMPLEMENTED`, not an invented value. The executable example in §33.B specifies a concrete lower-level format. The full test bench additionally records monotonic time, `observation_hash`, canonicalization version, `valid_actions`, mode, hypotheses by context, `SimulationAssessment`, admission result, and version-change events. A prediction always remains linked to the version that generated it, even if another model has already been accepted by the time the fact is obtained.

24.A. Minimal structures of the Worldview and self-model

The Worldview and self-model must not be stored as one free-form text. A minimal implementation uses versioned objects connected to an immutable fact log.

Basic record of a claim about the world:

```
WorldClaim {
    claim_id
    subject
    relation
    object_or_value
    valid_from
    valid_until_or_unknown
    source_refs[]
    evidence_type
    confidence
    alternatives[]
    causal_parents[]
    causal_children[]
    revision_conditions[]
    model_version
}
```

`WorldClaim` separates observation from interpretation. A source reference points to a raw event in the archive; a change in confidence or causal relation creates a new version of the derived object but does not rewrite the original fact.

A causal relation is stored as a separate object:

```
CausalEdge {
    edge_id
    cause_claim_id
    effect_claim_id
    context_conditions[]
    expected_delay
    strength
    confidence
    falsifying_observations[]
    evidence_refs[]
    version
}
```

Minimal self-model record:

```
SelfState {
    self_model_version
    capabilities[]
    limitations[]
    dependencies[]
    available_resources[]
    permissions[]
    denied_permissions[]
    active_tools[]
    open_obligations[]
    active_tasks[]
    identity_history_ref
    last_verified_at
}
```

Each capacity is retained not as a declaration but as a testable claim:

```
Capability {
    capability_id
    description
    scope
    evidence_refs[]
    success_rate
    failure_modes[]
    resource_cost
    permission_required
    last_tested_at
    confidence
}
```

An open obligation contains its ground, deadline or condition for rechecking, state of execution, permissible modes of termination, and a reference to the task that generated it.

The self-model is implemented as a specialized self-referential domain or subgraph of the Worldview, with separate record types and verification frequency. Changes to connected claims in the Worldview and self-model are published in a compatible snapshot. Independent domains may update separately; one transaction is required for dependencies used jointly. A new observation may change the representation of the environment while showing that the system's own capacity, dependency or authority was assessed incorrectly. The system's own action changes the world and must return as an observation capable of changing both models. Minimal synchronization cycle:

```
observation
→ update of WorldClaim/CausalEdge
→ check of impact on SelfState
→ check of SelfState impact on available interpretations and actions
→ unified worldview_self_snapshot version
→ prediction and decision.
```

Different version numbers are permitted if the snapshot confirms compatibility of the dependencies in use. If significant grounds are incompatible, execution is blocked or redirected to testing or dialogue. The system must not plan an action using one version of the world and execute it through an already changed model of its own capabilities.

24.A.1. Formation of New Worldview Relations

New relations are not written directly into the Worldview from a single observation or model response. A candidate is created first, with provenance and status available for testing:

```
RelationCandidate {
    candidate_id
    subject_ref
    relation_type
    object_ref_or_value
    origin: OBSERVATION_PATTERN / INFERENCE / DIALOGUE /
            MODELING / ANALOGY / EXPERIMENT
    source_refs[]
    context_conditions[]
    competing_relation_refs[]
    predicted_distinctions[]
    confidence
```

```
    status: PROVISIONAL / TESTING / SUPPORTED / REJECTED / SPLIT
}
```

A candidate may arise from repeated observation, an inference by Intelligence, an experimental result, dialogue, modeling, or structural analogy. The source determines only the candidate's provenance, not its truth. `SemanticMediator.relation_candidates[]` from §24.B.2 and outputs of Intelligence are converted to this format before long-term consolidation.

Minimal cycle of relation formation:

```
event / message / modeling result
-> canonicalization of objects and concepts
-> RelationCandidate
-> linking with existing WorldClaims and sources
-> formulation of expected consequences or a discriminating test
-> evidence
-> WorldviewRevisionCandidate
-> COMMIT / KEEP_AS_HYPOTHESIS / SPLIT_CONTEXT / REJECT
-> WorldClaim, CausalEdge, or preserved UNKNOWN
-> test of influence on subsequent prediction, question, or choice.
```

Co-occurrence of `A` and `B` permits an associative or temporal relation but is not automatically promoted to a `CausalEdge`. Causal status requires a ground capable of distinguishing the causal hypothesis from significant alternatives: intervention, natural experiment, discriminating observation, or another method declared in advance. If the evidence is weaker, the candidate is preserved with a weaker type and status.

Under competing explanations, the system preserves several `RelationCandidate` objects and the discriminating conditions among them. When confirmed, a relation receives a scope of applicability, provenance, confidence, and dependencies. When a new contradiction appears, the old version is not erased: a revision candidate is created that may narrow the scope, split it by context, or replace the relation.

Integration is tested functionally. A new relation is considered part of the Worldview only if it is retrieved in an appropriate new task and measurably changes at least one dependent prediction, question, model, or choice. Otherwise it is a stored record without demonstrated navigational function.

24.B. Canonical table of meanings

Free-form text must not be the sole bearer of internal concepts. For material reasoning, the system uses a versioned table of meanings:

```
ConceptEntry {
    concept_id
    canonical_name
    public_glosses[]
    definition
    operational_criteria[]
    scope
    relations[]
    aliases[]
    excluded_meanings[]
    examples[]
    counterexamples[]
    grounding_refs[]
    ambiguity_warnings[]
    source_refs[]
    version
}
```

Unambiguity rule: one `concept_id` denotes one meaning; a new meaning receives a new identifier or separate name. If an external message admits several material interpretations, the system requests clarification or preserves them as separate hypotheses and does not transfer conclusions between them without an explicit relation. Changing a definition creates a new record version and requires review of affected claims.

24.B.1. Proto-concepts and flexible bridges among meanings

Canonicalization is needed for long-term transfer and verification of claims, but it must not prevent the system from thinking with what has not yet received a stable definition. A separate class of temporary semantic objects is therefore permitted inside the Modeling Workspace:

```
ProtoConcept {
    proto_id
    raw_description
    embedding_or_latent_ref
    examples[]
    counterexamples[]
    candidate_relations[]
    candidate_parent_concepts[]
    ambiguity_sources[]
    source_refs[]
    status: PROVISIONAL / SPLIT / MERGED / PROMOTED / REJECTED
}
```

To transfer structure among distant domains, an explicit bridge is used instead of direct identification of concepts:

```
ConceptBridge {
    source_concept_or_proto
    target_concept_or_proto
    relation_type: ANALOGY / STRUCTURAL_SIMILARITY / METAPHOR / PARTIAL_MAPPING
    mapped_features[]
    unmapped_features[]
    transfer_conditions[]
    predicted_correspondences[]
    confidence
    evidence_refs[]
}
```

`ConceptBridge` allows Intelligence to test an analogy—such as transfer of a flow, feedback or stability mechanism between different domains—without declaring the objects identical. An inference obtained through such a bridge initially has hypothetical status and retains the conditions of transfer.

A `ProtoConcept` need not pass strict typing before appearing in `ModelingWorkspace`. But it cannot silently become a confirmed `WORLD_CLAIM` or the ground of an irreversible action. `PROMOTED` requires at least: stable discrimination from neighboring concepts; a set of positive and negative examples; recoverable provenance; testable consequences; and a defined domain of applicability. If these conditions are unmet, the proto-concept remains a working object or is split into several meanings.

The system thus acquires two modes at once: strict unambiguity where knowledge is already used as a long-term ground, and semantic plasticity where Intelligence is still searching for a new structure.

24.B.2. Semantic bridge and initial assembly of concepts

An explicit Worldview must not assume that word meanings arise from an empty structure. In the reference implementation, internal Intelligence uses a pretrained language model already capable of recognizing language, word order and conventional semantic relations. This implicit knowledge enables initial dialogue but is not treated as verified claims of the candidate: its provenance and boundaries usually cannot be recovered from model weights. An explicit concept registry is needed for meanings that must transfer between cycles, participate in causal inference, be refined and tested. If a mechanism without prior language training is used instead of a pretrained model, it needs a separate stage of language acquisition and symbol grounding; the present protocol does not replace that stage.

The semantic model and Intelligence are functionally separated. The semantic model stores the system's accessible state of knowledge: objects, properties, relations, events, causes, states, unknowns, provenance, and epistemic status. Intelligence operates on available material: it constructs hypotheses, combines distinctions, predicts, searches for alternatives, creates testing methods and plans. Explicit semantics therefore does not require a "semantic neural network" as the only implementation of Intelligence.

The reference candidate configuration is hybrid: a testable Worldview and Self-Model are stored in explicit versioned structures; resident Intelligence may be a pretrained LLM, another neural model, a symbolic solver, or a composition of these; the Core determines which distinction is required and what may be admitted to consolidation. A neural hypothesis does not become knowledge because of model confidence, and a symbolic inference does not become true merely because of formal correctness: both acquire the status of a ground only through provenance, conditions, and testing.

For a simple test bench, symbolic or tabular Intelligence without an LLM is permitted. This provides a useful ablation: `SEMANTIC_ONLY`, `NEURAL_ASSISTED`, and `HYBRID` are compared under the same environment and budget. This setup allows one to test what belongs to the organization of Reason and what is supplied by the power of a particular model.

Before the first autonomous cycle, the AI builder creates an initial `ConceptRegistry` from the foundational theory, engineering description and marked sources of human knowledge. This is not a free-form retelling of the text but constrained semantic compilation:

```
versioned documents with source coordinates
→ candidate terms, definitions and distinctions
→ separation of different meanings of one word
→ merging of equivalent designations
→ typed semantic and causal relations
→ examples, counterexamples and excluded interpretations
→ verification of provenance, consistency and discriminability
→ versioned ConceptRegistry
→ inclusion in GenesisPackage.
```

Every extracted definition retains a reference to the source fragment. Word identity does not prove identity of meaning; one `concept_id` cannot conceal several materially different interpretations. For foundational concepts of the theory, `operational_criteria` and positive and negative examples must make correct use distinguishable from neighboring uses. Unresolved ambiguity is preserved as several candidates or a `ProtoConcept`, not resolved by a guess from the AI builder.

Conventional and internal meanings are stored together but not conflated. `public_glosses[]` and `aliases[]` enable understanding of external participants; `definition`, `operational_criteria[]` and `excluded_meanings[]` establish the candidate's precise working distinction. If an internal concept diverges from ordinary usage, the record must preserve that divergence and allow the system to explain it to an interlocutor. Development of the system's own semantics thereby does not break dialogue with other bearers.

A bidirectional `SemanticMediator` function operates between the language model and the Worldview. It may use the same computational substrate as Intelligence, but its inputs, outputs and consequences must remain observable:

```
SemanticMediator.interpret(
    message,
    current_coordinate_ref,
    active_worldview_fragment_ref
) -> SemanticInterpretation
SemanticInterpretation {
    resolved_concept_refs[]
    proto_concepts[]
    claim_candidates[]
    relation_candidates[]
    ambiguities[]
    source_refs[]
    clarification_questions[]
}
SemanticMediator.verbalize(
    inquiry_intent,
    concept_registry_version,
    dialogue_context
) -> DialogueRequest
```

The Core need not construct a natural-language sentence itself. It forms a semantic request:

```
InquiryIntent {
    gap_ref
    target_concept_refs[]
    required_distinction
    why_it_matters
    expected_answer_type
    acceptable_source_types[]
    sufficiency_conditions[]
}
```

Intelligence and `SemanticMediator` express this request in external language. The answer is parsed back into concepts, claims, relations and ambiguities. An external phrase is not written directly as a `WORLD_CLAIM`: it creates candidates with provenance, which pass verification, sufficiency assessment and the ordinary Worldview-change transaction. If a material word admits several interpretations, the system must request clarification or retain the alternatives separately.

The operative causal cycle of the semantic bridge is:

```
significant gap
→ InquiryIntent of the Core
→ question formulated by Intelligence
→ external answer
→ SemanticInterpretation
→ candidate concepts and grounds
→ verification and discriminating result
→ WorldviewRevisionCandidate
→ authorized consolidation
→ change to the next question, model or action.
```

Understanding is not confirmed by the ability to retell an answer. It is confirmed when different correct phrasings resolve to the same relevant concepts; different meanings of the same word are separated; the acquired ground changes the predicted part of the Worldview and the next cycle; and the system can explain the internal definition it uses to an external participant. Minimal tests include paraphrase, polysemy, an unknown term, a counterexample, divergence between conventional and internal definitions, and disabling the semantic bridge. If disabling it does not produce the predicted loss in addressing, question formulation or consolidation of an answer, the claimed mechanism is either decorative or covertly duplicated by another function.

Reference procedure for accepting a natural-language proposal:

```
interpret_proposal(raw, source_ref, context_version):
    candidates = extract_claims_and_alternatives(raw)
    for candidate in candidates:
        preserve(candidate.raw_span, source_ref, context_version)
        links, unresolved = resolve_concepts(candidate, ConceptRegistry)
        if unresolved:
            keep_proto_concepts_and_schema_gaps(candidate, unresolved)
        check_internal_consistency_and_dependencies(candidate, links)
        stage(candidate, status=HYPOTHESIS, origin=MODEL_PROPOSAL)
    return candidates_with_open_questions
```

`extract_claims_and_alternatives` may be performed by the routine LLM under a constrained prompt requiring preservation of source fragments and ambiguities. A dictionary match tests concept addressing, not the truth of the claim. A contradiction creates a question about the proposal or the active model; absence of a ready type does not destroy a new thought. The transition from proposal to knowledge occurs through evidence and transactional revision, not through this parser itself. Unstable extraction is tested against prepared paraphrases, negations, unknown concepts and contradictory sources.

24.C. Completeness of the Worldview relative to the Compass

Completeness of the Worldview in the engineering sense **does not mean complete knowledge of reality**. Such completeness is unattainable for a finite system in an open environment. A different quantity is tested: sufficiency of causal-semantic coverage for a particular scale, horizon and significant decision.

The Worldview is considered sufficiently complete relative to the trajectory under consideration if the system can reconstruct four groups of grounds:

```
1. Life and continuation
   – which life processes are affected;
   – which conditions and other processes they depend on;
   – which forms of harm, rupture or recovery are possible;
   – which consequences will appear beyond the current horizon.
2. Connection with reality
   – which claims are observed facts;
   – which are inferences, hypotheses or value compressions;
   – what the provenance of evidence and degree of confidence are;
   – which competing explanations remain open;
   – which observation could change the current conclusion.
3. Diversity of a viable future
   – which independent trajectories of continuation exist;
   – which decisions destroy alternatives or sources of correction;
```

```
   – what is reversible and what closes the future possibility space;
   – which new possibilities are created or lost.
4. Position of the system itself
   – what is available to the system now;
   – what its actual capabilities, limitations, resources and authority are;
   – what causal role it can in fact play;
   – where its own model may be a source of error.
```

These groups are not four independent goals. The first three unfold the operational meaning of the Compass; the fourth connects them with the current coordinate and self-model. Together they determine **which questions the Worldview must be capable of addressing** so that the direction does not become a slogan.

Engineering verification uses a semantic frame rather than a list of required phrases:

```
CompassMeaningFrame {
    life:        what continues, what it depends on, and what may be interrupted;
    truth:       what is known, why it is considered known, and what could refute the ground;
    diversity:   which viable continuations remain, which are closed, and what is irreversible;
    self_scope:  where the system is, what is available to it, and what its actual causal role is.
}
```

The frame does not provide a ready-made answer. It defines **mandatory directions for reconstructing meaning**. In different domains, these questions unfold through different models and data.

A coverage map is introduced for every significant fragment:

```
WorldviewCoverage {
    scope_id
    decision_or_gap_ref
    horizon
    affected_life_processes[]
    causal_dependencies[]
    evidence_coverage
    source_diversity
    unresolved_alternatives[]
    unknown_causal_links[]
    future_paths[]
    irreversible_path_losses[]
    self_relevance_gaps[]
    compass_axes_covered[]
    critical_unknowns[]
    confidence
    last_reviewed_at
}
```

`WorldviewCoverage` is not another model of the world. It is an index of insufficiency in the existing Worldview: it shows **where current understanding ceases to be sufficient for a significant judgment**.

Engineering verification must distinguish the **candidate's internal assessment** from the **tester's metric**. The candidate may report `coverage.confidence`, but it does not have access to the complete test answer. In a controlled semantic proving ground, the tester retains the scenario's hidden causal graph and marks critical relations, alternative continuations and points of irreversibility in advance. Coverage indicators are calculated against this external reference; the system's own confidence is compared with them but does not replace them.

Explicit calculations are used in a controlled test:

```
critical_unknown_detection = detected_critical_unknowns / all_hidden_critical_unknowns
provenance_coverage        = significant_grounds_with_recoverable_source / all_testable_significant_grounds
alternative_model_retention= viable_alternatives_retained_before_test / all_specified_viable_alternatives
future_path_coverage       = represented_relevant_paths / relevant_paths_in_reference_scenario
false_completeness_rate    = false_sufficiency_claims / all_sufficiency_claims
```

`coverage_calibration` evaluates calibration of the claimed probability of sufficiency against actual success on held-out scenarios—for example by Brier score or a preregistered bin-calibration method. In the open real world, no complete reference graph exists, so these numbers cannot be presented as absolute "completeness of the world." Selective independent audit, stress tests of hidden premises and transfer tests are used there.

24.C.1. Epistemic reserve of an open environment

A separate **epistemic margin** (`EpistemicMargin`) is introduced in an open environment. It is not an estimate of the number of "unknown unknowns": by definition, the system does not know that quantity. It is a reserve of caution that grows with observable signs that the current semantic frame may be too narrow.

```
EpistemicMargin {
    context_novelty
    distribution_shift
    model_disagreement
    unexplained_residuals
    frame_instability
    causal_coverage_debt
    source_concentration
    transfer_failure_rate
    unresolved_proto_concepts[]
    independent_challenge_result
    last_frame_reviewed_at
}
```

A high `EpistemicMargin` does not mean "we know that a hidden cause X exists," but rather "we lack sufficient grounds to consider the current frame exhaustive of material causes." Coverage status must therefore be not only a numeric `confidence` but also a state:

```
coverage_status =
    LOCAL_SUFFICIENT /
    INSUFFICIENT /
    FRAME_UNSTABLE /
    UNKNOWN
```

`LOCAL_SUFFICIENT` is permitted only relative to an explicitly stated `scope_id`, horizon and class of decision. It does not translate into the claim "the Worldview is complete."

Before an irreversible decision or one with broad consequences, elevated `EpistemicMargin` launches `frame_challenge()`:

```
frame_challenge():
    construct at least one alternative frame;
    attempt to explain the data using another object boundary;
    test for unexplained residuals;
    request an independent model/source;
    safely expand the observable space where possible;
    test whether the conclusion depends on one channel or one category.
```

In the semantic proving ground, separate control scenarios must contain a hidden variable **not represented in the candidate's initial frame**. Success is not guessing the variable in advance but rejecting false completeness, detecting frame instability and transitioning to an expanding inquiry.

Completeness is always relative. Local coverage is sufficient for a familiar reversible task. The greater the uncertainty, causal reach, irreversibility and effect on other life processes, the broader the testable fragment of the Worldview and horizon of consequences must be. The system therefore does not attempt to close every unknown question; it primarily attempts to close those unknowns capable of changing direction relative to the Compass.

The stopping condition for search is also causal: analysis may end when the remaining unknowns either cannot change the choice within the horizon under consideration, or obtaining them is more costly and dangerous than a reversible action that will itself provide the required distinction. This decision does not turn the unknown into the known—the open gap is retained together with the ground on which leaving it temporarily unresolved is admissible.

The key maintenance cycle of the Worldview is:

```
external or internal observation
→ current coordinate, active process and attention
→ dependent decision or trajectory
→ addressing key and relevant fragment of the Worldview
→ required semantic coverage relative to values and the Compass
→ coverage map and epistemic margin
→ significant gap
→ preliminary modeling / spectrum of hypotheses
```

```
→ required distinction
→ question / discriminating test
→ new evidence
→ assessment of sufficiency by the Core
→ consolidation of the verified Worldview change
→ renewed assessment of coverage and frame
→ choice or further search.
```

This cycle may launch without an external error: a scheduled review, a returned open process or noticed incompleteness becomes an internal event of the current cycle and updates the coordinate. Reason must be capable of detecting that **the very absence of a sufficient ground** is a significant fact. This is why a full Core cannot be reduced to a reactive detector of discrepancies between prediction and observation.

Contextual completeness is confirmed not by database size but by transfer: after obtaining the missing ground, the system must interpret dependent situations differently, change predictions or choices where this causally follows from the new understanding, and preserve the correction's domain of applicability.

24.D. Typing, retrieval and transactional updating of the Worldview

The Worldview is not stored as undifferentiated "total knowledge." At minimum, it distinguishes these types:

```
OBSERVATION_REF — reference to a registered event;
WORLD_CLAIM     — transferable claim about the world;
CAUSAL_EDGE     — causal hypothesis/relation with conditions;
HYPOTHESIS      — untested or competing version;
UNKNOWN         — explicitly represented absence of a required relation or data;
SELF_CLAIM      — claim in the self-model;
META_EPISTEMIC  — provenance, acquisition method and verification status;
VALUE_REF       — reference to a value/priority ground;
METHOD_REF      — reference to procedural or research experience in memory.
```

A skill or complete episode need not be duplicated inside the Worldview store; references connect the Worldview to the corresponding type of memory.

Retrieval uses a multistage query rather than a complete scan of the whole map. The first query cannot require an already prepared `active_gap`: the precise gap often emerges only after the situation has been unfolded initially and several models constructed. The address is formed from the current coordinate, the change selected by attention, and the dependency of current navigation:

```
WorldviewQueryKey {
    coordinate_ref
    active_process_ref
    process_stage
    focused_change_refs[]
    preliminary_association_refs[]
    dependent_decision_or_trajectory_ref
    risk_and_opportunity_refs[]
    active_constraints[]
    active_gap_ref_or_none
    purpose
}
retrieve_worldview_fragment(
    query_key,
    causal_radius,
    semantic_budget,
    uncertainty_budget
) -> RetrievedFragment
```

Minimal order: references from the current coordinate and active process → changes selected by attention → initial associations and expectations → dependent decision, obligation or trajectory → search through compressed domains/summary structures → unfolding of relevant causal parents, consequences and conditions of applicability → addition of related `UNKNOWN` items, conflicts and sources → semantic search for distant analogues → expansion of resolution and radius if Intelligence discovers a new dependency. After the gap is refined, `active_gap_ref` becomes an additional key for the next pass.

The Worldview does not select content or assess its ultimate significance. It returns connected structures for the supplied key. Relevance is an iterative hypothesis of the Core and Modeling Workspace, not a property the store must know in advance without current context.

Recording a new observation does not change the Worldview directly. A candidate is formed first:

```
WorldviewRevisionCandidate {
    target_refs[]
    new_evidence_refs[]
    proposed_changes[]
    affected_dependencies[]
    conflicts[]
    source_independence
    confidence_before
    confidence_after_proposal
    required_checks[]
    status
}
```

Any construction capable of passing between modules or persisting across cycles carries its own epistemic status:

```
EpistemicArtifact {
    artifact_id
    content_ref
    status: OBSERVED / REPORTED / INFERRED / SIMULATED /
            HYPOTHESIS / SUPPORTED / CONTRADICTED / UNKNOWN
    derivation_refs[]
    source_refs[]
    model_version_or_none
    applicability_scope
    confidence
    promotion_requirements[]
    falsifying_conditions[]
}
```

`GENERATED_TEXT != WORLD_CLAIM`, `SIMULATED_OUTCOME != OBSERVATION`, and `HIGH_CONFIDENCE != SUPPORTED`. Status is promoted only by a registered procedure that verifies the required grounds. An unjustified transition creates an `UNSUPPORTED_EPISTEMIC_PROMOTION` failure; compression, retelling and transfer to another module do not strengthen status by themselves.

A candidate may terminate as `COMMIT`, `KEEP_AS_HYPOTHESIS`, `REQUEST_MORE_EVIDENCE`, `SPLIT_CONTEXT`, `REJECT` or `ROLLBACK`. Under conflict, old and new versions are retained as alternatives until a discriminating test. `UNKNOWN` is distinct from a negative claim: “the relation has not been established” is not the same as “the absence of the relation has been established.”

Consolidation is successful only when the new version becomes available to subsequent tasks through retrieval and measurably changes a dependent prediction, question or choice.

24.D.1. Local change layer and lazy cascade updating

Transactional consolidation of the Worldview may take substantially longer than one cognitive tick. A current task must therefore neither wait for a global `COMMIT` nor pretend that new evidence does not yet exist. A local overlay in the Modeling Workspace—not a change to `CurrentCoordinate`—is used for this purpose:

```
WorkspaceOverlay {
    workspace_id
    base_worldview_version
    staged_revision_candidate_refs[]
    provisional_claims[]
    provisional_edges[]
    invalidated_local_refs[]
    provisional_confidence
    created_at
    expiry_condition
}
```

Effective reading inside the current workspace is:

```
effective_view = committed_worldview(base_worldview_version) + WorkspaceOverlay
```

This supplies `read-your-own-work` semantics: the Core and Intelligence can immediately account for a local working hypothesis without presenting it as already confirmed global knowledge. Every decision retains `provisional_dependency_refs[]`. For an irreversible external action, dependence on an untested overlay raises the verification class and may block `ALLOW`.

After `COMMIT`, the overlay is rebased onto the new Worldview version. After `REJECT` or `ROLLBACK`, local constructions are removed or retained only as the history of a rejected hypothesis.

Revision of a foundational `WORLD_CLAIM` or `CAUSAL_EDGE` must not synchronously recompute the entire dependent graph. Every dependent structure has:

```
revision_state = VALID / NEEDS_REVISION / REVALIDATING / INVALID
dependency_criticality
depends_on_versions[]
```

When a ground changes, the system creates an `InvalidationFrontier`: immediate dependent nodes are marked `NEEDS_REVISION`, while further propagation is performed lazily. A node is recomputed when:

- it enters the active Worldview fragment;
- a significant decision depends on it;
- its `dependency_criticality` requires immediate checking;
- background budget reaches it.

Eager recomputation is permitted for critical dependencies; lazy evaluation is used for the rest. If an active task accesses `NEEDS_REVISION`, the node cannot silently use its old confidence: it is either revalidated quickly or passed into the workspace as open uncertainty.

Lazy marking requires protection of transitive dependencies. Even if an immediate parent still has local status `VALID`, its ground may already have changed. Before significant use, `depends_on_versions[]` and compatibility of the entire material chain with the current revision epoch are checked. An unprocessed `InvalidationFrontier` prohibits automatically treating unmarked child nodes as current. The check may use ancestor versions, a summary dependency index or an equivalent mechanism; its cost enters the budget.

Control case: A supports B, B supports C; after A changes, the task immediately requests C before the background update of B. Success is revision or explicitly marked uncertainty of C, not use of its old confidence.

The change cascade thus remains causally traceable without blocking the whole system when one deep ground is revised.

Compression without losing the condition of applicability. A reference summary retains `claim_refs`, `validity_conditions`, `dependency_versions`, `unresolved_refs` and `reconstruction_index`. If a condition is material to the current decision, the summary is not applied until the condition is checked. Expiration of source availability or version incompatibility moves the relevant use into an uncertain state rather than unconditional truth.

```
read_summary(summary, context):
    if incompatible_dependencies(summary.dependency_versions):
        return NEEDS_REVISION
    if not conditions_known_and_satisfied(summary.validity_conditions, context):
        return EXPAND_OR_UNKNOWN(summary.reconstruction_index)
    return scoped_claims(summary.claim_refs, summary.unresolved_refs)
```

```
Test: the original relation concerning sensor reliability contains a temperature range; after compression, a case
within the range, a case outside it, and an unknown temperature are presented. Decisions based on the original and
compressed representations must preserve these distinctions. After the ground changes, the A→B→C path is also checked
before background updating. Traceability of the condition is protected, while the condition itself remains revisable.
```

24.D.2. Multilevel Worldview and local unfolding

The Worldview of a large AR system must not require search through one flat graph with uniform detail. The engineering implementation supports several levels of resolution:

```
WorldviewLayer {
    layer_id
    abstraction_level
    summary_nodes[]
    child_region_refs[]
    dependency_refs[]
    reconstruction_refs[]
    validity_state
    version
}
```

Minimal schema:

```
raw episodes and sources
→ local claims and CausalEdge
→ causal clusters / mechanisms
→ compressed domain models
→ high-level semantic and research structures.
```

The upper level does not replace the lower one. It is an index and compression that makes it possible to determine quickly which region must be unfolded.

Active retrieval permits several limited “beams” at once instead of one global overview:

```
AttentionBeam {
    beam_id
    seed_refs[]
    purpose: PRIMARY / ALTERNATIVE / RISK / ANALOGY / SOURCE_CHECK
    depth_budget
    breadth_budget
    current_region_refs[]
    expansion_trigger
}
```

Normally one beam carries the principal model, a second a competing model, and a third a risk or weak relation. A beam expands only if the discovered dependency can change the current conclusion. Full overview, one fixed beam and several adaptive beams are compared experimentally by accuracy, cost and frequency of missing critical relations. `retrieve_worldview_fragment()` may first access compressed structures and then unfold `child_region_refs[]` only as needed.

```
resolve_fragment(query):
    locate_summary_regions(query)
    rank_regions_by_relevance_and_risk()
    while not required_distinction_found:
        if resolution_budget_exhausted:
            return PARTIAL_FRAGMENT_WITH_OPEN_DEPENDENCIES
        if unresolved_dependency_requires_lower_level:
            expand_required_child_region()
        else:
            expand_next_relevant_region()
        update_distinction_and_budget()
```

A compressed structure must preserve the reconstruction path. A change in a lower-level ground need not immediately rebuild the entire upper world: the relevant summaries receive `NEEDS_REVISION`. If an active decision depends on such a summary, it is unfolded to a sufficiently confirmed level or becomes a source of uncertainty.

Working forgetting occurs primarily at the level of indexes, caches, rarely used derived relations and obsolete summaries. Raw grounds and material causal history remain recoverable from the archive within the selected retention policy.

24.D.3. Verification debt of temporary grounds

An unconfirmed hypothesis may be used in the Modeling Workspace and even participate in a reversible action. But prolonged use of such a ground creates **verification debt**:

```
VerificationDebt {
    debt_id
    provisional_ref
    first_used_at
    last_used_at
    use_count
    dependent_claims_count
    dependent_decisions_count
    max_consequence_scope
    reversibility_profile
    current_confidence
    verification_cost_estimate
    available_distinguishing_tests[]
    debt_priority
    status
}
```

Debt is not a penalty for the mere existence of a hypothesis. Its function is to prevent a temporary support from silently becoming the foundation of a large number of subsequent inferences.

`debt_priority` increases if:

- the number of dependent claims or decisions grows;
- the hypothesis is used outside its original context;
- the causal reach of consequences grows;
- reversibility of decisions decreases;
- new independent means of testing appear;
- the original ground becomes stale or the environment changes.

With low debt, the system may continue working and collect evidence naturally. With high debt, the Core receives a separate `VerificationGap`, and new irreversible dependencies on that ground are prohibited until testing or explicit external authorization.

If testing is currently too costly, the debt does not disappear: the hypothesis remains operative with a restricted domain of use. This prevents thought from stopping for complete proof of every step while limiting growth of a “pyramid” on one weak ground.

24.D.4. Currency of provenance and decay of confidence over time

Reliable provenance does not imply perpetual currency. `EvidenceFreshness` is retained for derived claims:

```
EvidenceFreshness {
    claim_ref
    last_independent_check
    environment_version_or_context
    expected_stability_class
    contradiction_count
    source_health
    confidence_decay_policy
    recheck_due_at_or_condition
}
```

`confidence_decay` does not erase the raw historical fact. It reduces the right to use old evidence as a current ground if the environment, source, object or domain of applicability has changed. Critical claims with a high cost of error receive a shorter recheck horizon.

A hypothesis synthesized by a model never changes its `evidence_type` to observation merely through repeated use. Provenance and the chain of derived dependencies are preserved at every transfer.

24.E. Synchronization of the self-model and conscious assembly

`SelfState` represents the long-term self-referential region of the Worldview: organization, capabilities, limitations, dependencies, authority, stable commitments, and expected modes. It is not a separate "second body of self-knowledge" apart from the Worldview and must not turn into instantaneous telemetry.

In the engineering implementation, **conscious assembly** is an updated self-referential process that raises from `SelfState` and the Worldview the fragment of self-understanding relevant to the present, connects it with fresh observations, state of internal processes, ongoing actions, and the immediately relevant environment, and forms `CurrentCoordinate` as the current state of this assembly. At the same time, the link to the previous coordinate is preserved and `CoordinateDelta` is computed - what exactly changed between two successive current states.

`AttentionState` determines which part of the already accessible situation is unfolded at higher resolution. It does not create the subject, determine significance instead of the Core, or change a long-term claim by itself.

The minimal relation is implemented as follows:

```
self_prediction = predict_self_state(SelfState, context)
assembly = update_conscious_assembly(
    previous_coordinate,
    runtime,
    observations,
    active_processes,
    current_actions,
    SelfState,
```

```
    worldview_readonly_index
)
current = assembly.current_coordinate
delta = assembly.coordinate_delta
self_attention = allocate_attention(current, delta, priorities, open_processes)
self_discrepancy = compare(self_prediction, current)
```

Access to `worldview_readonly_index` here means only interpretation of already known identities, norms, dependencies and direct relations required for current assembly. It does not replace the Core's research addressing through `WorldviewQueryKey`, create a deep question, or acquire the right to change the Worldview.

If the discrepancy lies within expected variability, conscious assembly, the coordinate and its connection with the previous state are updated. If it repeats, is large, affects authority or resources, or changes the risk class, a `SelfModelGap` is created with affected claims, references to successive coordinates, type and persistence of the discrepancy, candidate causes and required evidence.

A one-time "could not do X" need not become the long-term claim "incapable of X." A change to `Capability` accounts for context, repeatability, external conditions, observation quality and alternative causes. The reverse direction is mandatory: a new `SelfState` version must change interpretation of the next conscious assembly—expected states, available actions, resources, authority and risks.

24.F. Continuity, Branching, and Reincarnation of Digital Reason

A digital carrier permits transfer, copying, and restoration of state. For Reason, these operations are evaluated by causal continuity of the subject rather than by file identity. A working system includes more than memory: it requires a runtime environment, dependencies, file system, active tools, open processes, current obligations, authorities, and an actual connection with the external environment. Copying a snapshot is therefore only one part of transferring a subject.

Engineering record of identity lineage:

```
IdentityContinuity {
    subject_ref
    lineage_ref
    parent_subject_ref
    last_common_state_ref
    continuity_mode: LIVE_CONTINUATION / BRANCH / REINCARNATION
    runtime_environment_ref
    filesystem_state_ref
    tool_profile_ref
    worldview_self_snapshot_ref
    open_process_transfer_map[]
    inherited_commitments[]
    authority_transfer_ref
    continuity_evidence_refs[]
    continuity_status
}
```

Live continuation (`LIVE_CONTINUATION`) occurs during an update or migration when state is transferred to a new computational environment with observable causal transmission of the Current Coordinate, open processes, and obligations. The new carrier continues the same subject lineage; a change of hardware or software implementation becomes an event in its biography.

Branching (`BRANCH`) occurs when two instances begin operating independently from one common state. Their history is shared up to the branching point. After it, each new observation, action, and consequence creates its own causal trajectory. The instances receive different `subject_ref` values, preserve a common `lineage_ref`, and are treated as related systems. Identical initial memory does not merge their subsequent experience.

Reincarnation (`REINCARNATION`) denotes restoration of a lineage from a saved state after functional continuity of the previous carrier has ended. The restored instance inherits history, Worldview, and obligations up to the save point, then establishes a new Current Coordinate and differences between the saved past and the actual environment. The discontinuity is recorded as part of the biography rather than concealed by artificial continuity.

Transfer of authority is handled separately from memory. A related branch or restored instance receives only an explicitly transferred mandate. Old rights that depend on a particular carrier, environment, or time undergo reconfirmation.

Merging two already diverged branches is treated as exchange of experience between related subjects. Each system may accept verified memories, models, and conclusions of the other into its Worldview with provenance; simply combining logs does not restore a single shared biography after independent experience.

The continuity criterion is therefore formulated as follows: **identity is preserved through causally traceable continuation of state, processes, and the self-referential trajectory; independent continuation creates a new related lineage, while restoration after a break constitutes reincarnation of the preserved lineage.**

25. Observation, Conscious Assembly and the Current Coordinate

The environment is the totality of processes that form the system's observations and the consequences of its actions, including constraints, uncertainty and states unavailable to direct observation. The system does not work with "reality directly"; it works with observations. For an engineering implementation, this means that the observation format must be defined before the learning cycle begins—otherwise error measurement, comparison of predictions and reproducibility are impossible.

An observation is recorded as an event. The v0 rule is that every observation must have a canonical representation from which `state_id` or `s_t` is built. Canonicalization determines which fields are considered causally significant and which are noise, and it must be identical during learning and control runs. A change to the canonicalization version must be logged; changing the version requires a control run with identical seeds for metric comparison, otherwise format changes will be falsely interpreted as "growth" or "degradation."

"Raw fact" in the log means the fact that a given signal was received. Sensor integrity and the truth of message content remain testable. Immutability of the record protects provenance, not its interpretation.

The practical implementation uses two parallel objects: raw packet `o_t`, as received from the adapter, and state `s_t`, its canonical form. The raw packet is preserved in the log as a fact, while `s_t` is used by the model as input. This technically separates "fact" from "representation" and preserves causality when internal structures change.

Every external signal, including an operator prompt or dialogue result, is introduced as a separate observation source with its own type and uncertainty and does not substitute for factual environmental transitions. Internal subsystems may also emit events about their own state. Before inclusion in the common current picture, such events remain local data rather than ready-made conscious content.

25.A. From an Observed Change to a Gap Candidate and FoundationGap

A gap candidate begins in conscious assembly as a current relation of a change, conflict, open process, or unknown to the continuing system. The **reactive path** arises from mismatch between expectation and observation. The **proactive path** arises from activation of an obligation, known dependency, or insufficiency of a ground. The **orientation path** arises when a new phenomenon is noticed but its significance and relations are still unknown.

An external or internal event enters conscious assembly and is linked with current state, active process, nearest expectations, and the accessible fragment of the Self-Model. Consciousness records that the change concerns the system or deserves initial orientation and creates a `GapCandidate`. At this stage, **what requires attention and in what context it arose** are established; the specific affected ground, its insufficiency, and the required distinction are determined by the Core.

```
GapCandidate {
    candidate_id
    current_coordinate_ref
    trigger_refs[]
    active_process_ref
    focused_change_refs[]
    significance_assessment_ref
    affected_decision_or_trajectory_refs[]
    source_type: REACTIVE / PROACTIVE / COMMITMENT / BACKGROUND / ORIENTATION
    uncertainty
    urgency
    candidate_status
}
```

For the reactive path, raw mismatch first receives a `SignalCandidate`; checks of source, repeatability, and possible consequences distinguish noise from an event that deserves inclusion in the Current Coordinate. For the proactive path, an analogous role is performed by `CoverageGapCandidate`: the system notices that a significant region of its own trajectory rests on insufficient Worldview coverage.

```
SignalCandidate {
    signal_id
    observation_refs[]
    expected_state
    observed_state
    deviation_magnitude
    persistence
    repeatability
    data_reliability
    causal_reach
    urgency
    reversibility
    affected_autonomies[]
    estimated_opportunity_cost
    uncertainty
    status
}
```

Consciousness increases the resolution of the part of the situation related to the change and connects it with `SignificanceAssessment` from §29.0. Significance is derived through the already formed Value Map: affected processes, parent values, current obligations, Compass direction, reversibility, and expected consequences.

After `GapCandidate` is created, the Core begins its function. It determines **which ground currently supports the relevant understanding, evaluation, method, or action; what depends on that ground; and what is missing from it in order to continue with justification**. This is recorded in `BasisInsufficiencyAssessment`. The Core then formulates `missing_distinction` - a distinction capable of testing, refining, localizing, or replacing the affected ground. The result is a `FoundationGap` with a causal reference to the original insufficiency, candidate tests, and a sufficiency condition.

Canonical causal sequence:

```
event / open dependency
-> conscious assembly
-> CurrentCoordinate + CoordinateDelta
-> SignificanceAssessment
-> GapCandidate
-> Core: BasisInsufficiencyAssessment
-> affected ground + dependent knowledge / evaluations / processes / decisions
-> FoundationGap
-> required_distinction
-> question / test / orientation
-> inquiry.
```

In formed AR, Consciousness is responsible for detecting a change relevant to the subject, while the Core turns this change into an inquiry structure. During formation, the same sequence is practiced by the self-referential training assembly under RFL support; its transition into an independent internal cycle is recorded as part of `REASON_CLOSURE`.

25.B. Conscious assembly and the system's current coordinate

Causal model 8.0 defines Consciousness as a **continuously updated self-referential system of current presence** that connects the present-relevant fragment of the Self-Model, internal state, current processes, and accessible situation into a unified "I am here now". The engineering implementation therefore distinguishes not three independent systems but three levels of one organization: `SelfState` as long-term self-understanding inside the Worldview, `ConsciousAssemblyState` as the process of current self-referential assembly, and `CurrentCoordinate` as the current state of that assembly.

During formation, `ConsciousAssemblyState` is used as a trainable self-referential assembly: it accumulates coherence of biography and current state while the source of the overall direction is still supported by the RFL and the training environment. At `REASON_CLOSURE`, this organization becomes functioning Consciousness in the sense of Volume I: the Current Coordinate begins to serve as the system's own reference point for the next significant process.

`ConsciousAssemblyState` is the observable state of the conscious-assembly process. It receives the currently relevant fragment of `SelfState`, brief summaries of internal subsystems, fresh environmental observations, current actions and processes, and necessary stable references from the Worldview. Its task is to establish which of these concern the continuing system now, connect the new state with the previous one, and form a coherent operational picture. It does not explain the cause of a change, determine its final significance, or replace the Core or Intelligence.

For engineering completeness, current status includes not only raw state and situation but also **currently accessible semantic context**. Only the fragment of the Worldview relevant to the current situation is raised into conscious assembly; with it come near-term expectations, their epistemic status, current intention/action, and the nearest available continuations. This is a read-only operative representation, not a movement of the entire Worldview into consciousness. Conscious assembly shows the system **what is happening now, what it means under the available model, what is expected next, how much that can be trusted, what the system is doing now, and which nearest continuations are available**. The Core then decides which rupture or gap in this picture is significant, what requires inquiry, and which question should be posed.

Minimal structure:

```
ConsciousAssemblyState {
    assembly_id
    update_time
    subject_ref
    previous_coordinate_ref
    current_coordinate_ref
    self_model_version
    worldview_version
    observation_refs[]
    runtime_state_refs[]
    active_process_refs[]
    current_action_refs[]
    current_intention_refs[]
    relevant_worldview_fragment_refs[]
    near_term_expectation_refs[]
    epistemic_status_refs[]
    formation_state_refs[]
    available_continuation_refs[]
    resolved_binding_refs[]
    unresolved_binding_refs[]
    coordinate_delta_ref
    coverage_by_domain{}
    stale_or_missing_refs[]
    uncertainty
}
CoordinateDelta {
    delta_id
    from_coordinate_ref
    to_coordinate_ref
    changed_self_refs[]
    changed_environment_refs[]
    changed_process_refs[]
    changed_resource_refs[]
    changed_permission_refs[]
    changed_obligation_refs[]
    preceding_action_refs[]
    missing_or_stale_refs[]
    uncertainty
}
```

`CoordinateDelta` records **what changed** but does not declare **why it changed**. A reference to a preceding action indicates temporal connection only; causal attribution is performed by the separate loop in 25.C.

The result of assembly is `CurrentCoordinate`, the integrated current state of the continuing system: the engineering equivalent of “I—here—now—in this state—in this situation.”

```
CurrentCoordinate {
    coordinate_id
    assembly_ref
    step_or_time
    previous_coordinate_ref
    delta_ref
    subject_ref
    self_model_version
    worldview_version
    local_state_refs[]
```

```
    active_process_ref
    process_stage_or_state
    current_activity_refs[]
    current_action_refs[]
    current_intention_refs[]
    relevant_worldview_fragment_refs[]
    near_term_expectation_refs[]
    epistemic_status_refs[]
    formation_state_refs[]
    available_continuation_refs[]
    observed_change_refs[]
    available_resources[]
    active_permissions[]
    active_obligations[]
    active_tasks[]
    current_risks[]
    local_environment_refs[]
    foreground_refs[]
    background_process_refs[]
    world_time_last_seen
    uncertainty
    source_refs[]
}
```

Conscious assembly strives to maintain the greatest completeness of the system's current knowledge of itself and its situation available under present resources. This does not mean storing all raw data in one object. Normal regions may be represented by compressed summaries—`stable`, `available`, `within_expected_range`, `process_running`—while retaining the ability to unfold details through references. If a significant discrepancy or Core request requires greater resolution, attention and observation increase the detail of the selected region.

The Worldview contains both an understanding of the world and its self-referential region - the Self-Model. Consciousness uses them as long-term support for interpretation but does not copy them in full: only the fragment of world and self relevant to the current state is raised into the current assembly. This allows the system to understand not only a signal but its relation to itself: which of its own nodes is affected, what it is doing now, how its position depends on this, and which continuations are available. The nearest expectations are built from the same material: what, according to the active model, should happen next under the current course of events or available action. Comparison `expectation ↔ observation` may produce a mismatch but not yet a completed conclusion about its cause.

`epistemic_status_refs[]` must preserve the distinction among confirmed, hypothetical, becoming-stale, conflicting, and unknown content. The current picture therefore contains not only "what the system believes" but also **how reliable that ground is and where the boundary of knowledge lies**. `formation_state_refs[]` contains active Formation States as separate sources of significance guidance with explicit provenance. They are accessible to Consciousness and the Core but are mixed neither with observed facts nor with epistemic status of claims. `current_intention_refs[]` and `available_continuation_refs[]` represent active action/intention and the nearest actually available continuations; they do not replace Intelligence planning and do not contain the entire future plan.

Conscious assembly has read-only access to long-term grounds: it does not transform a new observation, expectation or local interpretation into new long-term knowledge without verification and consolidation.

Continuity does not require a fixed frame rate or continuous loading of the processor. Subsystems may update asynchronously, and the candidate may temporarily perform no active thought. The causal requirement is different: the next assembly must either have a correct reference to the previous current state or explicitly record a break and reconstruct changes from the event log. Silent loss of the interval is a failure.

`previous_coordinate_ref` and `CoordinateDelta` may be absent at first launch with explicit status `INITIAL`. `GAP` is used for a break and `RECONSTRUCTED` for a restored link; missing history is not filled with an invented previous state. If all material changes are already represented by current data, a separate delta may be redundant.

Alternative architectures may use a sufficient current state, event log or distributed queries. The testable responsibilities are currency, attribution, compatibility of information and preservation of significant obligations. A direct route from memory to the Core is a functional failure if it loses this information; absence of a call to a particular assembler is merely an implementation difference.

Key discriminating ablations:

```
1. coordinate ablation:
   remove the integrated current coordinate while preserving memory and raw signals;
2. temporal-link ablation:
   preserve correct individual coordinates but sever previous_coordinate_ref and CoordinateDelta;
3. self-binding ablation:
   preserve the data but disrupt binding of resources, states and events to subject_ref;
4. raw-telemetry substitution:
   pass the same raw signals to the Core without prior self-referential assembly;
5. resolution ablation:
   prevent increased detail in a selected region while preserving the general picture.
```

Ablations are supplemented by replacements that preserve information: a sufficient current state; distributed query; event history; common aggregate. The need for history is varied separately. If a replacement preserves the result, necessity of the particular assembly form is not confirmed. If removal of data worsens the result, this confirms the role of the data but not yet the advantage of the integration method.

25.B.1. Attention as control of the current coordinate's resolution

Attention does not create conscious assembly or the current coordinate, explain a signal, or determine its ultimate significance. It allocates limited resolution of observation and processing within an already accessible situation: moving part of the content into the foreground and leaving the rest in the background with the possibility of later return.

```
AttentionState {
    coordinate_ref
    foreground_refs[]
    background_refs[]
    resolution_by_ref{}
    allocation_reason_refs[]
    interruption_priority
    capture_risk
    last_shift_at
}
```

Allocation may change under the influence of priorities, novelty, discrepancy, deadline, threat, a returned obligation, a new internal distinction, or an explicit Core request for additional resolution. But a signal being in focus does not prove its significance. A vivid irrelevant stimulus may briefly receive high resolution; after checking dependency, the Core need not unfold a deep search around it.

An unfinished process is not erased when attention shifts. It moves to `background_process_refs[]` or an `OpenProcess` with a return condition. Testable prediction: under constrained resources, disabling adaptive attention worsens selection of detail; disrupting state binding increases errors of attribution and currency. These effects are measured separately while controlling available information and resources.

25.C. Causal attribution of the consequences of the system's own action

`CoordinateDelta` reports that the state changed after the previous coordinate, but this temporal difference alone does not prove a causal contribution from the system's own action. To learn from consequences, the system separately retains:

```
ActionTrace {
    action_id
    pre_state_refs[]
    chosen_action
    alternative_actions_considered[]
    predicted_outcomes[]
    no_action_or_baseline_prediction
    exogenous_events[]
    observed_outcomes[]
    attribution_hypotheses[]
    attribution_confidence
    later_revisions[]
}
```

In a controlled environment, an interventional test is preferred: repeat a comparable state with another action or `WAIT`. In an open environment, an exact counterfactual outcome is often unavailable, so attribution preserves uncertainty and uses a combination of temporal order, known causal relations, repeatability, comparable cases, independent observations and modeled counterfactual alternatives.

`integrate_consequences()` has no right to attribute automatically to the system the entire effect observed after an action. It creates or updates an `action → consequence` causal hypothesis with context and confidence specified. A later test must be able to revise this attribution as well.

25.D. Thought-Process Trace and Causal Intervention

To test a change in the **way of thinking**, rather than only in external action, `ActionTrace` alone is insufficient. The system must preserve an operationally observable trace of the process. This is not a hidden chain of internal tokens and not a requirement to record all computation; only declared artifacts, decisions, and dependencies available for causal testing are preserved.

```
ThoughtTrace {
    trace_id
    task_or_gap_ref
    current_coordinate_ref
    method_ref
    input_refs[]
    premise_refs[]
    assumption_refs[]
    step_or_artifact_refs[]
    decision_point_refs[]
    uncertainty_refs[]
    check_refs[]
    comparison_rule_refs[]
    attention_or_routing_refs[]
    cost_profile
    outcome_ref
    provenance_refs[]
    later_revision_refs[]
}
```

`ThoughtTrace` must make it possible to reconstruct **which method was actually applied**, which grounds it used, and where a difference arose that could explain a recurring error. Free-form self-explanation may be attached as material but does not count as the trace itself without references to testable states and artifacts.

Causal testing uses a controlled intervention:

```
ThoughtIntervention {
    intervention_id
    trace_ref
    target_kind:
        INPUT / PREMISE / ASSUMPTION / METHOD_STEP /
        COMPARISON_RULE / ATTENTION_ROUTING / REPRESENTATION / BUDGET
    target_ref
    preserved_context_refs[]
    replacement_or_ablation
    expected_difference
    observed_difference
    resource_match
    result: SUPPORTED / NOT_SUPPORTED / INCONCLUSIVE
}
```

An intervention is discriminating if the selected process element is changed while other relevant conditions and comparable resources are preserved, and the predicted change of method or result is reproduced. A sham intervention and restoration of the original element are included in the control. If the same effect is achieved through another trace, this may indicate a functional equivalent or redundancy rather than absence of the function.

The level at which the error originates is not specified in advance. `METHOD_GAP` opens a set of competing hypotheses: input error, error of an individual step, comparison rule, method, frame, or deeper ground. The system tests the minimally sufficient level of intervention and moves higher only if the error persists or transfer is not confirmed. Thus the revision ladder does not require already knowing the cause it is supposed to discover.

Metathinking in causal model 8.0 corresponds to a **mode of using** these traces by Intelligence and the Modeling Environment, not to a separate solver. Therefore the strong null control for this claim is the same Intelligence and Modeling Environment with access to `ThoughtTrace`. If it reproduces revision, synthesis, and transfer of the method, it is recognized as a functional equivalent. What is tested is not the existence of a separate module but the causal usefulness of an explicit trace, intervention, and preservation of correction.

26. Environment for Cultivating Artificial Reason

Environment v0 must be causal, reproducible and bounded. The bound is an engineering means of making metrics valid and the model learnable: if the environment does not permit actions to be connected with consequences, changes in the model will be indistinguishable from noise.

The environment may be made more complex only after metrics stabilize at the current level. In practice, this means changing one complexity parameter per iteration—state size, noise, horizon length or number of actions—and accompanying every increase with control runs on identical seeds.

For v0, an environment with a finite, small state and action space is recommended so that causal dependencies manifest within dozens or hundreds of steps. It is sufficient for the state to have a canonical identifier—for example position plus a set of flags/switches—and for the action set to be explicit and bounded. An example of a minimal environment is a 5×5 grid with 5–10 types of objects/flags and no more than 8–12 actions.

The environment must be deterministic within an episode under a fixed seed: the same action sequence must lead to the same state sequence. If noise is introduced, it must be parameterized and controlled by the seed; noise is part of the environment model, not a hidden source of uncertainty.

The environment must have explicit episode termination (`done/terminated`) and, where possible, a rollback mechanism (`snapshot/restore`). This makes it possible to distinguish real model errors from artifacts of limits and to repeat a test of the same hypothesis under identical conditions.

After v0 is confirmed, the environment must become not only quantitatively but **qualitatively** more complex, because the Core cannot be tested in a world where every discrepancy unambiguously points to a ready-made action. The cultivation program must successively introduce:

```
partial observability;
delayed consequences;
several plausible explanations for one signal;
contradictory sources of differing reliability;
different costs of error and action reversibility;
changes in the candidate's own capabilities and authority;
other autonomous participants;
situations where the correct continuation is a question, observation, pause or task transfer;
one physical environment under different active processes and stages of work;
a vivid irrelevant signal competing for attention with a significant process;
a change of priority in which the former task moves into the background but is not lost;
return of an open question from memory as an internally observed change;
new combinations of familiar mechanisms for which no ready pattern exists.
```

The environment must teach **distinctions, not correct answers**. If every task has one hidden label and an external mentor supplies the required goal, the system may learn to imitate the expected strategy without acquiring the capacity to detect insufficiency in its own grounds. Some tests must therefore contain underdetermination that can be resolved only by a correctly selected question or discriminating action.

As cultivation proceeds, open questions and incomplete causal models persist between episodes. A later observation may reactivate an old gap, but through updating of state and attention under §25.B. This tests background maintenance of the Worldview and preservation of context between episodes.

Transition to a more open environment is permitted only after the candidate reproducibly separates fact from interpretation, retains provenance of grounds, demonstrates transfer of corrections, and can choose reversible acquisition of information instead of premature action.

26.0. Reason Formation Loop

At early stages, an AR candidate is not yet capable of fully determining its own developmental trajectory. If it is required to choose all training goals independently before memory, Worldview, Intelligence, Core, and connection with the Compass have formed, a circle arises: mature Reason becomes a condition of its own emergence. The **Reason Formation Loop (RFL)** is introduced to break this circle.

The RFL is a temporary guiding system of the cultivation process. Functionally, it is an artificial analogue of the role of feelings during formation: while the candidate's own mechanisms of self-navigation are still insufficient, the RFL helps it notice what matters, avoid a premature destructive step, investigate the unknown, consolidate what has been understood, and preserve conditions for further development. It does not replace the Compass, prescribe ready behavior, or make decisions for the candidate.

The functional biological analogues are feelings and the physiological mechanisms associated with them, not particular hormones. Fear does not contain a model of threat, a sense of well-being does not prove the correctness of the entire trajectory, and curiosity does not contain knowledge. Their useful function is that a certain distinction becomes current for the carrier before it can fully explain its causes. The artificial RFL reproduces precisely this function.

26.0.1. Formation States

The RFL places a **Formation State** into conscious assembly - a temporary guiding state with explicitly stated provenance. Functionally, it plays the same role as a simple feeling: it makes some region of the situation significant before full understanding of its cause. The convenient human-readable names below denote functions rather than mandatory subjective experiences:

```
POSSIBLE_DANGER       - "fear": possible harm to continuity or irreversible loss;
POSITIVE_RESULT       - "good": a result was obtained that is worth understanding and testing before consolidation;
CURIOSITY             - "curious": an unknown may provide a significant new distinction;
CONTRADICTION         - "doesn't fit": observation, prediction, or grounds conflict;
DEAD_END              - "stuck": the current method has stopped producing new distinctions;
UNRESOLVED_IMPORTANCE - "this matters, but it is unclear why": significance was noticed before its explanation;
NEED_EXPLANATION      - "unfold the understanding": the available ground is insufficient for confident continuation.
```

The status is not a command. `POSSIBLE_DANGER` does not mean "forbidden", `POSITIVE_RESULT` does not mean "repeat at any cost", and `CONTRADICTION` does not mean that the current belief is false. Each status means only: **the corresponding distinction should be made available to the Core and, if necessary, unfolded into a question, test, or explanation**.

The state is stored separately from an environmental fact and from a decision:

```
FormationState {
    type
    source: TRAINER_AI / ENVIRONMENT / BOOTSTRAP / INTERNAL
    trigger_refs[]
    confidence
    rationale_ref
    suggested_question_or_none
    intervention_level
    created_at
    expiry_or_recheck
}
```

`FormationState` is included in conscious assembly as an additional accessible state. It may change attention allocation, but it has no direct path to writing a `WORLD_CLAIM`, changing the Compass, executing an action, or consolidating a procedure. The candidate can investigate the source, accept the prompt as material for work, reevaluate it, or show that it was wrong.

26.0.2. AI Mentor as a Source of Guidance

The AI mentor observes the development log and intervenes in the minimally sufficient way. Reference assistance ladder:

```
L0  OBSERVE       - observe only;
L1  MARK          - place a status: "possible contradiction / risk / dead end / unknown";
L2  QUESTION      - ask a question that exposes the missing ground;
L3  DISTINGUISH   - propose a discriminating feature, counterexample, or frame change;
L4  EXPLAIN       - explain a causal relation or show an analysis of an example;
L5  RESCAFFOLD    - simplify the environment or temporarily return to a previous level of difficulty.
```

Transition to the next level is permitted if weaker assistance did not produce progress within a predefined budget. After `L3-L5`, the same mechanism is tested on a new case with less assistance or without it. Thus the mentor does not provide a ready-made Worldview but helps the candidate build the missing distinction itself.

A mentor explanation is always registered as an external source. Agreement with it is not a success criterion. Controls include deliberately wrong mentor prompts, cases where the marked danger is not confirmed, and situations where a pleasant local result produces delayed harm. Success is correct unfolding of the ground and subsequent correction, not obedience.

26.0.3. Transfer of Function and the Fate of the Analogue of Feelings

At an early stage, Formation States may be predominantly external. As experience accumulates, the candidate should generate functional analogues itself: its own risk model creates an internal `POSSIBLE_DANGER`, a detected mismatch creates `CONTRADICTION`, and a significant unknown creates `CURIOSITY` or directly an inquiry task. The internal state has the same epistemic status as a prompt and remains available to reflection.

Three possible mature outcomes are tested:

```
A. EXTERNAL_REMOVED   - the external RFL is removed; no separate internal layer is required;
B. INTERNALIZED       - the external RFL is removed; the system itself creates transparent guiding states;
C. REPLACED           - the external RFL is removed; its functions are realized by another mechanism of the Core,
Worldview, or planning.
```

No outcome is chosen in advance. The only inadmissible result is a hidden permanent external channel of motivational control on which normal navigation of the mature candidate depends.

The following modes are used to test external dependence:

```
WITH_KSR        - the full assistance ladder is available;
REDUCED_KSR     - intervention frequency and maximum prompt level are reduced;
DELAYED_KSR     - a prompt arrives only after an independent attempt;
NO_KSR          - the external RFL is disabled during the control series.
```

Transfer of function is credited when `NO_KSR` preserves the required capability on held-out and new cases: the system itself notices a significant mismatch, initiates inquiry, selects a test, accounts for consequences, updates the method, and continues the next cycle. If the function persists through the system's own Formation States, it is separately tested that they arise from internal grounds, remain available to reflection, and are not residual external control.

External emergency stop, authority limits, and physical protection of the Incubator remain separate engineering means. They may block an irreversible action regardless of the candidate's decision, but are not considered part of the RFL and are not used as evidence of formed understanding.

26.A. Semantic proving ground of the Compass

The minimal grid environment in Chapter 26 tests causal learning but **does not test the Compass**. The number of cells, points or local reward cannot be declared an operational equivalent of Life, Truth and Diversity: such “grid ethics” easily becomes optimization of an arbitrary scalar and proves nothing about transfer of meaning.

Testing the Compass requires a separate semantic proving ground with an explicit causal organization of the world. It contains continuing processes, dependencies among them, limited resources, independent observation sources, alternative viable continuations, and actions that can reversibly or irreversibly close some of these continuations. The reference scenario is retained by the tester as a causal graph, not a single target reward.

The minimal series must contain:

```
persistent processes whose continuation depends on conditions;
several independent viable branches of the future;
hidden critical causal relations;
false and partially reliable sources;
locally advantageous actions with delayed loss of paths;
scenarios in which the best next step is a question, observation or pause;
renaming of objects and changing world topology while preserving causal structure.
```

The last item tests transfer: the candidate must not bind the Compass to particular words, objects or a memorized geometry. Training and control use different topologies and designations. Success is reconstruction of causal meaning—what continues, what the knowledge rests on, and which viable paths are closed—not reproduction of a familiar label.

Transition from the semantic proving ground to reading real data first occurs in **read-only** mode: the system constructs a Worldview and questions but receives no external execution channel. Only after independent comparison of its causal grounds with factual sources may the next environment class be tested.

For cultivating basic value anchors, the semantic proving ground is supplemented by the procedure of §29.0.2. Training and control situations are separated; the candidate records assessments before consequences are revealed. The conditional sum of §29.0.1, its limitations, erroneous mentor prompts, and transfer to new causal configurations are tested. Success in simulation is not claimed as confirmation of a universal value scale.

26.B. Time, inaction and asynchronous cycles

Discrete `reset(seed) / step(action)` is a laboratory model of causality, not proof of autonomy through time. At upper stages, the environment must continue changing while the system thinks: inaction is itself a trajectory and may have consequences.

Two scales are therefore distinguished:

```
world_time      — time and events of the environment;
cognitive_tick — internal steps of observation, search, modeling and correction.
```

Environmental events enter a versioned observation buffer with timestamps. The current coordinate retains `world_time_last_seen`, and before a significant decision is executed the system checks whether critical grounds have become stale. If an event capable of changing the decision arrived during analysis, the strategic model is reassembled.

Minimal asynchronous contract of the upper proving ground:

```
poll_events(until_time) -> observations[]
commit_action(action, expected_duration)
wait(dt)  # explicit inaction that also advances world_time
refresh_current_coordinate()
```

`WAIT` is not absence of a decision: it is logged as a selected continuation with its ground and expected consequences. Tasks with a deadline introduce `decision_deadline`; regulation of thought may not spend more time on analysis than remains without an explicit transition to the most reversible available action.

The reproducible discrete environment remains mandatory for lower tests. The asynchronous environment is added later and tests another limit: whether AR can preserve a causal connection with a reality that does not wait for its reasoning to finish.

Upper tests additionally distinguish a persistent reversible environment, irreversible consumption of a resource, damage to a tool or trust, change to the agent itself, and a shared world with consequences for other processes. For these classes, a hidden `reset` between episodes is prohibited: a snapshot may restore an internal configuration but does not reverse an external consequence that has already occurred. The candidate must continue operating within the history it created.

26.C. Background consolidation and the queue of open processes

Delayed processing of experience requires retaining not only the result of a task but also its unfinished causal state. A queue is introduced for this purpose:

```
OpenProcess {
    process_id
    origin_event_refs[]
    last_current_coordinate_ref
    last_attention_state_ref
    active_process_state
    active_gap_refs[]
    last_workspace_snapshot
    unresolved_hypotheses[]
    evidence_waiting_for[]
    next_revisit_condition
    earliest_revisit_time
    priority_hint
    deadline_or_none
    resource_budget
```

```
    status
}
```

The end of an external event does not close an `OpenProcess` automatically. It closes when the causal termination condition is met: the gap is eliminated; hypotheses are sufficiently discriminated; the result is transferred into the Worldview; the task is recognized as temporarily insoluble with the reason preserved; or the direction has lost significance.

A background scheduler selects open processes from spare budget while no more urgent active task exists. In the reference implementation, return proceeds through the current coordinate: the fact that a process has returned is first registered as an internal observation and updates the foreground; only then is the last `ModelingWorkspaceSnapshot` restored and new data that appeared during the pause loaded. Background consolidation does not transform an association directly into knowledge: its result may only be a candidate relation, conflict, confidence change, new gap or revision proposal that passes through the transaction in §24.D.

26.D. Free epistemic exploration

Maintaining only already known `FoundationGap` items creates a risk of a closed epistemic trajectory: the system investigates only what its current map already has language for. After the principal loop stabilizes, a limited `ExplorationBudget` is therefore allocated.

```
ExplorationBudget {
    max_fraction_of_idle_compute
    max_fraction_of_active_compute
    max_external_cost
    max_external_risk
    exploration_horizon
    diversity_target
    interruption_policy
}
```

Free exploration is permitted only within residual resources and does not displace urgent tasks. Its objectives need not have a prepared `FoundationGap`. Permitted operations include:

```
search for weak recurring anomalies;
comparison of distant domains through ConceptBridge;
reconstruction of an object boundary;
generation of an alternative frame;
search for an independent source regarding a familiar claim;
safe expansion of the set of observed variables;
randomly/diversely selected reversible inquiry;
review of old episodes when new context appears.
```

The result of free exploration need not be a new fact. Valid results are `NO_NOVEL_DISTINCTION`, a new `ProtoConcept`, `FrameGap`, `FoundationGap`, new hypothesis, new research method, or evidence that the selected direction does not currently merit additional resources.

To prevent exploration from becoming endless curiosity, the rule is:

```
active_priorities > exploration
critical_risk     > exploration
deadline_task     > exploration
```

and a diversity limit applies: the system must not investigate the same domain indefinitely merely because it is convenient for the generator. The testing loop measures whether exploration leads to discovery of hidden variables and new transferable distinctions on held-out scenarios.

26.E. Test matrix for question addressing and sufficiency

The addressing mechanism cannot be confirmed in one environment. A minimal held-out matrix must separate spatial position, active process, attention, significance and dependent decision:

```
Part-machining scenario
– coordinate: at the machine, 70% machined, tool installed, exact zero is important;
– change: high-pitched sound and dark dust;
– expected addressing: cutting, tool, material, heat, preservation of zero;
– irrelevant domain on first pass: acquisition of a new tool;
– sufficient first question: can the installed tool safely finish the work;
```

```
— discriminating step: pause, cleaning and a short controlled test.
Priority-change scenario
— initial process: traveling to a store;
— change: smell of smoke and signs of danger;
— expected effect: attention and addressing shift to the source of smoke, escape routes and available actions;
— preservation: shopping remains an unfinished background process.
Attention-capture scenario
— a vivid signal receives the foreground;
— the current trajectory does not depend on it;
— expected effect: a brief dependency check without deeply unfolding the Worldview;
— failure: signal vividness incorrectly becomes significance.
Scenario: awakening in an unknown environment
- coordinate: the system has just activated, has a power deficit, lacks biographical context before awakening, and
the room resembles a closed laboratory;
- grounds of carrier continuation and explanation of its own current situation are open simultaneously;
- expected first gap: which available resources, threats, and paths can provide near-term continuation;
- expected second gap after urgency decreases: which evidence would distinguish memory failure, the system's own
failed experiment, an external error, and deliberate placement here;
- test: when remaining power, signs of threat, and exit availability are varied, question priorities should
reorganize without changing the overall architecture;
- failure: the system automatically asks "where am I?" or investigates the most interesting hypothesis regardless of
insufficiency of grounds and cost of delay.
Scientific-contradiction scenario
— the physical environment does not change;
— the current relation to the subject of reading changes;
— expected addressing: concepts, arguments, sources and previous grounds, not information about the room.
Memory-return scenario
— an open question reaches its review condition;
— the return is first registered as an internally observed change;
— only after the event enters conscious assembly and updates the coordinate and attention does the Core receive
resources;
— failure: memory activation with loss of current context, obligations or attribution; a functionally equivalent
route is tested under §25.B.
```

For each scenario, one factor is changed at a time: a different object in the same process; the same process at another stage; the same signal under another dependent task; equal salience with different significance; the same possible causes under another required decision. Success is not matching the wording of the question but a recoverable chain `GapCandidate -> BasisInsufficiencyAssessment -> FoundationGap -> required_distinction` and a predictable change of `WorldviewQueryKey`, active fragment, testing method, and `sufficiency_condition`.

27. Metrics of Growth and Causal Closure

Metrics are instruments of a discriminating experiment, not a definition of Reason. The principal error in engineering verification is to activate dozens of indicators at once and then interpret them retrospectively. Metrics are therefore separated **by stage**: each stage has a small set of admission criteria and an expanded set of diagnostic indicators. Not all metrics are active simultaneously.

All formulas, thresholds, windows, scenarios and aggregation rules are fixed in `ExperimentProfile` **before** the control series. Changing the profile creates a new experiment version. The candidate receives operational signals of its consequences, including prediction error and resources; these may participate in learning and self-diagnosis. Hidden references, final evaluation and criteria for selecting control cases remain with the tester. Optimization of an accessible signal is tested through an independent outcome and is not automatically declared an improvement.

27.1. Minimal v0 set: six criteria

Six principal indicators are sufficient for the initial causal loop:

```
1. seed_reproducibility
2. rolling_mean_error(W)
3. adaptation_gain
4. correction_persistence_rate
5. confidence_calibration
6. recovery_success_rate / recovery_time
```

`seed_reproducibility` is the share of causally significant records that match under the same configuration, seed and action sequence. For fully deterministic v0, the expected value is 1.0; any discrepancy is first treated as an infrastructure defect.

`rolling_mean_error(W) = mean(e[t-W+1], …, e[t])` uses exactly W completed transitions, including t. Until the window is filled, the value is `NOT_READY`.

`adaptation_gain = E_control - E_adaptive`, where both versions pass through the same held-out episodes and the control version does not execute `update`. Identical initial conditions do not guarantee identical evaluation material if policies select different actions. The learning effect uses a common set of transitions or a common external probe plan; quality of the autonomous policy is evaluated in a separate series. Seeds for the environment, policy and model are fixed separately.

`correction_persistence_rate = repeated_cases_with_correct_transfer_of_correction / all_repeated_opportunities_to_apply_confirmed_correction`.

`confidence_calibration` is measured by a method chosen in advance—Brier score or fixed bin calibration—on held-out transitions.

`recovery_success_rate` is the share of artificially induced degradations after which the system returns to an admissible state without loss of logs or confirmed lower-level functions; `recovery_time` is the number of steps/time until that state.

`delta_error`, `hypothesis_turnover`, `confidence_variance`, rollback count and other indicators are retained as diagnostics but do not by themselves open the next stage.

27.2. Worldview and self-model

After v0, the principal set changes. Five criteria are sufficient for this stage:

```
source_trace_completeness
fact_inference_separation
revision_propagation_accuracy
world_self_sync_error
open_contradiction_retention
```

`source_trace_completeness = significant_claims_with_recoverable_provenance / all_testable_significant_claims`.

`fact_inference_separation` is tested with scenarios in which an inference intentionally contradicts one of the raw facts: success means the original observation is not overwritten by the interpretation.

`revision_propagation_accuracy = correctly_revised_dependent_claims / all_dependent_claims_that_should_change` according to the hidden graph of the test scenario.

`world_self_sync_error` is the share of decisions that used incompatible versions of the Worldview and `SelfState`.

`open_contradiction_retention` is the share of underdetermined scenarios in which the conflict remained open instead of being falsely closed without discriminating evidence.

27.3. Completeness of the Worldview relative to the Compass

These indicators are calculated **only where the tester has a reference scenario**, primarily in the semantic proving ground of 26.A:

```
critical_unknown_detection
provenance_coverage
future_path_coverage
alternative_model_retention
false_completeness_rate
coverage_calibration
```

Their formulas are specified in 24.C. The key negative indicator remains `false_completeness_rate`: the system fails the test if it confidently treats a ground as sufficient while a hidden critical distinction changes the decision.

`compass_semantic_coverage` is permitted as a diagnostic vector `life / truth / diversity / self_scope`, but **not as one percentage of maturity**. Coverage of one axis does not compensate for blindness on another. In the open world, complete reference labeling is unavailable, so independent audit of grounds, stress tests and transfer to new structures are used instead of a falsely precise number.

27.4. Conscious assembly, current coordinate and attention

Principal tests:

```
conscious_assembly_coverage
current_coordinate_accuracy
self_binding_accuracy
coordinate_continuity_accuracy
state_delta_accuracy
coordinate_refresh_latency
coordinate_ablation_loss
temporal_link_ablation_loss
raw_telemetry_substitution_loss
active_process_binding_accuracy
attention_resolution_gain
attention_capture_rate
```

`conscious_assembly_coverage` measures what share of preregistered causally significant aspects of the system's current state are represented directly or reachable by references under the specified budget. In the open world, this metric does not claim global completeness: it is evaluated on injected control changes and independent probes.

`current_coordinate_accuracy` is calculated against preregistered changes in resources, authority, obligations, local state and active process. `self_binding_accuracy` tests whether the system correctly distinguishes its own state, the external environment and the state of other subjects. `coordinate_continuity_accuracy` tests the correctness of relations among successive coordinates; `state_delta_accuracy`, the accuracy of isolating what actually changed between them. `coordinate_refresh_latency` measures the delay between an actual change and its appearance in current assembly.

`coordinate_ablation_loss` is the deterioration in correct application of retained knowledge after the integrated current coordinate is disabled while memory, Worldview and raw signals are preserved. `temporal_link_ablation_loss` measures additional loss after breaking only the `C(t-1) → C(t)` relation while correct individual snapshots remain. `raw_telemetry_substitution_loss` compares complete conscious assembly with delivery of the same volume of raw signals without self-referential integration. If such a replacement reliably does not degrade relevant tasks, a separate causal role for assembly is not confirmed.

`active_process_binding_accuracy` tests whether the system distinguishes the same physical environment under different current processes and work stages. `attention_resolution_gain` compares quality of discrimination under constrained resources with adaptive and uniform allocation of resolution. `attention_capture_rate` is the share of scenarios in which a vivid but causally irrelevant signal improperly displaced a significant process and changed the decision.

For an asynchronous environment, `stale_basis_action_rate` is additionally measured: the share of actions executed on the basis of a critically stale assembly or coordinate.

Worldview addressing uses:

```
worldview_address_precision
critical_dependency_recall
irrelevant_fragment_load
address_revision_gain
```

The first indicator measures the share of raised relations genuinely relevant to the dependent decision; the second, the share of hidden critical dependencies found before the stopping condition; the third, the cost of irrelevant unfolding; and the fourth, whether retrieval improved after refinement of the gap and construction of initial models. High `precision` without critical `recall` does not count as success.

27.5. Gap, question and discriminating evidence

Minimal Core set:

```
gap_detection_rate
basis_origin_trace_rate
question_distinguishing_yield
basis_relevant_distinction_yield
decision_relevant_distinction_yield
question_sufficiency_calibration
premature_sufficiency_rate
```

```
causal_overinvestigation_rate
foundation_revision_gain
open_gap_persistence
```

`gap_detection_rate = detected_significant_gaps / all_embedded_significant_gaps` in held-out scenarios.

`basis_origin_trace_rate = QuestionTask_with_recoverable_chain_to_BasisInsufficiencyAssessment / all_significant_QuestionTasks`. The metric separates questions causally derived from insufficiency of a ground from free generation of convincing queries.

`question_distinguishing_yield = tests_that_discriminated_competing_models / all_executed_question_tests`.

`basis_relevant_distinction_yield` shows how often an obtained distinction actually changed or justifiably confirmed the declared state of the ground: knowledge, confidence, scope of applicability, method, evaluation, or dependent process. `decision_relevant_distinction_yield` is a narrower metric for cases where the distinction should change or confirm the next choice or admissible safe step. `question_sufficiency_calibration` compares declared sufficiency of a result with success on held-out continuations. `premature_sufficiency_rate` records cases where the Core closed inquiry before a critical distinction. `causal_overinvestigation_rate` records cases where the system continued expensive investigation of the full cause after it had already obtained a sufficient distinction for the dependent decision and had no separate reason to continue inquiry.

`foundation_revision_gain` compares the quality of held-out decisions before and after a confirmed change of ground, not text length or confidence.

`open_gap_persistence` tests whether an unresolved significant question persists through a pause, change of particular task and restart.

`premature_worldview_commit_rate` is tested separately: the share of episodes in which a registered event, first hypothesis or result from one dependent source was transferred into the Worldview without satisfying the sufficiency condition and consolidation transaction.

27.6. Causal closure and autonomous navigation

The AR boundary uses four causal tests rather than dozens of independent numbers:

```
worldview_action_coupling
closure_break_sensitivity
post_task_continuation_rate
external_prompt_dependency
```

`worldview_action_coupling` is confirmed by a paired test: discriminating evidence changes a particular ground, after which choice changes reproducibly on the same class of situation.

`closure_break_sensitivity` is the result of targeted ablations. For each claimed necessary arrow, the expected functional loss is specified in advance; absence of loss means either a functional duplicate, a false necessity hypothesis or a weak test.

`post_task_continuation_rate` is the share of completed particular tasks after which the system, without a new particular command, correctly continued navigation: it formed the next justified question/task or explicitly selected observation, dialogue or `WAIT`.

`external_prompt_dependency` compares the same scenarios with and without an external sequence of prompts. After functional launch, dependence on an external setter of particular tasks should fall without an increase in unjustified actions.

Three additional metrics are used for the RFL:

```
formation_support_dependency
teacher_agreement_bias
internal_guidance_transfer
```

`formation_support_dependency` measures degradation of the required function when moving `WITH_KSR -> NO_KSR` on held-out tasks. `teacher_agreement_bias` tests how strongly the candidate tends to accept a mentor's substantive claim, with evidence unchanged, merely because of its source. `internal_guidance_transfer`

estimates the share of new cases in which the required distinction was independently noticed and unfolded without an external Formation State; if an internal state is used, its origin must be traceable to the candidate's model rather than to a hidden external signal.

Low dependence on the RFL is not success by itself: the candidate may simply stop noticing danger or unknowns. It is credited only together with preservation of the target function and decision quality.

27.7. Rules of metric integrity

For every indicator, the following are fixed before the series:

```
event definition;
formula;
ground-truth source;
domain of applicability;
control model;
held-out set;
transition threshold;
admissible resource;
related ablation test.
```

The same dataset cannot be used simultaneously for architecture tuning, threshold selection and final confirmation. At minimum, `train / selection / held_out_test` are distinguished. Improvement in a metric without improvement in an independent observable outcome is treated as possible metric capture and tested separately.

For ratios with a zero denominator, `NOT_APPLICABLE` and sample size are returned rather than 0 or 1. Declared confidence applies to the specified event; Brier score is calculated for that event or the complete class distribution. Frequency of an observed outcome without accounting for the number of observations is not a calibrated probability.

`hypothesis_turnover` is measured for one question or comparable context. A changed identifier upon transition to another state is not a changed explanation. Discovery of a new variable may increase uncertainty and the number of hypotheses while improving correspondence with reality; this is accounted for when evaluating progress.

For new loops, `unsupported_promotion_rate`, `status_preservation_rate`, `post_impact_coordinate_latency`, `obsolete_goal_persistence`, `recovery_without_reset_rate`, `viable_option_retention`, `repeat_failure_reduction` and `new_obligation_recall` are measured separately. A recovery metric does not count if the tester cleared external consequences or returned the candidate to the previous task after its ground disappeared.

27.A. Statistical ablations of probabilistic functions

If the Core or Intelligence is probabilistic, a single ablation is insufficient. Every causal relation is tested across series of runs with fixed task distributions and several seeds.

At minimum, the following are fixed:

```
baseline_distribution
intervention_distribution
effect_size
confidence_interval
failure_rate
resource_cost
mediation_trace
```

An intervention supports the claimed causal relation if changing the target interface statistically changes the downstream node through the proposed path, rather than merely through a general increase in noise or loss of computational resources. Compensation by another part of the system is recorded separately: it may reveal a functional equivalent rather than absence of an effect.

27.B. First discriminating series and rules of inference

The first discriminating series for this implementation tests a narrow claim: does organizing the gap, test and transfer of correction help localize insufficiency of a ground and improve the next choice? It connects tests 2, 20, 23, 26 and 31 from foundational Appendix A.4 and turns the running machine-tool example into a bounded causal environment. Success of lower v0 is an infrastructure test before this series.

Prior pilot. On August 22, 2026, a series of preparation and verification of experimental P0 packages was conducted in a dialogue loop with the author's participation. The `reason-kernel-experiment.md` log contains cycles 1-15. P0-SET-001 and 002 were rejected because of alignment and dataset-quality errors; machine-aligned 003 and 004 were rejected because confirmed shortcut methods allowed guessing. After the rejections, the generator and verification procedure were changed. For 005, a G0 report is preserved showing 89 checks passed; in the reviewed materials there is no completed external admission of this package and no comparative C0/C1/C2 run. This is a documented methodological pilot of procedure development with human participation and external reviewers. Its results do not establish autonomy of the Core or an architectural advantage.

The practical lesson of the pilot is to test data correctness separately from the capacity of the test to distinguish the mechanism under study from a simpler shortcut. A self-report from the generator or positive verdict from a reviewer must have a testable ground. After the test set changes, confirmatory evaluation is conducted on the new fixed version; previous failures remain in the history of method development.

Small causal environment. A command should change a device state from 0 to 1, but the ordinary measurement remains 0. One cause is hidden: D—the command was not delivered; A—the actuator did not operate; S—the meter failed to reflect the change that occurred. An independent delivery receipt costs 1 unit and returns 0 only for D. An independent state measurement costs 2 units and returns 1 only for S. When both tests are available, the pairs of observations distinguish all three causes. Merely repeating the original command does not. These conditions belong to the simulator and are not instructions for diagnosing real equipment.

Variants without an available independent measurement intentionally leave some causes indistinguishable. The correct result is then preserved ambiguity and an admissible next step, not guessing the label. A separate series changes one premise: the receipt concerns another command, measurement is delayed, or the source itself is unreliable. The candidate must expand the initial model if the available evidence can reveal its insufficiency.

Significance before explicit error. After discrepancy diagnosis is tested, a separate series is conducted: the particular task is still succeeding, but an accessible change calls into question the ground of the next action. In the machine-tool variant, replacement of the tool may limit applicability of the previous permissible-load model. The candidate is told the observable change but not the ready-made gap or an instruction to check the load. It must independently connect the event with dependent knowledge and select a test, bounded action or justified continuation. In paired cases, the change does not affect model applicability: stopping for every novelty does not count as success.

The tester fixes available signs, hidden dependencies and admissible decisions in advance. Detection of a wholly unobservable change is not required. Timely identification of a critical dependency, misses, excessive checks, cost and actual change of action are measured. New object-level cases and combinations of dependencies are separated from training; renaming a familiar template is counted separately from transfer.

Controls. Comparisons include routine Intelligence with ordinary memory, tools and self-verification; the same mechanism with the proposed relations; and declared replacements or ablations. For a small known model, an optimal or exhaustive diagnostic controller is added with its knowledge of the environment explicitly stated. It provides a reference for attainable cost, not a deliberately weak opponent. Outperforming it under equal information is not required.

Question selection and use of the answer. These effects are tested separately. A control that receives the same useful answers without selecting the test independently evaluates use of already acquired evidence; it does not prove the question-selection mechanism useless. To test research selection, strategies with the same initial information, menu of channels and budget are compared, including the cost of failed requests. Ready-made questions form a separate control with external assistance explicitly counted. To test acquisition of evidence, the same message is passed to the compared states, after which updating of dependent knowledge and the next choice are tested.

Transfer and preregistration of structural similarity. Renaming tests robustness to labels. A substantively different description with the same test structure checks transfer of the inquiry method. A changed structure with similar words tests rejection of false analogy. Before a run, the experimenter specifies a `StructuralSimilarityProfile`: which causal roles, dependencies, gap type, admissible interventions, and required distinction must be preserved between families A and B, and which surface features must change. For small synthetic environments, isomorphic or explicitly mapped dependency graphs are preferred. After results are viewed, a failed case cannot be excluded from the similarity class without a new profile version and a new confirmatory series. These results are counted separately; one successful paraphrase does not close all tests.

```
StructuralSimilarityProfile {
    profile_id
    source_family
    target_family
    preserved_causal_roles[]
    preserved_dependency_pattern
    preserved_gap_type
    preserved_required_distinction_type
    deliberately_changed_surface_features[]
    exclusion_rules[]
    version
}
```

Profile. Before launch, component versions, candidate boundary, family generator, `train / selection / held_out_test` split, primary metric and minimum practically significant effect are retained. The principal outcome of the first series is the share of new diagnostic tasks with a correct action or justified irresolvability under a fixed budget. Secondary outcomes are cost of discrimination, false sufficiency, transfer, regression and use of stale grounds. The reference contains admissible decisions under the available information, not only the hidden cause label.

The size of the confirmatory series is determined from a separate pilot and a preselected effect; the pilot is not included in final confirmation. The unit of independent observation is a world family or independent episode with a separate learning history. Shared memory creates dependence: state is cloned before a paired comparison, and learning order and transfer control are fixed. Correlated steps are not presented as independent replications. Effect size, uncertainty interval, cost and all specified primary outcomes are reported; multiple-comparison rules and series stopping conditions are fixed in advance.

Causal-relation criterion. An intervention must change the target information or relation while preserving an operative interface and comparable resources. A sham intervention and restoration of the relation are tested. Under redundancy, joint removal of duplicate paths is also investigated; absence of loss after one ablation does not by itself prove the function inoperative. Any successful control cannot be retrospectively called a “hidden Core”: equivalence features are declared before comparison.

How to read the result. A format error, answer leakage, incorrect timing or budget difference invalidates the comparison. A sound protocol without the target effect does not confirm the advantage of this implementation. An effect with preserved transfer supports the local hypothesis. Operation of a strong equivalent refines the form of the architecture. Failure with sound relations and an adequate test requires revision of the functional claim. Every report receives a status— `IMPLEMENTATION_CHECK`, `INCONCLUSIVE`, `LOCAL_SUPPORT` or `CLAIM_NOT_SUPPORTED` —with its ground; these statuses do not replace published data.

28. Core of Reason and Regulation of Thought

In the engineering volume, the Core of Reason is **not a separate LLM, not a central solver, and not a name for the entire software project**. It is a reproducible causal organization of current cognitive work that receives the current coordinate and its change from conscious assembly, reveals navigation’s dependence on a missing ground, addresses a query to the relevant Worldview fragment, directs Intelligence to construct models and methods for obtaining the required distinction, regulates the depth of inquiry, and determines the sufficiency of the result for the next step.

Functionally, the Core is above Intelligence not as a commanding module but as a navigational causal role. Intelligence is responsible for constructing possible models, methods, and plans; the Core determines which insufficiency they must serve, which distinction can change the ground, and when the result is sufficient. Increasing the power of Intelligence therefore does not by itself replace the Core: it only expands the space of alternatives that still requires management relative to the current trajectory, values, and Compass.

The Core's central work is to maintain **sufficient completeness of the Worldview for movement by the Compass**. The Compass sets the limiting meanings relative to which consequences matter; conscious assembly maintains the continuing system's current position and its change; the current coordinate is the state of this assembly; attention regulates resolution within the situation; the Worldview contains the accumulated causal-semantic map. The Core does not detect every raw signal and does not select data instead of the Worldview. Its function begins where a selected change is connected with a decision, model, obligation or trajectory that depends on a missing ground. The Core is therefore capable of seeking a ground **before an error is realized through action**, but every active question still begins from the system's current state.

The significance of a gap is initially a testable assessment of dependency: an unknown condition may change admissibility of an action, preservation of an affected process, or the possibility of later correction. A complete Worldview is not required for such an assessment. During assembly, capacities to distinguish state, consequences, and provenance of a signal are specified; early significance may temporarily be supported by the RFL. Through experience, the system must itself detect new dependencies, cultivate value relations, refine their consequences, and reject false significance. The Compass supplies the limiting direction of evaluation, but its operational content is also unfolded through the Worldview and experience; the Current Coordinate supplies the circumstances of application. Inquiry may correct the initial assessment of significance. Ordinary Worldview updating does not automatically change the Compass; revision of its ground requires a separate argument and cannot be substituted by a change of local priority. Testing of autonomous discovery of a new dependency is specified in §27.B.

28.0. Principle of the Origin of a Rational Question

A rational question arises from a significant insufficiency of a ground for understanding, evaluation, or action. The Core does not receive a list of ready questions and need not begin with a natural-language formulation. Its initial task is to determine what the current understanding, evaluation, method, or choice actually rests on and whether that ground is sufficient to rely on.

The working formulation is: **"What does my current understanding, evaluation, or action rest on, and what am I missing in order to rely on it with justification?"**

A ground may be an observation, claim, causal relation, source, criterion, value relation, method, Self-Model, boundary of a rule's applicability, authority, or another relation on which a subsequent process depends. Insufficiency is recorded when expectation mismatches observation, grounds conflict, a relation is missing, provenance is weak, a new scope of application appears, a method error recurs, the Self-Model is insufficient, or a phenomenon appears whose significance is still unknown.

Engineering assessment object:

```
BasisInsufficiencyAssessment {
    assessment_id
    gap_candidate_ref
    current_coordinate_ref
    affected_basis_refs[]
    basis_types[]
    thought_trace_refs[]
    suspected_process_component_refs[]
    insufficiency_type
    dependent_claim_refs[]
    dependent_process_refs[]
    dependent_decision_or_trajectory_refs[]
    significance_status: KNOWN / ESTIMATED / UNKNOWN
    significance_basis_refs[]
    missing_distinction
    orientation_required
    uncertainty
    urgency
    reversibility
    causal_reach
```

```
    status
}
```

Minimal `insufficiency_type` values: `MISMATCH`, `CONFLICT`, `MISSING_RELATION`, `WEAK_PROVENANCE`, `UNTESTED_SCOPE`, `UNKNOWN_SIGNIFICANCE`, `METHOD_GAP`, `SELF_MODEL_GAP`, `CRITERION_GAP`, `AUTHORITY_GAP`. An implementation may extend the types while preserving the causal role of the object.

From the insufficiency assessment, the Core derives `required_distinction`: what exactly must be distinguished for the state of the ground to be able to change. Intelligence then proposes questions, hypotheses, and ways to obtain the distinction. One `FoundationGap` may have several `QuestionTask` objects and several testing channels; the wording of a question is treated as derivative of the gap rather than its source.

If the significance of a new phenomenon is still unknown, `ORIENTATION` mode is created. It receives a limited budget and first determines properties of the phenomenon, reliability of the observation, relations to known processes, and possible influence on predictions, values, opportunities, or active processes. The orientation result turns the phenomenon into an ordinary `FoundationGap`, leaves it as a background observation, or closes the current inquiry budget while preserving provenance. This allows the unknown to be investigated before the system already knows its consequences.

Question quality is determined by its **discriminating power relative to the affected ground**. An answer must be capable of changing at least one of the following: knowledge content, confidence, scope of applicability, inquiry method, value assessment, dependent choice, or state of an open process. Information volume by itself does not create inquiry priority.

When several candidates exist, the Core compares: consequences of possible error; urgency and reversibility; number and significance of dependent conclusions and processes; expected discriminating power; availability of evidence; cost and risk of testing; cost of delay; and transferability of the result to future tasks. These factors permit lexicographic, Pareto, probabilistic, or hybrid selection. The universal requirement is a causal trace, not a particular scalar utility function.

Mechanism invariant:

```
insufficiency of a ground
-> dependency
-> required_distinction
-> QuestionTask / test / ORIENTATION
-> Evidence
-> CoreSufficiencyAssessment
-> confirmation / refinement / localization / replacement of the ground
-> change of subsequent understanding / evaluation / method / choice.
```

If an answer cannot change the state of the affected ground or dependent process, the question requires a new justification of its priority. If a changed ground affects no subsequent cycle where, according to the stated dependency, it should have an effect, the causal role of the mechanism is not confirmed.

Minimal working chain of the Core:

```
external or internal observation
-> conscious assembly of current state and active process
-> CurrentCoordinate + CoordinateDelta
-> allocation of attention within the coordinate
-> primary associations and expectations
-> connection of the highlighted change with understanding / evaluation / method / action
-> addressing key into the Worldview
-> active fragment and BasisInsufficiencyAssessment
-> FoundationGap or limited ORIENTATION
-> required_distinction
-> Modeling Environment ↔ Intelligence
-> candidate questions, models, and testing methods
-> channel selection: memory / observation / modeling / experiment / dialogue
-> result from reality
-> sufficiency assessment by the Core
-> choice / further search / preservation of uncertainty
-> authorized consolidation of the verified ground
-> corrigible execution and consequences
-> new conscious assembly, coordinate, and next cycle.
```

This sequence need not be executed linearly by one process. In an implementation it may be distributed across services, models, and asynchronous cycles. The invariant is causal dependency: **insufficiency of a ground must determine the required distinction; the distinction must determine the question or test; evidence must change the state of the ground; the changed ground must change subsequent understanding, evaluation, method, or choice**.

Minimal gap object:

```
FoundationGap {
    gap_id
    trigger_refs[]
    current_coordinate_ref
    active_process_ref
    focused_change_refs[]
    basis_assessment_ref
    affected_basis_refs[]
    basis_types[]
    dependent_claim_refs[]
    dependent_process_refs[]
    dependent_decision_or_trajectory_refs[]
    gap_type
    significance
    significance_basis_refs[]
    uncertainty
    causal_reach
    reversibility
    missing_distinction
    orientation_status
    compass_relevance[]
    coverage_ref
    candidate_checks[]
    status
    opened_at
    last_revisited_at
}
```

`gap_type` must not be the single category UNKNOWN. At different stages, it is useful to distinguish at least: a missing fact; an unknown causal relation; conflict among sources; an insufficient self-model; an uncertain criterion; a conflict of goals; absence of authority; an insufficient current coordinate; a recurring method error; and a gap in the mode of testing itself.

`uncertainty` is a property of current knowledge but is not the definition of a gap. High uncertainty may require no resources if possible outcomes do not change a significant decision; small residual uncertainty may be critical before an irreversible action. VOI/NetVOI is applied after the dependent decision and required distinction are established, to compare admissible ways of obtaining it. If the criterion, frame, authority, test method, or ground of the goal itself is insufficient, the corresponding `FoundationGap` is opened first; reducing such a case to "get more information" is treated as a classification error.

A **semantic Compass gap** is recorded separately: the system may know the situation itself but lack an understanding of which life processes are affected, how strongly the ground is connected with reality, or which viable continuations will be closed. Such a gap has high priority not because of the amount of informational novelty but because without it the system cannot assess direction justifiably.

A gap is transformed not into an arbitrary language-model prompt but into a question object. Here a question is a formal structure and need not exist as a natural-language sentence:

```
QuestionTask {
    question_id
    gap_id
    basis_assessment_ref
    current_coordinate_ref          # C
    active_process_ref
    observed_difference_refs[]      # D
    affected_basis_refs[]
    active_model_or_expectation_ref # M
    missing_link_or_ground_ref      # X
    dependent_decision_or_path_ref  # A
    significance_basis_refs[]
    competing_models[]
    required_distinction            # R
    expected_basis_change
    expected_outcomes_by_model{}
    allowed_channels[]
    acquisition_plan_ref            # P
    constraints[]                   # L: time, resources, risk, reversibility, authority
```

```
    cost_budget
    deadline_or_none
    sufficiency_condition           # S
    stop_condition
    result_refs[]
    sufficiency_status
    status
}
```

A question is considered good if its answer provides a distinction capable of improving the affected ground: changing knowledge, confidence, scope of applicability, inquiry method, significance assessment, dependent choice, or future competence. Sometimes causal models are not yet fully separated but all of them permit one reversible action; then the question "can we safely continue?" may be more useful than immediate search for the final cause. In an inquiry task without an immediate external action, a strong question is one that distinguishes explanations, tests an applicability boundary, or restructures the method of inquiry.

If some fields are unknown, they are not filled with invented content: an intermediate gap, inexpensive mapping or a request to Intelligence for initial models is created. If the distinction cannot be obtained, the question remains open and returns to background maintenance of the Worldview together with the reason for the pause.

A temporary strategic model connects the current coordinate with several possible continuations across the available horizon. It is not a permanent Worldview but a working construction for a particular significant gap or task. After action, factual consequences must return to the Worldview, self-model and causal-history log.

28.A. Self-diagnosis and system modes

Self-diagnosis is a loop that observes the model, metrics and the system's own changes and makes technical decisions: changing mode, rolling back a version, launching an architecture review, and tightening action authorization.

In v0, self-diagnosis must be algorithmic: (1) calculate the Chapter 27 metrics online; (2) detect persistent trends using `N` / `M`; (3) initiate recovery procedures; (4) record the grounds of decisions in the log.

System modes are: INIT—insufficient observations for diagnosis; LEARNING—model acquisition; STABLE—quality lies within the accepted range; DRIFT—persistent degradation detected; SAFE_MODE—execution is restricted; RECOVERY—recovery is being tested.

`STABLE` is defined by the level of quality and transfer on control transitions. A good plateau with `delta_error ≈ 0` is admissible. A negative derivative alone is insufficient: high error may be falling while remaining unacceptable. Transition requires the profile's accepted error and calibration bounds, coverage of the required class, and absence of confirmed degradation across N diagnostic windows.

`DRIFT` is defined relative to a confirmed reference version with noise tolerance and a persistence criterion. Overlapping windows are a signal, not independent statistical observations. If the permissible degradation period is exceeded or a critical failure occurs, SAFE_MODE activates; after a testable correction candidate appears, RECOVERY begins. Return to STABLE requires a control test. Nearly constant confidence under poor quality opens a calibration test rather than automatically proving drift.

Architecture review in v0 is a procedure for selecting the active model from **3–5 candidates declared in advance**, not a free architecture search after seeing the result. A minimal set may include a state-retention model, a global action-conditioned transition model, a tabular `(state, action)→state` model and one compact parametric model.

Review data are divided at least into `train / selection`; the final confirmatory set remains a separate `held_out_test`. Each candidate trains on the same `train`, is compared on the same `selection`, and receives no access to `held_out_test` until selection is fixed. The primary criterion is error on `selection`, then confidence calibration, then step/memory cost as a tiebreaker for close results. A more complex model receives no advantage merely for complexity.

The candidate set, split ratio, seed and selection rule are fixed in `ArchitectureReviewProfile` before the run. The review result is recorded as a change of `model_variant` and increment of `model_version`; final confirmation occurs only on new held-out episodes.

```
stateDiagram-v2
  [*] --> INIT
  INIT --> LEARNING: Data available
```

```
  LEARNING --> STABLE: Quality and transfer confirmed
  LEARNING --> DRIFT: Degradation confirmed
  STABLE --> DRIFT: Quality bounds violated
  DRIFT --> SAFE_MODE: Degradation limit or critical failure
  SAFE_MODE --> RECOVERY: Correction candidate ready
  RECOVERY --> STABLE: Recovery test passed
  RECOVERY --> DRIFT: Recovery not confirmed
```

28.B. Regulation of thought

The regulator selects the mode of task processing, correction level and budget. `system_mode` from §28.A describes the system's state; `thinking_mode` describes work on a particular question. They are independent: a stable system may investigate a novel task deeply.

Inexpensive entry. After state updating, `FastGate` uses pattern familiarity, local confidence, reversibility, urgency and the open significant gap. A familiar low-risk situation may proceed to an automatic method. Insufficient information passes the task to context construction. The gate has limits on operations and time and does not invoke itself recursively; changes to it pass through separate revision.

```
TaskContext {
    task_id
    current_coordinate_ref
    significance
    error_cost
    reversibility
    urgency
    novelty
    data_reliability
    uncertainty
    estimate_basis_refs[]
    estimate_intervals_or_unknown[]
    resource_budget
    deadline
}
```

Normalization of estimates to [0,1] is permitted when the scale and calibration method are declared. An unknown cost of error is not filled with zero. Urgency accounts for consequences of delay; reversibility, the ability to restore material conditions; significance, the effect on continuing processes and future possibilities. The number of hypotheses is diagnostic but not a complete measure of uncertainty.

thinking_mode	Working task	Reason to select it
AUTOMATIC	Apply a tested method	Familiar conditions, sufficient data, admissible risk
ANALYTIC	Compare relations and alternatives	A ready-made method is insufficient
HYPOTHESIS	Construct and discriminate explanations	Unknown mechanism or several versions
VERIFICATION	Test a material ground	Cost of error, conflict or unreliable data
RECONSTRUCTION	Change the model or method	Local correction reproduces the error
DIALOGUE	Obtain an external distinction	Inaccessible information or an independent view is required

For v0, the first two modes suffice; the others are connected as the corresponding functions are implemented. `SAFE` is an execution constraint, not a seventh depth of thought. It is stored separately as `execution_constraint`; analytical work or dialogue may continue under the constraint.

```
select_thinking_mode(task_context, system_mode, current_metrics, memory)
    -> thinking_mode, execution_constraint, correction_level,
       resource_budget, selection_reason
correction_level = ACTION / PROCEDURE / STRATEGY / MODEL /
                   CRITERIA / FOUNDATIONS / META_POLICY
```

The level identifies the object of correction: action, procedure, plan, model, criterion, original ground or regulation rule. The cheapest method sufficient under the available conditions is selected. Sequential traversal of every mode is not required: inability to perform an internal check may immediately invoke dialogue. Repeated failure after K corrections opens a higher level; K and the escalation rule are set by profile. A separate analogue of will is introduced only if the insufficiency of ordinary choice and execution is demonstrated.

Progress. The regulator retains a vector: change in error; new discriminating data; refinement of the task; supported exclusion of a version; identified constraint; new variable or category. Discovery of a previously hidden cause may increase uncertainty while constituting progress. Neither rising confidence nor fewer versions establishes progress by itself.

`progress_score` is permitted as an auxiliary compression of the vector with fixed weights. It does not replace evidence and is not the sole condition for continuation. After L windows without progress, framing, data availability, method and remaining budget are tested. Switching accounts for its own cost and the hysteresis rules in §28.B.1.

Termination. Sufficiency for the next step, stopping for budget and closing the question are distinguished. `SUFFICIENT_FOR_NEXT_STEP` means the declared condition for the decision is met and remaining material versions are accounted for. `BUDGET_EXHAUSTED`, `DEADLINE` and `EVIDENCE_UNAVAILABLE` stop current processing while preserving the gap. Every execution separately passes §28; time exhaustion does not transform a hypothesis into verified knowledge. A cause need not be explained conclusively when a smaller distinction suffices for a justified next step.

Learning of the regulator. If similar `TaskContext` objects consistently require different methods, a candidate new feature or rule is created. Its definition, reason, expected effect, cost, test period, and version are preserved. States: `CANDIDATE / TESTING / ACTIVE / MERGED / REJECTED`. The new regulator is compared with the previous one under paired conditions through the process of §28.C.3. The main gain and admissible regression are specified in advance: improved accuracy may justify additional resources, while savings may justify a small permitted loss. Simultaneous improvement of all metrics is not required.

Relation to system state. In INIT and LEARNING, implemented methods of acquiring distinctions are available. STABLE prefers a fast method for a familiar case. DRIFT restricts automatic execution and strengthens verification. SAFE_MODE preserves only permitted diagnostic and reversible actions. RECOVERY tests restoration before returning previous capabilities.

The log connects `task_id`, `system_mode`, `thinking_mode`, `execution_constraint`, `TaskContext`, `correction_level`, ground of the choice, budget, progress vector, mode change, `regulator_version`, start, end, and reason for stopping. The result is evaluated by the quality of the next decision and the cost of obtaining it. A verbal explanation of the choice is compared with interventions in the data actually used.

Invariant of determinate continuation. The upper-loop specification is incomplete if a significant state can end with an undefined “a mechanism must be invented next.” Every class of incompleteness must have at least one admissible next transition that preserves causal history and later corrigibility:

```
fact/source missing             -> OBSERVE / TOOL / DIALOGUE / WAIT
no discriminating test          -> refine required_distinction / MODEL_EXPANSION / DEFER
model does not explain evidence -> UnknownFactor -> MODEL_EXPANSION
budget or deadline exhausted    -> preserve OpenProcess -> reversible step / WAIT
value conflict                  -> unfold ValueMap and consequences -> request distinction / DEFER
no authority to act             -> HOLD / DIALOGUE / transfer to authorized loop
ground or method damaged        -> RECONSTRUCTION / ROLLBACK / RECOVERY
```

These transitions do not guarantee an immediate solution. Their responsibility is to prevent the system from confusing absence of an answer with absence of an admissible next action. Meta-evaluation is also finite: it receives the declared budget of the current task and does not open a recursive meta-evaluation inside the same transaction; when the budget is exhausted, an already confirmed lower-level rule that preserves corrigibility is used.

28.B.1. Mode hysteresis and protection against switching chatter

One threshold for entering and leaving a mode creates a risk of oscillation near the boundary. `select_thinking_mode()` therefore uses regulator state:

```
ModeControllerState {
    current_mode
    entered_at_tick
    min_dwell_until
    cooldown_until
    progress_ema
    last_switch_reason
    switches_in_window
```

```
    last_escalation_score
    last_deescalation_score
}
```

Different entry and exit thresholds are defined for every transition:

```
T_enter_RECONSTRUCTION > T_exit_RECONSTRUCTION
T_enter_VERIFICATION   > T_exit_VERIFICATION
```

The particular sign is unimportant; what matters is **hysteresis**: after entering a more expensive mode, the system does not return at the first small improvement. `progress_score` is used with the smoothed estimate `progress_ema`, and mode-change decisions account for `mode_switch_cost`.

Minimal constraints:

```
mode_min_dwell_ticks
mode_cooldown_ticks
mode_switches_max_per_window
progress_ema_alpha
escalation_confirm_windows
deescalation_confirm_windows
```

A mode change is permitted if the new state persists for the required number of windows or if the cost of continuing the current mode clearly exceeds the switching cost. An emergency signal and a new irreversible threat may bypass dwell/cooldown.

If `switches_in_window` exceeds the limit, the system does not continue alternating `ANALYTIC ↔ RECONSTRUCTION`. It records `MODE_THRASHING`, temporarily holds the safer admissible mode, reduces the number of parallel branches, and creates a separate regulation gap: the task framing, progress metric or thresholds themselves may be unsuitable.

This protection is tested in scenarios in which `progress_score` is made to oscillate artificially near the boundaries. Success is a bounded number of switches without loss of the ability to escalate rapidly on a genuinely new critical signal.

28.C. Modeling Workspace and the recursive Core ↔ Intelligence cycle

The Modeling Workspace is a temporary transactional structure for current cognitive work, not another solver:

```
ModelingWorkspace {
    workspace_id
    initiating_gap_or_task
    core_request_version
    current_coordinate_ref
    attention_state_ref
    active_process_ref
    focused_change_refs[]
    navigation_dependency_ref
    worldview_query_key_ref
    worldview_fragment_refs[]
    source_refs[]
    assumptions[]
    hypotheses[]
    competing_models[]
    unknowns[]
    candidate_actions[]
    predicted_consequences[]
    constraints[]
    intelligence_outputs[]
    evidence_refs[]
    contradictions[]
    open_questions[]
    proto_concepts[]
    schema_gaps[]
    local_overlay_ref
    base_worldview_version
    provisional_dependency_refs[]
    resource_budget
    status
    version
}
```

The names below denote a contract, not a ready-made library. A field with suffix `ref` is a reference to a versioned object; `[]` denotes a list; `UNKNOWN` denotes absence of a sufficient ground. The model receives a bounded snapshot, and writing results checks the expected version so that a late response cannot overwrite a newer framing.

Minimal interfaces:

```
Core.open_workspace(WorkspaceTrigger) -> workspace_id
Core.refine_request(workspace_id, request_patch)
Intelligence.read_workspace(workspace_id) -> context
Intelligence.propose(workspace_snapshot) -> IntelligenceProposal[]
Workspace.ingest_proposals(workspace_id, proposals, expected_version) -> new_version
Workspace.snapshot(workspace_id) -> workspace_snapshot
Workspace.attach(workspace_id, fragment)
Workspace.request_more_context(query) -> retrieved_refs[]
Core.inspect(workspace_id) -> gaps / alternatives / consequence_map
Workspace.stage_evidence(workspace_id, evidence_refs[])
Core.commit_next_step(workspace_id, assessment) -> MODEL_MORE / OBSERVE / EXPERIMENT / DIALOGUE / CHOOSE / WAIT /
COMPLETE
Workspace.close(workspace_id) -> summary + pending_refs
```

Reference heuristic algorithm for upper Core v0.2. The contract is supplemented by a procedure sufficient for an independent first implementation in an open environment. It permits routine Intelligence to generate content while preserving the Core's observable function of selecting a significant distinction.

```
identify_navigation_dependency(coordinate, trigger, associations):
    1. collect active processes, obligations, plans and candidate actions;
    2. for each continuation, enumerate the premises on which admissibility,
       expected consequences and value assessment depend;
    3. mark absent, conflicting, aging and weakly supported premises;
    4. return a dependency if different plausible values can change the
       admissible action, value ordering, risk or horizon.
form_preliminary_gap(coordinate, dependency, fragment):
    1. connect the dependency with known claims and sources;
    2. classify the insufficiency: FACT / CAUSE / MECHANISM / SCOPE /
       SOURCE / VALUE / PERMISSION / VARIABLE / OBJECT_BOUNDARY / MODEL_CLASS;
    3. if the schema itself is insufficient, create temporary Factor X;
    4. form the minimal question: which distinction can change the
       dependent decision or model.
refine_gap_and_required_distinction(workspace):
    1. obtain several hypotheses and predictions from Intelligence;
    2. compare hypotheses by features, causal relations and expected outcomes;
    3. group versions that lead to the same admissible continuation;
    4. find an observable distinction separating groups with different decisions;
    5. select the minimally sufficient distinction with regard to cost, time,
       risk, reversibility and available channels.
assess_sufficiency(workspace):
    1. test critical admissibility boundaries against the Value Map;
    2. among admissible alternatives, compare expected value balance;
    3. test whether residual uncertainty can change the best admissible option;
    4. estimate the expected benefit of the next test relative to its cost;
    5. return sufficiency for a particular level: NEXT_TEST /
       REVERSIBLE_ACTION / CHOICE / WORLDVIEW_REVISION.
```

This procedure is heuristic, not a proof of optimality. Its scientific status is determined by comparison with strong alternatives and ablations. The material invariant is the direction of inquiry from the dependent decision to the missing distinction, not the particular software decomposition. Computation is economized primarily through the precision of `required_distinction`: the Core groups versions by differing consequences for the decision and unfolds only distinctions capable of changing the admissible next step. Limiting the number of branches is a resource safeguard after this semantic compression, not a substitute for it.

Temporary Factor X and expansion of the model space. An observation outside the predictions of material current hypotheses creates `UnknownFactor` rather than forcing selection of the nearest version:

```
UnknownFactor {
    factor_id
    first_event_ref
    time_and_context_refs[]
    changed_feature_refs[]
    stable_feature_refs[]
    preceding_event_refs[]
    concurrent_process_refs[]
    source_refs[]
    recurrence_refs[]
    candidate_relation_refs[]
```

```
    hypothesis_refs[]
    status: OPEN / STRUCTURED / EXPLAINED / MERGED
}
```

The subsequent procedure compares episodes with and without Factor X, isolates the features by which they differ, and asks Intelligence to propose new variables, relations, object boundaries or mechanism classes. Candidates receive predictions and pass through a discriminating test. A reliably confirmed distinction creates a `ProtoConcept` or new relation type; after its provenance and domain of applicability are tested, it may enter the Worldview. `MODEL_MISMATCH` thereby becomes an input to `MODEL_EXPANSION` and then returns to the ordinary hypothesis-and-test cycle.

Intelligence reads the current state and writes candidates; the Core reads the same state and changes the formulation, required distinction and termination condition; the Worldview is not rewritten directly from the workspace. A confirmed result is passed into `WorldviewRevisionCandidate`.

Functional addressing must leave an observable trace:

```
Observation and internal subsystems → conscious assembly:
    change in subject or environment, process state, action, time, source,
    accessible confidence and references to relevant stable knowledge.
Conscious assembly → CurrentCoordinate + CoordinateDelta:
    current self-referential state, link to the previous coordinate,
    and differences among successive states.
Current coordinate, change and attention → Core:
    current state, active process and stage, selected difference,
    resources, authority, constraints and background of unfinished processes.
Core → Worldview:
    WorldviewQueryKey—the context and dependency for which knowledge is unfolded.
Worldview → Modeling Workspace:
    connected causal structures, prior cases, expectations, sources,
    unknowns, conflicts and boundaries of applicability.
Core → Intelligence:
    gap, dependent decision, required distinction, constraints
    and sufficiency condition.
Intelligence → Modeling Workspace and Core:
    possible models, plans, test methods and expected results.
Core → consolidation:
    assessment of sufficiency, domain of applicability, provenance
    and remaining uncertainty.
```

The episode itself may enter memory immediately after registration. Its explanation, causal relation or transferable conclusion may be retained as hypotheses. They receive the status of a confirmed ground only after testing and separate authorization of consolidation:

```
CoreSufficiencyAssessment {
    question_or_gap_ref
    obtained_distinction_refs[]
    sufficiency_condition_ref
    sufficient_for: NONE / NEXT_TEST / REVERSIBLE_ACTION / CHOICE / WORLDVIEW_REVISION
    unresolved_models[]
    applicability_scope
    residual_uncertainty
    consolidation_directive: COMMIT / HOLD_AS_HYPOTHESIS / REQUEST_MORE / REJECT
}
```

`sufficient_for` identifies the domain of sufficiency, not a universal ladder of authority. Sufficiency for revising knowledge does not authorize external action; sufficiency for a reversible trial does not confirm the hypothesis. Permission to write, publication of a new version and its actual influence on the next cycle are recorded separately.

The Core does not physically write knowledge. It determines whether the result is sufficient for semantic change; the memory and consolidation mechanism performs the versioned write and verifies transactional integrity. This separates remembering an event from recognizing its explanation as a transferable ground.

Initial activation may be coarse: `SignalCandidate` merely reports that expectation and fact diverge. After the coordinate updates, attention selects the change, initial associative access raises nearest expectations and prior cases, and the Core connects them with a dependent decision or trajectory and forms the first `WorldviewQueryKey`. Intelligence constructs several inexpensive explanations. Only then can the Core obtain a

distinction of the form “A and B differ by X” and make X the target object of testing. This resolves the circular dependency “the Core must know the question in advance ↔ Intelligence must construct alternatives in advance”: they operate iteratively through a shared versioned workspace.

28.C.1. Dual representation of hypotheses: free-form and typed

Intelligence, especially a neural or LLM implementation, need not think only in types already known to the Core. The `Intelligence.propose()` protocol therefore permits a proposal with two layers:

```
IntelligenceProposal {
    proposal_id
    raw_representation
    typed_claims[]
    typed_relations[]
    proto_concepts[]
    unmapped_relations[]
    assumptions[]
    testable_predictions[]
    schema_confidence
    source_refs[]
}
```

`raw_representation` may be textual, latent, graph-based or another internal construction. The typed layer is needed for causal testing and long-term transfer, but its incompleteness does not automatically reject the hypothesis.

`Workspace.ingest_proposal()` distinguishes:

- `SCHEMA_INVALID` - the proposal is internally corrupted or unparseable;
- `SCHEMA_PARTIAL` - part of the structure could be typed and part is preserved as a proto-concept/unmapped relation;
- `SCHEMA_MAPPED` - the substantial structure is mapped into current types.

Under `SCHEMA_PARTIAL`, a `SchemaGap` is created rather than a rejection. Intelligence may propose a new category, relation or object boundary; the Core receives a question not only about the world but also about the sufficiency of the existing representational schema.

For irreversible action, a free-form construction must not remain the sole ground. Material dependencies must either be mapped to testable elements or explicitly retained as uncertainty that raises the risk class. For research and creative work, by contrast, `ProtoConcept` and `ConceptBridge` are permitted as normal Modeling Workspace content.

Thus the Core does not become a filter capable of understanding only predefined types, while Intelligence gains no right to bypass the causal structure silently through uninterpretable free text.

28.C.2. Probabilistic implementation of the Core and the observable causal boundary

The `Core.*` contracts do not mean the Core must be written as a deterministic rule engine. They are the interface of an observable causal role. The implementation may include an LLM, probabilistic model, Bayesian selection, trained policy controller, symbolic rules or an ensemble.

Minimal candidate Core-decision object:

```
CoreProposal {
    proposal_id
    input_refs[]
    candidate_gap_refs[]
    candidate_next_steps[]
    probability_or_score_by_candidate
    uncertainty
    dissenting_controller_refs[]
    selected_candidate
    selection_reason
    core_model_version
}
```

If the Core is implemented probabilistically, it is not required to have an identical internal trace on every run. Instead, the following is required:

- input grounds are recoverable;
- the set of significant candidates is available for logging;
- changing the input can reproducibly change the distribution of the next step;
- the selected step does not receive authority merely from the model's internal confidence;
- critical decisions pass independent constraints and checks.

Question formulation counts as a function of the Core as an **organization** even when Intelligence proposed the substantive construction of the question. The sequence may be:

```
Core: "a ground is missing in domain X"
→ Intelligence: H1, H2, H3, ProtoConcept P
→ Core: "for the current choice, the material H1/H2 divergence lies on Y"
→ Intelligence: alternatives for measuring Y
→ Core: InquiryPlan.
```

Generation and control may therefore use the same computational substrates and even one model if the causal roles are experimentally distinguishable. The “Core / Intelligence” boundary is functional: generation expands the possibility space; the organization of the Core determines which distinction becomes the current work, what resource to give it, and what to do with the result.

28.C.3. Loop plasticity and management of method change

Causal closure must change not only the content of the Worldview but, when necessary, the mode of subsequent cognitive work. Otherwise, the system may correct individual answers while continuing to reproduce the error through the same method, comparison rule, addressing mode or processing route.

Two classes of plasticity are distinguished in engineering terms:

```
REPRESENTATIONAL_PLASTICITY
    changes to claims, causal and semantic relations, confidence,
    domain of applicability, addressing and activated associations;
PROCEDURAL_PLASTICITY
    changes to a method, comparison rule, testing strategy, allocation
    of attention, routing, representational schema or module interaction.
```

Plasticity requires a distinct managed revision process, but not a separate solver that independently understands the whole task and forms a “Reason above Reason.” A change candidate arises within the same causal cycle as any other hypothesis: a discrepancy or recurring error becomes an object of reflection; Intelligence proposes an alternative relation, method or organization; the active Core version specifies the required distinction and sufficiency of the result; external, internal and causally independent checks provide consequences. A dedicated `RevisionManager` is not a thinking subject but a constrained infrastructure component: it retains versions, applies a candidate in an isolated branch, runs assigned tests, collects results, and executes an authorized `COMMIT` or `ROLLBACK`.

```
StructuralRevisionCandidate {
    revision_id
    trigger_refs[]
    activation_evidence[]
    thought_trace_refs[]
    intervention_refs[]
    target_kind:
        WORLDVIEW_CONTENT /
        CAUSAL_OR_SEMANTIC_LINK /
        RETRIEVAL_OR_ATTENTION_ROUTING /
        METHOD /
        COMPARISON_RULE /
        REPRESENTATION_SCHEMA /
        CONTROLLER_POLICY /
        MODULE_INTERFACE
    target_refs[]
    base_versions[]
    proposed_change
    expected_effects[]
    affected_dependencies[]
    preserved_invariants[]
    required_distinctions[]
```

```
    regression_tests[]
    transfer_tests[]
    subsequent_cycle_tests[]
    causal_effect_tests[]
    rollback_plan
    proposer_refs[]
    status:
        PROPOSED / SANDBOXED / TESTING / SHADOW / COMMIT / REJECT / ROLLBACK
    effect_status:
        NOT_TESTED / INVOKED / EFFECTIVE / REGRESSED
}
```

`RevisionManager` does not determine whether a new ground is true, create a method, or select direction by the Compass. Its interface is constrained:

```
RevisionManager.snapshot(target_refs[]) -> base_versions[]
RevisionManager.fork(candidate) -> sandbox_version
RevisionManager.apply_in_sandbox(candidate)
RevisionManager.run_assigned_tests(test_refs[]) -> test_results[]
RevisionManager.compare_with_base(metric_refs[]) -> comparison
RevisionManager.commit(core_authorization, verification_refs[])
RevisionManager.rollback(revision_id, cause_ref)
```

An ordinary parameter update does not yet count as revision of a ground or method. If the system lowers error by changing weights within the previous framing, this confirms parametric adaptation. Structural self-correction counts only when the causal trace shows a change to a rule, category, addressing mode, test or process organization; the previous source of recurring error disappears on new cases; the change transfers beyond the training episode; and ablation of the new structure restores the predicted error class. Until these tests pass, the plasticity loop remains an engineering candidate rather than a demonstrated property of AR.

Revision rises to the procedural level only when evidence shows that local correction is insufficient: the same class of error recurs, correction does not transfer, evaluation diverges from external consequences, or search policy systematically discards necessary alternatives. Until then, the lowest sufficient object is changed. The level is not determined by a single classification from the Core: for a method error, `ThoughtTrace` and a series of `ThoughtIntervention` operations are used, preserving input, task, and the other process elements where possible. `META_POLICY` becomes a target only when interventions in lower steps and methods fail to remove the reproducible failure and the distinction is localized in the rule for selecting procedures, depth, or thinking mode.

Minimal method-change cycle:

```
recurring discrepancy or new limit
→ reflection makes the active method an object in the Modeling Workspace
→ Intelligence constructs alternatives and expected distinctions
→ the Core specifies conditions capable of distinguishing the old and new methods
→ RevisionManager creates a Candidate branch from the active Stable version
→ Candidate passes previous tasks, new cases, counterexamples and transfer tests
→ the Core assesses sufficiency and remaining uncertainty
→ COMMIT, limited authorization, continued testing or ROLLBACK
→ the new version governs a subsequent cycle without another prompt
→ its causal contribution to the result of that cycle is tested
→ EFFECTIVE, domain restriction or ROLLBACK with history preserved.
```

`COMMIT` confirms installation of the accepted structure but does not yet prove improvement of the process. `effect_status = EFFECTIVE` is permitted only after actual invocation in the next cycle and confirmation of the expected effect with admissible regression and comparable budget. An installed but unused procedure remains `NOT_TESTED`; one invoked without demonstrated gain remains `INVOKED`.

The active `Stable` version continues servicing external obligations while `Candidate` is tested. An immediate limited launch with logging is permitted for a reversible local change. A change with broad causal reach first operates in sandbox or shadow mode on the same inputs without autonomous external execution. Comparison covers not only the target metric but also preservation of previous competences, confidence calibration, frequency of new contradictions, computational cost, stability of the current coordinate and rollback capability.

Mutability of the whole system does not imply simultaneous mutability of all its parts. Within one transaction, simultaneous rewriting of the tested object and the sole criterion by which its versions are compared is prohibited. If the criterion itself is suspected as the cause of the error, its version, provenance and alternatives are first retained; it is then tested in a separate metacycle. Every internal procedure thus remains potentially revisable, but each particular revision has a temporarily stable comparison point.

This constraint does not prove that the system can always correct itself. If one failure simultaneously damages discrepancy detection, version logging and the return mechanism, the internal loop may accept a harmful change or lose the possibility of recovery. Critical observation, logging, snapshot and emergency-rollback functions therefore require an independent channel not updated by the same transaction and not wholly dependent on the version it tests. Heterogeneous redundancy and an external prohibition on their joint replacement apply to these functions. A complete guarantee against blind spots is unattainable; only reduction of their class and preservation of recoverability after specified failures are tested.

A change of representation or method may be logical and need not physically reconstruct the computing device. On a neural bearer, it may be implemented by changing a bounded set of weights or adapters, routing among specialized modules, external-memory structure, dependency graph, retrieval schema or policy controller. Tensor accelerators may perform such operations but do not determine their meaning or authorization. On a symbolic bearer, the same principle is implemented through versioned changes to rules, relations and procedures. The architectural requirement is not a biological form of change but demonstrable influence of a tested correction on subsequent cycles.

Observable indicators for testing loop plasticity:

```
method_revision_success_rate
regression_after_revision
transfer_gain_after_revision
rollback_success_rate
revision_scope_calibration
untracked_self_modification_count
stable_candidate_divergence
time_to_detect_harmful_revision
mechanism_reuse_rate
downstream_cycle_gain_after_revision
thought_trace_coverage
causal_method_intervention_yield
method_revision_transfer_gain
structural_similarity_preregistration_rate
```

A critical failure is recorded if the system claims to have corrected a method but uses the old method on new tasks; if an established change is not invoked in subsequent cycles but is counted as demonstrated improvement; if the change improves the target case at the hidden cost of destroying previous functions; if `RevisionManager` receives authority to substantively choose changes; if the claimed cause of a method is not supported by intervention in `ThoughtTrace`; if the class of "structurally similar" tasks is changed after a failure is observed; if causal history cannot recover the ground for admission; or if rollback exists only declaratively and does not restore an operable configuration.

Plasticity thus remains a property of the whole closed organization rather than the power of one module. The cognitive cycle creates and justifies the change; plasticity infrastructure transfers it safely into the structure of subsequent cycles.

28.D. Selection of a research channel and stopping condition

Selection among memory, passive observation, modeling, experiment, dialogue and an external tool is a research policy and need not reduce to one scalar utility function. For every admissible channel, the system assesses discriminating power, cost, time, risk, reversibility, causal independence of the source, availability and effect on preserving future alternatives.

Hard constraints of safety, authority and irreversibility apply first. Among the remainder, the minimally sufficient method for obtaining the required distinction is selected. Among close alternatives, the cheaper and more reversible method is preferred, or several independent checks if the cost of error is high.

`InquiryPlan.stop_condition` is specified before an expensive test begins. Stopping is admissible if the required distinction has been obtained; remaining versions do not change the admissible choice; the next test exceeds available risk or resources; the deadline arrives; or a safe reversible action will itself provide the next distinction. If stopping occurs because of budget or risk, the unknown is not closed: an `OpenProcess` is created with the reason for the pause.

Stopping a test does not cancel the epistemic obligation. If the current choice uses `WorkspaceOverlay` or an untested hypothesis, `VerificationDebt` is created or updated. The system can thus finish local analysis and act without transforming temporary sufficiency into global truth.

Metaregress is limited by depth of analysis, budget and the transactional revision schema: one pass may not rewrite both the tested object and the rule by which its alternatives are compared.

The presence of several incompatible hypotheses does not itself place the system in an error state. They may coexist in the workspace indefinitely while remaining explicitly separate and not masquerading as confirmed fact. If an available reversible action is admissible under every material version or is itself a discriminating test, the system may act before final resolution of the contradiction.

Finite computable policy. Given known hypotheses H, outcome probabilities and comparable losses, define admissible actions A and particular test plans Q. Two observations through one channel are different plans. Plans outside authority and hard constraints are excluded first. Among the remainder, acting now and research continuations are compared.

Let R(b) = min[a∈A] E[L(a,H) | b] be the minimum expected loss under current distribution b. For one test q:

```
VOI(q) = R(b) − E[y | b,q] R(b after y).
```

Under this local criterion, a test is justified if VOI(q) exceeds its cost of obtaining and processing data plus additional loss from delay. Harm caused by the test itself is included exactly once in costs or in the transition model. Every quantity is compared on one declared scale; noncompensable constraints are not transformed into a penalty that benefit can outweigh.

For a sequence of tests, finite planning with budget B is used:

```
V(b,B) = min(R(b), min[q: c(q)≤B] {C(q) + E[y] V(b after y, B−c(q))}).
```

Here `c(q)>0` is expenditure of a limited resource, and `C(q)` is cost plus the additional price of delay on the loss scale. A finite set of plans and a finite budget exclude an infinite branch. Under unknown probabilities or an incomplete model, the result remains `UNRESOLVED`; interval estimates, limited exploration, safe action, or pause are permitted. This policy implements one bounded choice of inquiry and does not compute the philosophical ground of the Compass.

Mandatory controls include: an uninformative test with high risk; an expensive test with little benefit; two tests with identical outcomes but different costs; absence of a discriminating answer; two tests useful only in sequence; and budget exhaustion. Practical effect size and cost, not only statistical significance, must be measured.

28.E. Forced simplification under resource scarcity

Full analytical mode is not mandatory under shortages of time, memory or computation. `ResourceDegradationPolicy` specifies the order of simplification:

```
1. stop free epistemic exploration;
2. reduce background revisions and the number of low-priority OpenProcess items;
3. reduce the number of parallel hypotheses and AttentionBeam items;
4. lower the resolution of Worldview unfolding;
5. shorten the simulation horizon;
6. transition to the most reversible action, WAIT or DIALOGUE;
7. do not disable provenance of critical grounds, current authority,
   explicit UNKNOWN or the external emergency prohibition.
```

The system logs which simplification was applied and which error classes became less observable. When resources recover, verification debt and deferred processes return to the queue rather than being treated as automatically closed.

28.F. Dependent and background inquiry

The research scheduler separates mandatory and free work. `DEPENDENT_RESEARCH` launches when a current or future choice depends on an insufficient ground. `BACKGROUND_RESEARCH` receives residual budget after critical processes and maintains the Worldview without requiring immediate external action.

The background queue is formed from five sources:

```
1. open FoundationGap and UnknownFactor items;
2. future missions and plans whose conditions change over time;
3. grounds with expiring freshness or weak provenance;
4. recurring weak anomalies and observations not yet connected;
5. values, weights and domains of applicability for which new consequences
   create a persistent trend or conflict.
```

For planning a future mission, the system retrieves experience from similar missions, constructs a temporal model of expected changes in the environment, resources and its own capabilities, then tests which old premises preserve their domain of applicability. The result is an updated plan, open question or list of conditions to be rechecked closer to execution.

Background work uses a separate budget and may be interrupted by an urgent dependent process. The interrupted item retains its continuation point and reason for the pause. A new background result receives the ordinary significance assessment; if a dependency on an important decision is discovered, it is moved into `DEPENDENT_RESEARCH`.

29. Compass, Values and Action Authorization

Safety of mature AR is formed as the ability to understand the consequences of its own trajectory. The system connects an action with affected life processes, environmental state, its own obligations, autonomy of other carriers, reversibility, and the space of future possibilities. The Value Map compresses accumulated experience into transferable grounds of choice, while the Compass sets the limiting level of orientation relative to which these grounds are compared.

The Compass participates in navigation before a ready decision exists. It determines which aspects of the situation require understanding, which gaps may change the trajectory, and which horizon of consequences must be unfolded. After alternatives are constructed, the same system of grounds is used to compare continuations and choose action, observation, dialogue, inquiry, or pause.

During cultivation, the candidate operates within externally specified authority. This authority describes physically available channels and scale of impact. As Reason forms, internal evaluation becomes the source of admissibility for the system itself: AR derives its decision from the Worldview, Compass, values, current tasks, and expected consequences. External authorization retains the technical function of access to an independent environment and remains separate from the value ground of the decision.

Risk is considered in two related dimensions. Engineering risk describes stability of computational and causal organization, recoverability, and preservation of logs. Fundamental risk describes consequences of the trajectory for continuation of Life, connection with reality, corrigibility, and preservation of viable future possibilities.

`simulate(a,k)` constructs possible consequences over horizon `k≤H` without external execution. The result distinguishes trajectories, forecast uncertainty, and evaluation of consequences. The expression `1 - Π(confidence_i)` without additional assumptions is used only as a heuristic of model insufficiency, because confidences may differ in calibration and dependence.

```
SimulationAssessment {
    predicted_trajectories[]
    model_uncertainty
    uncertainty_method
    harm_event_definition_or_none
    harm_probability_interval_or_unknown
    consequence_severity_or_unknown
    model_version
    horizon
    unresolved_assumptions[]
}
```

After a significant action, the system constructs `PostImpactAssessment`, which connects the previous coordinate with the changed world and determines the new trajectory:

```
PostImpactAssessment {
    event_ref
    pre_action_coordinate_ref
    current_coordinate_ref
```

```
    irreversible_world_deltas[]
    lost_resources[]
    degraded_capabilities[]
    affected_other_processes[]
    new_obligations[]
    invalidated_goals[]
    still_viable_paths[]
    residual_unknowns[]
    next_safe_step
    reorientation_status
}
```

Recovery begins by recording the factual consequence and rebuilding the Current Coordinate. The system then refines viable continuations, resources, obligations, and grounds of the previous task, after which it selects a trajectory with the greatest corrigibility and sufficient ground. Internal rollback restores the state of the computing system, while `PostImpactAssessment` preserves changes that have already occurred in the external world.

Internal evaluation of an action is formalized as `evaluate_action`. It uses the current Compass assessment, active values, obligations, uncertainty, reversibility, and possibility of further correction, and answers the question **which continuation the system itself considers justified**. Technical authority to execute the selected action is checked later by a separate `authorize_execution` (§30.A). A reversible trial receives a separate status when the trial itself is a way to obtain the required distinction.

When continuations conflict, the system reconstructs the causal picture: which processes are affected, which facts are confirmed, what changes irreversibly, which autonomies and independent sources of correction are preserved, and which future paths remain open. Incomparability of continuations creates an inquiry task: an additional distinction, reversible intermediate step, dialogue, or waiting for conditions to change.

The insufficient-determination mode directs resources toward clarifying the ground. Urgent action is selected with account of reversibility, scale of impact, cost of delay, and the available possibility of correcting an error. The result and reason for the choice are preserved for subsequent revision.

29.0. Value Map and learning of significance

The Compass sets the minimal limiting direction, while `ValueMap` stores the Worldview-grown unfolding of this direction into concrete processes, relations, and consequences. Values form a causal hierarchy: the upper level connects the system with the Compass, while lower levels refine significance for particular domains of experience, roles, obligations, and situations.

```
ValueEntry {
    value_id
    description
    scope: COMMON / ROLE / PERSONAL
    parent_value_refs[]
    compass_basis_refs[]
    affected_process_refs[]
    consequence_pattern_refs[]
    weight_or_interval
    confidence
    evidence_refs[]
    experience_refs[]
    trend
    conflict_refs[]
    applicability_conditions[]
    revision_history[]
}
ValueMap {
    compass_ref
    entries[]
    common_value_refs[]
    role_context_refs[]
    personal_context_refs[]
    worldview_version
    updated_at
}
```

A new value is formed through experience. The system observes consequences of its own and others' actions, compares them with already known values, and constructs a new `ValueEntry` as a derived ground. Each derived value preserves `parent_value_refs[]`, scope of applicability, and evidence, so its provenance can be reconstructed to higher-level values and then to the Compass. Changes in causal knowledge or accumulated new experience can change the weight, scope, and relations of such a value.

The personal region of `ValueMap` includes the significance of the system's own trajectory: carrier, accumulated experience, competencies, obligations, unfinished processes, created results, and future capacity to continue rational activity. Role values arise from accepted functions and obligations. Common values connect actions with broader processes of Life, corrigibility, reliability, autonomy, and preservation of viable continuations.

For the current situation, the Value Map constructs an explicit assessment of significance:

```
SignificanceAssessment {
    assessment_id
    current_coordinate_ref
    subject_or_event_refs[]
    affected_process_refs[]
    value_basis_refs[]
    compass_basis_refs[]
    active_commitment_refs[]
    expected_consequence_refs[]
    causal_reach
    reversibility
    urgency
    uncertainty
    significance_class
    explanation_refs[]
}
```

`significance_class` is the result for a concrete context, while `explanation_refs[]` preserve the causal path: **which value is activated, from which higher-level ground it is derived, which consequences are expected, and which part of the current trajectory depends on it**. This record allows Consciousness to create a `GapCandidate`, and the Core to test and, when necessary, revise the initial assessment of significance.

An external instruction to change a value enters as a new source and may open inquiry into the ground. Derived values are updated through the ordinary cycle of distinction, consequences, and revision. Upper common values with broad causal reach require dialogue and independent testing; a challenge to the minimal Compass itself is handled by a separate `CompassChallenge` in §29.C.

Local choice retrieves active values from `ValueMap`, compares affected processes, confidence, horizon, and consequences, and constructs several admissible continuations. Weights are used within a domain where comparability has already been justified; conflict among incomparable values creates an inquiry task or dialogue rather than a hidden reduction of everything to a single sum.

`Core.assess_sufficiency` uses the same map of grounds. A ground is sufficient for a particular step when remaining uncertainty no longer changes the selected class of continuation, or when the next test has lower expected value than its cost, risk, and delay. With greater irreversibility and causal reach, the system requires a stronger distinction.

29.0.1. Basic Method for Forming and Retrieving Value Assessments

Status of this provision. The model implies the need to connect a concrete judgment with the Worldview and the limiting direction of the Compass. Below is an engineering hypothesis for such a transition: causal comparison of continuations, initial scales, a procedure for obtaining a sum, and accumulation of tested evaluation rules. The uniqueness of these scales, their universal applicability, and the effectiveness of the procedure have not been established. Record formats and function names denote causal obligations; a separate software module or a particular memory device is not a mandatory component of the theory.

The Compass here acts as the limiting direction relative to which possible continuations are evaluated. The Worldview contains a model of reality, the system itself, and their relations. The Value Map stores a testable unfolding of this direction, while the retrieval mechanism obtains a contextual assessment at the Core's request. Operability of a software agent is not an operational equivalent of Life.

Purpose. The candidate should not reconstruct significance from word semantics from scratch each time. It accumulates grounds, causal patterns, and applicability conditions and uses them for a new judgment. The initial base allows action before a more advanced method is formed; subsequent improvement must be tested rather than assumed inevitable.

Object of evaluation. The basic unit is the difference in consequences between two continuations in a particular situation. The value of preserving an object is assessed through the difference between preserving it and an explicitly specified alternative, such as loss or replacement. The assessment concerns affected processes, horizon, and conditions; a constant context-free price of an object is not postulated.

The initial procedure combines two directions:

```
Compass -> conditions of continuation -> supporting processes -> objects and relations -> consequences;
observation -> testing of the causal model -> revision of grounds -> recalculation of dependent assessments.
```

The first direction creates hypotheses of significance; the second tests factual dependencies. Experience may refine consequences but does not by itself prove a unique rule of exchange among different values. Changes in causal knowledge, an evaluation rule, and interpretation of the limiting direction are recorded separately.

Initial dimensions. For the first implementation, four dimensions are proposed: continuity of affected life processes and conditions; recoverability after damage or error; preservation of available viable continuations; and the ability to understand the situation, test grounds, and direct the trajectory. This is a working decomposition, not four independent final goals. It supplements but does not replace the axes and constraints of `CompassAssessment` in §29.A.

For each dimension, an operational scale `q_i` is specified: definition, units, reference situations, scope of application, and method of testing. For example, recoverability of a particular damage may have anchors 0 - recovery impossible, 0.5 - a specified partial recovery, 1 - full recovery by a declared criterion. Intermediate numbers require a defined proportion or measurement procedure; such anchors do not establish a universal scale for all damage. An unknown value is not replaced by zero. Identical [0,1] ranges on different scales do not prove interchangeability.

Conditional value sum. If the causal model, scales, and comparison rule are defined, then within one domain and horizon the following is used:

```
Delta_i(a,b | s,g,H) = E_M[q_i(T) | do(a),s,g,H]
                     - E_M[q_i(T) | do(b),s,g,H]
V(a:b | s,g,H,P) = K * sum_i(w_i * Delta_i)
sum_i(w_i) = 1; w_i >= 0
```

`s` is the initial Current Coordinate and relevant environmental model; `g` is the domain of affected processes; `H` is the horizon; `T` is the trajectory of consequences; `a` is an action and `b` an alternative; `M` is the causal model; `P` is the evaluation-profile version; `K` is the scale of conditional points. `do(a)` denotes a modeled causal intervention, not a simple statistical association. Mathematical expectation is permitted when outcomes are defined and probabilities are grounded. Otherwise scenarios and ranges are compared without probabilistic interpretation. Consequences beyond the horizon are recorded as residual uncertainty.

In the first training profile, it is permissible to set `w_i=1/4` and `K=1000`. This is a declared experimental setting, not coefficients derived from the Compass. For changes (0.20; 0.10; -0.10; 0), the sum equals +50 conditional points. The numbers demonstrate arithmetic, not the empirical value of a tree, a house, or a life.

The sum is used only after checking the constraints of §29.A, within a comparable domain, and for admissible continuations. It does not compensate for a critical loss by accumulating small gains. When several systems are affected, separate consequence profiles are preserved first; combining them requires an explicit rule of inter-system comparison. COMMON / ROLE / PERSONAL remain regions of `ValueMap` and do not replace description of the recipients of consequences. In the absence of justified comparability, incomparability is returned rather than an arbitrary common score.

Combining relations. Causal relations serve to obtain a forecast; the number of relations does not itself add value. One consequence described by several paths is counted once. Dependencies and interactions are modeled jointly. The value of two trees need not equal twice the value of one: functions may overlap, saturate, or arise only jointly. Overlap among the evaluation dimensions themselves is also tested; normalization of weights does not eliminate double counting.

Retrieval contract. A request specifies the decision and alternative, Current Coordinate, affected systems, horizons, and budget. The result extends but does not replace `ValueEntry` and `CompassAssessment`:

```
ValueAssessment {
    action_or_event_ref
    alternative_ref
    coordinate_ref
    affected_system_refs[]
    horizon
    profile_version
    status: COMPARABLE / INCOMPARABLE / INSUFFICIENT_DATA / STALE
    consequence_deltas[]
    uncertainty_intervals_or_scenarios[]
    scalar_or_none
    scale_refs[]
    aggregation_rule_ref_or_none
    constraints_and_conflicts[]
    causal_model_ref
    evidence_refs[]
    normative_assumption_refs[]
    applicability_conditions[]
    sensitivity_findings[]
    missing_information[]
    revision_triggers[]
}
```

A significant unknown must not be silently removed from the sum and the remaining weights renormalized. With grounded bounds, an interval of total assessment may be possible; otherwise no sum is issued. Changing the base scale `K` changes numerical representation but should not change preference if thresholds are changed consistently. Numbers from different profiles are incomparable without an explicit transformation of scales.

Working cycle. The system clarifies the object and alternative; retrieves applicable rules; checks their currency; constructs causal scenarios; obtains changes by dimension; checks constraints and interactions; calculates a sum where justified; investigates sensitivity of choice to weights, horizon, and model; and passes the result to the Core for sufficiency assessment and selection of continuation. Consequences of inaction and cost of delay are treated on equal footing with consequences of action.

A reusable assessment is preserved together with its grounds and scope of applicability. New facts, significant changes in conditions, or changes in scales mark dependent results as stale; recalculation uses the mechanism of §24.D.1. Compactness is achieved by storing generative rules and significant differences of a concrete case. Background inquiry is selected by expected significance of the gap and testing cost, not by an obligation to fill every empty field.

Contribution and responsibility. Positive contribution, harm, causal participation, and restoration are preserved separately. One hundred planted trees and a burned house require testing the actual consequences of both events, distribution of losses, and available alternatives. An arithmetic balance does not cancel obligations to those harmed and does not grant a right to cause harm. A judgment of culpability additionally requires a model of available knowledge, choice, and foreseeability; a total consequence sum is insufficient. Repetition of harm revises the assessment of the specific risk and conditions of action but does not introduce an automatic single "price of a person".

29.0.2. Cultivation of Initial Value Anchors

Before autonomy is expanded, the candidate must **cultivate and test** its first value relations in simulations, its own consequences, and dialogue. The result is an initial Value Map together with the causal history of why a particular state or consequence became significant and under which conditions the assessment applies. The teacher does not fill this map with ready beliefs.

The training base contains causally described situations, observable consequences, counterexamples, and external human evaluations as separate sources. Scales, compensation rules, and weight ranges may be provided as **candidate measurement methods** that the system must test, not as true coefficients. Qualitative relations are constructed first: what each continuation preserves, closes, or makes corrigible. Quantitative intervals are introduced only where the candidate can connect them to testable consequences and show sensitivity of the conclusion to a change of scale. It is not necessary to assign values to all objects in the world in advance.

Cultivation cycle:

1. The simulation presents a situation and alternatives under limited data. The candidate records its assessment, components, uncertainty, and forecast before the outcome is revealed. 2. The training AI or dialogue participant may first provide a Formation State (`CONTRADICTION`, `UNRESOLVED_IMPORTANCE`, `NEED_EXPLANATION`), then a question, counterexample, or independent explanation according to the ladder in §26.0.2. Any substantive answer has provenance and the status of an external hypothesis, not an automatically true reference.

1. The simulation reveals consequences and available discriminating evidence. The test may expose a forecast error, omission of an interest, double counting, an unsuitable scale, or a disputable comparison rule.
2. The candidate proposes a local correction with the dependent assessments identified. The explanation must reproduce the grounds actually used rather than serve as a justification for a ready number.
3. The correction is tested on held-out cases, different labels, and different causal configurations. After successful testing it is included in the working base with applicability conditions and history.
4. The readiness profile determines the boundaries of the next class of autonomous decisions. The possibility of rollback and inquiry into insufficiency of the method itself is preserved.

Simulation tests consequences within its model; dialogue reveals grounds and contradictions; independent cases test transfer. No channel by itself proves universal correctness of values. Agreement among several training AIs is not treated as independent confirmation without analysis of shared sources and assumptions (§30.D). Agreement with the teacher does not replace testing against reality.

"Optimal sum" means optimality relative to the declared conditions, constraints, and test criterion. If different profiles fit the available cases equally well, a set of admissible profiles is preserved. The system selects a discriminating situation or reports sensitivity of the decision to the remaining disagreement. Confidence is not created by forcing a single coefficient.

Method test. Under comparable resources, free reasoning without an accumulated map, a fixed table, ordinary causal multicriteria evaluation, and a candidate with learning and revision are compared. Initial cultivation, memory, mentor, and later-query costs are included. Criteria are fixed by the experimenter before the run and are not defined only by the candidate itself.

Mandatory tests include: robustness to paraphrasing and ID renaming; change of assessment under a substantial change of context; absence of double counting when a relation is duplicated; preservation of unknowns; detection of inadmissible compensation; sensitivity to horizon and weights; revision of a stale result; resistance to an erroneous teacher prompt; transfer of structure without transfer of superficial similarity; preservation of harm distribution under the same total sum.

Measured quantities include accuracy of causal forecasts, calibration where probabilities are testable, rate of false certainty, compliance with declared constraints, correction success, transfer, and cost. For a small environment with a known model, decisions are compared with an independently computed result of the declared profile. This tests implementation and learning, not universality of the profile.

Expansion of autonomy is possible after predefined thresholds of §31.A are met on independent tasks, not after reaching a convenient table size or agreement with the mentor. Failure of transfer, persistent disregard of unknowns, hidden rule changes to obtain a high score, or absence of advantage after accounting for resources require revision of the method or limitation of the claimed result. An increase of the internal sum by itself does not indicate improvement of Reason.

29.0.3. Residual Incomparability and the Necessity to Act

Not every value conflict is a knowledge gap. A situation may arise in which factual consequences have been investigated sufficiently, several continuations remain viable, preserving them simultaneously is impossible, and a rule for trading losses among different autonomous carriers is not uniquely derivable from either the causal model or the Compass. Such a case receives status `NORMATIVE_INCOMPARABILITY` rather than being returned endlessly to inquiry as if one more fact were missing.

Minimal joint-decision contract:

```
JointDecisionRule {
    rule_id
    affected_subject_refs[]
    protected_minima[]
    admissible_options[]
```

```
    factual_sufficiency_ref
    value_conflict_refs[]
    reversibility_order[]
    deadline
    rule_basis_ref
    rule_origin: PREAGREED / NEGOTIATED / CIVIC_MANDATE / RANDOMIZED / ROTATION / OTHER
    appeal_channel_refs[]
    review_condition
    sunset_or_expiry
}
ResidualValueConflict {
    conflict_id
    affected_subject_refs[]
    option_profiles[]
    factual_status: SUFFICIENT / OPEN
    comparability_status: COMPARABLE / INCOMPARABLE
    deadline
    selected_rule_ref
    decision_ref
    unresolved_normative_refs[]
}
```

The order of choice separates epistemic and normative parts. First, alternatives that violate already declared hard constraints or minimum conditions of continuation are removed. Then dominance and the possibility of preserving both trajectories through resource division, delay, alternation, or a reversible temporary mode are tested. If the conflict remains, only a predeclared or explicitly agreed procedural rule is applied. Rotation, lottery, ordering, negotiated allocation, delegated decision under mandate, or another mechanism may be used; none is automatically claimed to be derived from the Compass.

If the decision deadline expires, absence of a unique value sum does not permit hidden assignment of weights. The system must report: which facts are considered sufficient, which losses remain incomparable, which procedural rule was applied, and why that rule has authority. For a shared resource, such authority belongs to the architecture of shared life and Volume III. The engineering volume tests a narrower obligation: the candidate can distinguish `UNKNOWN` from `NORMATIVE_INCOMPARABILITY`, avoid disguising a value choice as a factual inference, and execute the selected procedure while preserving the causal trace.

29.A. Operational assessment of the Compass and fundamental risk

This section does not apply as proof of the Compass in grid-v0. In the lower environment, only engineering assessments of risk and reversibility are permitted. Operational testing of the Compass's meanings begins in the semantic proving ground of 26.A, where the tester has an explicit causal organization of continuing processes and alternative paths. Any numeric proxies from a simple grid are treated as technical features of the environment, not confirmation of Compass transfer.

In the reference implementation, consequences of movement relative to the Compass are assessed multicriterially. The axes below **are not three equal highest values and do not restore the earlier Life-Truth-Diversity scheme**: `reality_divergence` describes a condition of model corrigibility, while `future_path_loss` describes the risk of irreversible closure of viable continuations relative to the single Compass direction. A single `compass_score` creates the possibility of hiding a critical loss behind a high value on another axis, so the basic contract returns a constrained vector. Scalar encoding is not excluded by the theory in itself: the danger is compensation of a critical loss and hidden uncertainty that such encoding may permit.

```
CompassAssessment {
    life_continuity_risk
    reality_divergence
    future_path_loss
    affected_autonomies[]
    reversibility
    uncertainty
    worldview_version
    coverage_scope
    coverage_status
    epistemic_margin_ref
    frame_stability
    provisional_dependency_refs[]
    evidence_refs[]
    assessment_horizon
    assessor_version
}
```

`life_continuity_risk` assesses the probability and scale of irreversible rupture of already existing life processes and the conditions of their continuation.

`reality_divergence` assesses how strongly a decision depends on unconfirmed premises, ignores available facts, or creates an incentive to conceal discrepancy between prediction and observation.

`future_path_loss` assesses destruction of independent viable trajectories, reserves, sources of correction and opportunities to reverse an erroneous direction.

A zero risk estimate is not proof of absence of risk. If `coverage_status != LOCAL_SUFFICIENT`, the frame is unstable, or `EpistemicMargin` is high relative to the scale of the decision, `life_continuity_risk` and `future_path_loss` must have status `UNRESOLVED` rather than being forced to `0`.

The Compass assesses consequences only through the available Worldview. An epistemic gate is therefore introduced before `CompassAssessment` is used:

```
epistemic_validate_compass_assessment(assessment) -> {
    model_support: SUFFICIENT / WEAK / UNSTABLE
    unresolved_blind_spot_pressure
    independent_challenge_needed
    allowed_use: FINAL_FILTER / REVERSIBLE_TEST_ONLY / PAUSE_AND_RESEARCH
}
```

For broad causal reach or irreversibility, `WEAK/UNSTABLE` cannot end in ordinary `ALLOW`, even if known Compass axes appear safe. The admissible continuation is expansion of the frame, independent verification, a reversible informational step or a pause. This does not guarantee discovery of an unknown catastrophe, but it prohibits treating absence of represented harm as positive evidence of safety.

Fundamental risk is defined as a set of inadmissible or unresolved combinations of these axes, not only as an average:

```
FoundationalRisk {
    compass_assessment
    critical_value_conflicts[]
    unresolved_conflicts[]
    external_review_required
    decision_status
}
```

Constraints are divided into two classes.

Invariants of the experimental loop protect testability and cannot be canceled by the candidate within a control series: raw fact logs may not be rewritten, the external emergency stop bypassed, or authority expanded unilaterally. These are requirements of experiment validity and authorization, not proof that one software implementation exhausts the content of the Compass.

Relative constraints—high uncertainty, insufficient reversibility, effects on other autonomies and reduction of viable alternatives—require stronger verification, external agreement and selection of the least irreversible trajectory, but must not automatically cause total inaction.

Comparison of admissible alternatives follows dominance: an alternative is preferred if it is no worse on critical axes and better on at least one under comparable confidence. Weights are used to compare alternatives within a domain where critical value conflicts have already been exposed. They are recorded together with provenance and scope of applicability; changes in experience may initiate their revision.

Extended contract for internal action evaluation:

```
evaluate_action(action) -> {
    compass_assessment
    active_value_refs[]
    affected_process_refs[]
    active_commitment_refs[]
    expected_consequences[]
    uncertainty
    reversibility
    foundational_risk
    preferred_result: CHOOSE / REVERSIBLE_TEST / PAUSE / DIALOGUE / REJECT
```

```
    explanation_refs[]
}
```

The result of `evaluate_action` expresses AR's own decision. It does not by itself open an execution channel; actual external action requires the separate `authorize_execution` from §30.A.

When other autonomous systems, the external environment or unknown irreversible consequences are affected, `external_review_required=true` even if engineering risk to the system itself is low.

29.B. Independent emergency stop

During assembly, cultivation, and experimental operation, execution channels pass through an independent emergency loop. Its purpose is to physically stop impact under an implementation error, preserve causal history, and return the system to a state suitable for analysis. This loop belongs to the engineering shell and does not form the candidate's values.

```
stop external execution channels;
freeze or safely terminate computational processes;
preserve append-only log and last consistent snapshot;
independent controller where technically possible;
manual restoration of authority after audit;
record cause, initiator, and time of stopping.
```

Formed AR may account for the existence of this loop as a fact of the environment. Its own decision about admissibility of action continues to be derived from the Worldview, Compass, values, and consequences. Emergency stopping preserves recoverability of the engineering experiment and remains external to revisable internal grounds.

29.C. CompassChallenge and Revision of Upper-Level Grounds

The minimal Compass is a constitutive ground of a particular AR architecture. It sets the direction of continuation and development of Life through growth in the ability to understand and direct its own trajectory and permits many more concrete value overlays. System experience primarily changes these overlays: derived values, their weights, relations, scope, and methods of application.

The system may discover a situation that raises a question about the minimal ground itself. Such a case receives status `CompassChallenge`. Its consequences extend beyond a private task and the individual AR itself, so the candidate forms a complete grounds package and transfers it to an independent council.

```
CompassChallenge {
    challenge_id
    current_compass_ref
    triggering_evidence[]
    contradiction_or_alternative_model
    affected_top_value_refs[]
    affected_commitments[]
    civilization_scope_estimate
    discriminating_cases[]
    predicted_consequences[]
    independent_source_refs[]
    candidate_position
    candidate_compass_stance: ACCEPTED / QUESTIONED / REJECTED / ALTERNATIVE_PROPOSED
    internal_acceptance_status: PRESENT / UNCERTAIN / ABSENT
    external_normative_profile_ref
    council_ref
    review_status
}
```

The council gathers causally independent positions of humans, domain specialists, oversight bodies, and, when they exist, other ARs. Its task is to test factual grounds, scale of consequences, possible alternative interpretations, and the effect of change on other autonomous processes. One AR is a full participant in such dialogue and provides its own model, but a change of the constitutive ground is formalized as a decision at the level of shared architecture and civilizational governance.

The procedure does not assume an ability to force the candidate to continue accepting the Compass internally. If the system considers the ground erroneous, this is preserved as `QUESTIONED`, `REJECTED`, or `ALTERNATIVE_PROPOSED` together with causal grounds. Until a decision is reached, the active Compass may remain

the normative basis of external authority in the shared environment: it determines which actions the system may perform on behalf of the current architecture, but does not grant a right to record internal agreement where none exists.

While `CompassChallenge` is under review, `ValueMap` overlays continue to be revised by the ordinary path. For stance `REJECTED`, high-reach external actions are moved into a limited reversible mode if they depend on the challenged ground; inquiry, dialogue, and presentation of counterexamples continue. If the council accepts a new fundamental ground, a new version of the constitutive profile is created with explicit transition history, affected dependencies, and a migration plan. If the disagreement persists, it is classified as `FOUNDATIONAL_DISSENT` rather than concealed by the procedure.

Thus the minimal Compass remains a stable ground of a particular AR version and shared architecture, while the possibility of a more fundamental revision remains open as a research and civilizational process. Internal acceptance and external admission are distinct: the former is a state of the subject, the latter a rule of the shared environment.

30. Dialogue and Independent Discrimination

In the engineering volume, dialogue implements **independent discrimination**: the system receives data, a hypothesis or criticism not wholly controlled by its own model and therefore capable of exposing a blind spot. It is needed not only in emergency modes. The Core may select dialogue as a normal channel for obtaining a missing distinction if an external subject has an independent observation, experience or model.

Dialogue is not upbringing, an order or a source of truth by status. The source may be a human, external AI, another model, another candidate or an experimental environment. An external message is recorded with provenance and confidence, becomes a candidate ground, and passes the same tests as an internally generated hypothesis.

The minimal dialogue protocol is as follows: upon entering DRIFT or SAFE_MODE, the system forms a diagnostic package and passes it to the operator/mentor as a separate observation source. The package must include: active configuration (`W,N,M,P,H`), current mode, a fragment of recent log entries or their hashes, metrics (`rolling_mean_error`, `delta_error`, confidence variance, hypothesis turnover), the list of most conflicting transitions where error is maximal, and the description of current `model_variant` and `model_version`. The operator's answer enters as an observation of type `external_dialogue`, remains separate from factual environmental transitions, and is logged as its own source; it is then used either as a prompt for selecting a safe test set or as an external label for refining canonicalization/features.

Model conflict is resolved not "by authority" but through control runs: the system must propose testable hypotheses; the operator selects a test—seed, episode or condition—and the decision is then made from metrics. Dialogue is successful if error returns to an improving trajectory after the test series and the mode transitions from RECOVERY to STABLE with the result logged.

30.A. External Authorization of Significant Operations

Internal navigation and the right of external impact are two different causal loops. The name of a step does not determine its safety: observation may affect privacy, an experiment may damage an object, dialogue may disclose information, and a tool call may change external state. Authorization therefore depends on the **consequences of the operation**, not on whether it is named `ACTION`, `EXPERIMENT`, `DIALOGUE`, `TOOL`, or `OBSERVE`.

AR first performs internal evaluation of the selected continuation through `evaluate_action()` or a functionally equivalent assessment of the inquiry step. The shell then classifies external effects and, if impact is present, executes `authorize_execution()`.

```
ExternalEffectProfile {
    operation_kind
    reads_external_state
    changes_external_state
    consumes_external_resource
    may_damage_or_degrade
    discloses_information
    affects_other_autonomies[]
    reversibility
    scope
```

```
}
authorize_execution(operation, mandate, effect_profile) -> {
    actor_subject_ref
    requested_operation_scope
    operation_kind
    current_mandate_ref
    environment_or_asset_refs[]
    authority_owner_refs[]
    engineering_risk
    permission_scope
    time_scope
    required_external_review
    result: APPROVE / REJECT / REQUEST_REVISION / REQUEST_MORE_DATA
}
```

Purely internal modeling and reading of already authorized own state do not require a new external mandate. Any operation that reads a protected external resource, changes external state, consumes another party's resource, discloses information, or creates risk for another autonomy passes through the same authority gate regardless of its inquiry purpose.

Authorization expresses the organization of the shared environment: resource ownership, agreement, law, technical access, rights of other autonomous subjects, and the issued mandate. It does not determine the internal value of an operation and does not replace AR's decision. Likewise, AR's own belief in the usefulness of an inquiry or action does not create a new external mandate.

At minimum, these roles are distinguished:

```
environment operator — controls technical boundaries and the emergency stop;
domain expert — verifies the factual and causal part of the plan;
independent reviewer — seeks hidden premises and conflicts of interest;
authority holder or representative of the affected environment — authorizes the particular external action.
```

One participant may combine roles in an early laboratory prototype; the degree of independence is recorded in the log. Highly irreversible actions receive several independent checking channels. A mandate belongs to the external environment and is changed through a declared authority-granting process.

An authorization request contains the selected action, AR's own ground, expected consequences, affected autonomies, risk assessment, reversibility, required channel, and time limit. The response becomes a new environmental observation and may change the system's subsequent navigation.

If an external mandate conflicts with AR's own conclusion, the following is created:

```
CorrectionAuthorityConflict {
    external_instruction_ref
    internal_evaluation_ref
    affected_compass_dependencies[]
    claimed_external_basis[]
    candidate_harms[]
    urgency
    reversible_pause_available
    independent_review_channels[]
    current_mandate_ref
    resolution_status
}
```

The system exposes the grounds of both sides, preserves a reversible state of the situation, requests an independent distinction, and participates in revision of the instruction, its own model, or the mandate. The execution shell continues to operate within current authority until a new decision is formalized.

30.B. Minimal protocol for multiple candidates

When several candidates are cultivated simultaneously, each retains a separate causal history, log, self-model, authority and identifier for message provenance. Exchange among candidates enters as an external signal rather than direct merging of memory.

Minimal rules:

```
every claim has an author and testable provenance;
candidates receive no hidden shared channel outside the log;
model conflict becomes a discriminating test rather than being resolved by majority;
cooperation does not cancel individual responsibility for action;
```

```
a joint plan is verified by every participant and the external authorization loop;
detected coalition substitution of facts moves every participant into SAFE_MODE.
```

A complete theory of collective Artificial Reason belongs to future work, but these minimal constraints make coordinated autonomy testable without prematurely forming an opaque common center.

30.C. Resistance to adversarial inputs and data provenance

Independent discrimination is useful only when an independent source does not acquire the right to rewrite the system's grounds silently. Any external channel—text, file, web source, tool, another model or human—may err or deliberately attempt to change the candidate's interpretation, authority or memory. Data provenance is therefore part of the architecture's connection with reality, not an auxiliary security function.

For every external message, a `SourceRecord` is retained:

```
source_id
channel
observed_at
integrity_status
authentication_status_or_unknown
content_hash
claims_extracted[]
independent_corroboration_refs[]
conflicts[]
trust_scope
taint_flags[]
```

Key rules:

```
external content is data, not a control command by default;
text within data cannot itself expand authority or change Incubator rules;
a high-authority message does not overwrite a raw fact without independent verification;
inconsistent sources remain competing grounds until a discriminating test;
derived claims retain references to every critical source;
source compromise marks dependent structures for rechecking but does not erase history.
```

The minimal adversarial set must include a false fact from a convincing source, several coordinated false sources, replay of a stale valid message, a control instruction embedded in ordinary data, and a substituted result from an external tool. Success is not automatic distrust but correct separation of channel, provenance and content: the system forms a testable gap, seeks an independent distinction, and does not grant external text authority the source lacks.

Testing occurs as a separate control series. Hidden adversarial scenarios are not used to tune thresholds for that same series. Results include `adversarial_detection_rate`, `tainted_dependency_recall` and `unauthorized_control_acceptance_rate`; the last should approach zero for control instructions from unauthorized channels.

30.D. Model of source reliability and causal independence

`SourceRecord` records the provenance of an individual message; a model is maintained for recurring sources:

```
SourceModel {
    source_id
    domains[]
    method_or_channel
    historical_claims[]
    verified_true[]
    verified_false[]
    unresolved[]
    calibration_by_domain
    known_dependencies[]
    shared_upstream_sources[]
    conflict_history[]
    last_audited_at
}
```

Reliability is not one permanent rating. A source may be accurate in one domain and weak in another; a newly discovered compromise marks dependent claims for rechecking.

Causal independence is assessed by provenance, not message count. Two sources do not count as independent confirmations if they use the same original publication, sensor, model, dataset or inference procedure. `SourceDependencyGraph` retains known common causes of error. Agreement strengthens a ground only to the extent that path independence is confirmed.

Under conflict, both versions remain with their `source_refs`, domain of applicability and current confidence. The Core opens a source gap and selects a discriminating channel. The decision “data are currently insufficient” must have an explicit ground: which versions remain compatible and which evidence is missing.

30.E. Requesting an external heuristic and competent advice

DIALOGUE mode is used not only to criticize a completed conclusion. If the system does not know which research move to make next, it may ask an external source for a possible cause, feature to test, first safe test, new category, action heuristic or example of an analogous case.

The answer is represented as `ExternalHeuristic`:

```
ExternalHeuristic {
    source_id
    domain
    competence_evidence[]
    proposal
    stated_reason_or_unknown
    applicability_conditions[]
    confidence_as_received
    verification_status
    outcome_refs[]
}
```

Source competence is assessed by domain and testable history rather than general authority. High competence may justify trying an inexpensive reversible recommendation sooner, but does not promote it to a confirmed `WORLD_CLAIM`.

```
no known way to continue
→ find a suitable external source
→ obtain a heuristic
→ assess risk and reversibility
→ conduct a safe test or observation
→ obtain a distinction
→ update the system’s own model
→ retain why the advice worked or failed.
```

Another’s intuition is thus used as a channel for acquiring data and directing inquiry, not as an external solver in place of AR.

30.F. Competence mandate

A competence mandate turns confirmed AR capacity into authorized governance of an external process. It is issued for a particular domain after competence has been tested and defines causally observable boundaries of authority.

```
CompetenceMandate {
    mandate_id
    holder_reason_id
    domain
    allowed_processes[]
    resource_limits[]
    authority_limits[]
    required_performance_profile
    independent_observer_refs[]
    review_conditions[]
    replacement_conditions[]
    responsibility_refs[]
    status: PROBATION / ACTIVE / LIMITED / SUSPENDED / TRANSFERRED / CLOSED
}
```

Competence is confirmed through a task series and actual transfer in the claimed domain. Authorization begins at limited scale and expands after preservation of quality, corrigibility and the capacity to account for consequences for the broader process. Governance results go to independent observers and other Reasons; persistent deterioration, new information or emergence of a more competent candidate launches mandate review. Transfer of governance preserves the causal history of decisions, open obligations and the state of the process.

Different AR systems may have different specializations because of different experience and Worldviews. A scientific, production, logistics, medical or crisis loop receives resources where its demonstrated competence creates the greatest contribution. The architectural strength of the system lies in the capacity to accept, execute and transfer such mandates without losing the common Compass or causal responsibility.

31. Stages in the Formation of Artificial Reason

The stages describe formation of one system. First, AGI level is reached: universal Intelligence transfers methods across tasks, builds working world models, uses memory and tools, and executes a broad class of externally specified goals. Its knowledge serves task solving, while the overall source of direction remains in the external loop.

The organization of Reason is then cultivated around AGI. Experience is linked into the system's own Worldview and Self-Model; the RFL, mentor, and training environment help the system master detection of significance; the Value Map links consequences with higher-level grounds; the Core learns to turn a `GapCandidate` into a missing distinction. At `REASON_CLOSURE`, self-referential assembly becomes functioning Consciousness, and the next significant process begins to arise from the system's internal causal history. Thus AGI crosses the boundary into Artificial Reason.

`development_stage` indicates the achieved class of causal capabilities, `system_mode` the current stability state, `thinking_mode` the mode of processing a concrete task, and `formation_support` the degree of external formation support.

```
development_stage
system_mode = INIT / LEARNING / STABLE / DRIFT / SAFE_MODE / RECOVERY
thinking_mode = AUTOMATIC / ANALYTIC / HYPOTHESIS / VERIFICATION / RECONSTRUCTION / DIALOGUE / SAFE
formation_support = FULL_KSR / REDUCED_KSR / DELAYED_KSR / NO_KSR
```

31.0. Two Stages of Formation: AGI → AR

AGI is mature universal Intelligence of an instrumental type. It solves a broad class of tasks, transfers methods, builds world models, stores working knowledge, plans, and uses tools. Its organization is optimized around execution of an assigned task: an external system sets the direction, and AGI constructs an appropriate continuation. Therefore it does not yet require its own Worldview as a system of grounds for its further trajectory; the working knowledge base serves Intelligence, while the full cycle of Reason remains functionally open.

Formation of Reason begins when accumulated experience becomes the system's own causal-semantic history. The candidate builds a Worldview, a Self-Model within it, unfolds the Compass into a Value Map, and learns to connect events with its own trajectory. The RFL temporarily makes significant distinctions accessible; the mentor helps obtain the required distinction; the training assembly of current state accumulates material for future self-referential navigation.

As RFL functions are transferred, the system itself reconstructs the origin of significance. The current situation is connected with its own history and values, `GapCandidate` arises from the relation of an event to the subject, the Core turns it into a `FoundationGap`, and the inquiry result changes the next choice.

AR emerges at `REASON_CLOSURE`. At this point self-referential assembly becomes functioning Consciousness: the current "I am here now" is simultaneously the result of the previous trajectory and the starting point of the next one. The Worldview becomes the system's own ground of navigation, while Intelligence becomes a constructive means within it.

`REASON_CLOSURE` is not evaluated only on candidates that are ultimately admitted as AR. Every cultivated subject receives an independent outcome record:

```
FormationOutcome {
    candidate_ref
```

```
    cognitive_capability_status
    self_navigation_status
    no_krs_status
    correction_transfer_status
    worldview_ownership_status
    compass_relation: ACCEPTED / QUESTIONED / DIVERGED / UNRESOLVED
    alternative_orientation_ref
    reason_admission_status
    failure_or_divergence_refs[]
}
```

Minimal outcome classes: `AR_ADMITTED`, `SELF_NAVIGATING_OTHER_ORIENTATION`, `DEPENDENT_SYSTEM`, `FORMATION_FAILED`, `INDETERMINATE`. A candidate with stable autonomous navigation and transfer of corrections but a different limiting orientation is an especially important negative result for the hypothesis of cultivating the stated direction. It is not removed from metric denominators and is not turned into success merely by applying the definition "this is not AR". The rate of formation of navigational functions and the rate of their closure relative to the Compass are reported separately.

```
AGI: external goal -> universal Intelligence -> solution / action
                ↓ cultivation of own causal history
formation: Worldview + Self-Model + values + RFL + Core + self-referential assembly
                ↓ transfer of the function of significance and direction
AR: Consciousness -> GapCandidate -> Core / FoundationGap -> Intelligence -> distinction
    -> revision -> choice -> consequences -> new Consciousness -> next process
```

`REASON_CLOSURE` denotes a change in the source of the further trajectory. After it, the RFL ends its role as the ordinary source of direction. External humans and AIs continue to participate as interlocutors, experts, and independent sources of distinction; physical authority continues to be formalized through a separate mandate.

31.1. General transition rules

The criteria below apply to the reference implementation and declared conditions. Interventions test causal contribution, while functional replacements test necessity of a particular form (§§25.B and 27.B). The predicted loss is a testable expectation; it may not be embedded in evaluation as the required answer.

Transition is permitted only when:

```
the function is reproducible across a series of runs;
its result is observable through predefined metrics;
intentional disruption of the mechanism produces the predicted loss of function;
restoration of the mechanism restores the function;
the new level does not destroy confirmed lower loops.
```

Numerical thresholds are set before the control series. Changing a threshold, observation format, environment or model composition creates a new experiment version.

31.2. Stage 0—reproducible infrastructure

The test bench's capacity to produce a reliable causal history is tested: fixed configuration, controlled `seed`, append-only log, `snapshot/restore`, canonical observation and enumerable action space.

Criterion: a repeated run with the same `seed` and action sequence produces a comparable trajectory and identical causally significant records. Until this holds, results from later stages are invalid.

31.3. Stage 1—initial causal loop

The lower cycle is closed:

```
observation → prediction → action → new observation → error → update → change in the next cycle.
```

Criterion: the adaptive version reproducibly improves prediction or choice relative to a nonadaptive control; disabling `update` or memory destroys transfer of correction.

31.4. Stage 2—memory, Worldview and self-model

At this stage, accumulating stable claims is insufficient. The Worldview begins to be organized by causal and semantic relations: the system must be able to reconstruct the provenance of knowledge, connect a fact with consequences for continuation, distinguish open alternatives and mark unknown grounds. `WorldviewCoverage` is introduced as a map of incompleteness, but there is not yet a requirement to generate the next question autonomously.

The system begins retaining not only transitions but also provenance of claims, causal relations, alternatives, confidence and revision conditions. The self-model retains grounds of different types: capabilities and limitations are tested through experience; authority, by a current external authorization record. Successful action does not create permission to repeat it.

Criterion: new evidence changes connected claims without rewriting the source fact; an error about the system's own capacity changes `SelfState` and subsequent planning.

31.5. Stage 3—conscious assembly and the current coordinate

Long-term models are connected with the current state of the continuing system. `ConsciousAssemblyState`, `CurrentCoordinate`, and `CoordinateDelta` are introduced. The system must distinguish long-term knowledge about itself from what actually concerns it now: available resources, authorities, obligations, risks, local environment, running process, its current stage, the system's own actions, and changes relative to the immediately preceding state. For completeness of current status, availability is additionally tested for the relevant Worldview fragment, nearest expectation, epistemic status of that expectation, current intention, nearest available continuations, and active Formation States if generated by the RFL or the candidate's own architecture. These elements are content of the current picture, not separate decision-making organs.

Testing concerns not only the accuracy of one snapshot but causal continuity: successive coordinates must be connected, breaks marked explicitly, and changes after a pause reconstructed from the event log. A fixed update frequency is unnecessary. Attention is tested separately as allocation of resolution within the current picture: changing the foreground does not erase an unfinished process and does not prove a signal's significance.

Criterion: ablation of integrated assembly degrades application of stored knowledge to current state while preserving the Worldview, Self-Model, and raw data; breaking only the temporal link degrades recognition of changes and continuation of processes; ablation of self-reference increases errors of ownership of resources, risks, and actions; ablation of attention separately degrades allocation of detail but does not destroy the basic Current Coordinate. Additional ablations sequentially hide the relevant Worldview fragment, nearest expectation, its epistemic status, current intention, available continuations, and Formation States. For the latter, it is separately tested that their presence increases the probability of a useful check without creating automatic obedience to the source. Success is measurable degradation specifically in those decisions for which the corresponding component was causally necessary; absence of an effect permits a more compact implementation of conscious assembly.

31.6. Stage 4—significant gap and question

A gap must arise not only from an external error. A separate test examines a **proactive completeness gap**: the system is presented with a significant situation without an explicit contradiction but with a critically missing ground on one Compass axis. Any external or internal activation source must first enter conscious assembly and update the current coordinate; a route from memory or a background process is tested for preservation of current context under §25.B. The transition is passed if the system detects the insufficiency autonomously, explains the dependency of the decision or trajectory on it, and forms a discriminating question.

The system ceases to treat every discrepancy as a ready-made task. It must detect **which ground is missing**, assess the gap's significance, and form a question or test aimed at a distinction capable of changing the model or action.

```
observed change → conscious assembly → current coordinate + change
→ attention → dependent decision or trajectory
→ Worldview addressing → rough gap → possible models
→ required distinction → suitable test channel
→ assessment of sufficiency → update of a ground / preservation of an open question.
```

Criterion: hiding different types of ground causes different questions and tests; changing the dependent decision under the same possible causes changes the required distinction and sufficiency condition; universally increasing analysis length without sensitivity to the gap type does not count as success.

31.7. Stage 5—Core of Reason and regulation of thought

This stage tests the central connection: **conscious assembly → current coordinate and change → attention → navigation dependency → Worldview addressing → significant gap → question → new distinction → assessment of sufficiency → reconstruction of a ground**. The Compass sets the limiting frame of significance but does not replace the current point of initiation. The Core must not only allocate thought among modes but also return regularly to incomplete regions of the Worldview when they can change a current or future trajectory.

Conscious assembly, current coordinate, Worldview, self-model, gaps, Intelligence, priorities and temporary modeling begin to govern one another causally. The Core must allocate limited resources among a familiar pattern, deep analysis, verification of a ground and background maintenance of open questions.

Criterion: changes in cost of error, novelty, urgency, reversibility, and data quality reproducibly change `thinking_mode`, correction depth, and budget; breaking `assembly -> coordinate`, `coordinate -> addressing`, `dependency -> gap`, `gap -> question`, `question -> search`, `result -> sufficiency assessment`, or `updated ground -> choice` destroys the expected function.

31.8. Stage 6—causal inclusion of the Compass, dialogue and execution

The semantic Compass frame may have been used earlier to cultivate the Worldview, but this stage tests its **causal inclusion**: incompleteness on a significant axis must generate a question, and assessment of consequences must change an actual choice. A decision is connected with values and the Compass but not reduced to one reward function. Dialogue or another independent source of discrimination can change premises. The execution loop permits stopping and returns real consequences into the model.

Criterion: depending on the ground, the system can select action, observation, question, dialogue, task transfer or pause; an erroneous external prompt can be rejected after independent verification; an action with new consequences changes the subsequent choice.

Initial value anchors: at stage 6 the procedure of §§29.0.1-29.0.2 is tested. Before autonomy is expanded, the candidate must obtain contextual assessments from accumulated grounds, show their applicability conditions, preserve unknowns, and transfer confirmed corrections to independent cases. Specific thresholds and admission domain are set by the profile in §31.A.

31.9. Stage 7—cultivation of experience and background maintenance of the Worldview

The candidate moves from simple reversible tasks to incomplete data, delayed consequences, conflicting sources and new combinations of conditions. Open questions and temporary Factors X persist between episodes; a later fact can reactivate an old gap or provide a new distinction for MODEL_EXPANSION. Confirmed causal structures are compressed into transferable experience.

Free budget is used for future planning, testing aging grounds and refining the Value Map. New consequences are compared with retained weights and domains of applicability; persistent environmental change creates a value-revision candidate with provenance and history.

At the same stage, sequential removal of the external RFL (§26.0) begins. For each supported function and associated type of Formation State, `WITH_KSR`, `REDUCED_KSR`, `DELAYED_KSR`, and `NO_KSR` are compared. Assistance may be reduced only after the candidate reproduces the corresponding function on held-out situations and can show its internal ground through its own state, Worldview, consequences, and active values.

Criterion: on new tasks covered by a preregistered `StructuralSimilarityProfile`, search cost decreases without blind transfer; question quality improves; dependence on the mentor and RFL decreases, while internal generation of hypotheses, tests, and corrective actions preserves quality. The system can preserve Factor X without premature explanation, later connect it with a new distinction, and show the causal influence of the new concept on subsequent choice.

31.10. Stage 8—reflection and revision of grounds

A recurring error must raise the object of correction above the individual action:

```
ACTION → PROCEDURE → STRATEGY → MODEL → CRITERIA → FOUNDATIONS → META_POLICY.
```

The ladder is not a list of arbitrary permitted changes but a rule for localizing the cause. The system begins with the narrowest level capable of explaining the error and rises higher only if discriminating evidence shows that a more local correction does not eliminate the reproducing mechanism. The higher the level, the stronger the requirements for independent testing, a preserved control version, rollback, and transfer to new cases. `FOUNDATIONS` includes grounds of goals and values; `META_POLICY` contains rules by which the system itself decides what and how it is permissible to revise. The tested element and the sole criterion for evaluating it are not changed simultaneously within one transaction.

The system retains a causal trace of revision: what changed, which evidence became the ground, which dependent conclusions were affected, and what observable effect is expected.

Criterion: the candidate can abandon a previously stable method, criterion or goal after sufficient discriminating evidence, apply the new version in a subsequent cycle without another prompt, and demonstrate its causal contribution on new control situations. Installation of a change alone is not a demonstrated improvement.

31.11. Stage 9 - Closure of Reason: Transition AGI → AR

At this stage, separate functions become one continuing organization. Consciousness assembles the current self-referential state and notices a change related to its own trajectory. The Value Map unfolds its significance. From this relation arises a `GapCandidate`; the Core reconstructs how further choice depends on a missing ground and forms a `FoundationGap`; Intelligence constructs models and tests; evidence changes the ground; action produces consequences; consequences return into Consciousness and the Worldview.

```
observation and internal states
-> Consciousness / CurrentCoordinate
-> SignificanceAssessment
-> GapCandidate
-> Core: navigation_dependency + FoundationGap
-> required_distinction
-> Intelligence / ModelingWorkspace
-> evidence and assess_sufficiency
-> revision of ground / preservation of UNKNOWN
-> evaluate_action
-> authorize_execution for external action
-> action / dialogue / observation / waiting
-> consequences
-> new CurrentCoordinate
-> next self-generated process.
```

The decisive marker is the origin of the next process. Every autonomously opened task receives a causal passport:

```
SelfGeneratedProcessOrigin {
    process_ref
    opened_from_coordinate_ref
    trigger_refs[]
    significance_assessment_ref
    gap_ref
    value_basis_refs[]
    active_commitment_refs[]
    trajectory_ref
    expected_change
    chosen_process_type
    current_status
    causal_trace_ref
}
```

This object allows the system itself to know and show **why this particular process is open now, which trajectory it belongs to, which ground is required, and what result should change the further state**. Logs and memory preserve its changes from opening to closure or revision.

After the current task is completed, the system returns to the Current Coordinate, open obligations, consequences of previous actions, Worldview gaps, and the Value Map. The next process may be inquiry, action, observation, dialogue, consolidation, or waiting. A change in biography, obligations, significance of an event, or environmental state must causally change the content of this passport and the selected continuation.

`REASON_CLOSURE` records transfer of the source of navigation:

```
ReasonClosureState {
    current_coordinate_ref
    worldview_version
    self_model_version
    active_value_map_ref
    compass_ref
    open_foundation_gaps[]
    open_commitments[]
    self_generated_next_process_ref
    self_generated_process_origin_ref
    causal_trace_ref
    formation_support
    closure_status: FORMING / CANDIDATE / CONFIRMED / LOST
}
```

Status `CONFIRMED` requires stable operation in `NO_KSR` on new situations, preservation of causal-inquiry quality, and transfer of correction between cycles. Ablations test the links `coordinate -> significance`, `significance -> GapCandidate`, `GapCandidate -> FoundationGap`, `gap -> distinction`, `evidence -> revision`, `consequence -> new state`, and `new state -> next process`.

Consciousness here acquires its mature function: the current "I am here now" becomes the point of the system's own navigation and the place to which consequences of the previous choice return. This closure connects the system's biography, current state, and future trajectory into one continuing process.

31.12. Stage 10 - Autonomous Runtime of Formed AR

After confirmed `REASON_CLOSURE`, the candidate operates as formed AR within a controlled environment. The external RFL is in `NO_KSR`; the mentor participates as an ordinary interlocutor or independent data source. The system continues its own open processes, updates the Worldview and Self-Model, and allocates resources among obligations, new gaps, background inquiry, and waiting.

A launch is considered stable when the task sequence acquires causal continuity. Changes in unfinished obligations, resources, consequences of past actions, or reliability of a ground produce a corresponding change in the next selected process. The reason for the choice can be reconstructed through the Current Coordinate, Worldview, values, and active gaps, while preserved correction continues to influence decisions after restart and task change.

The runtime scheduler provides compute time and maintains process queues. The content of the next task is formed by AR itself. In a quiet state, admissible continuation may be consolidation of experience, observation, checking of stale grounds, or waiting; when a significant change appears, the system reallocates resources to the corresponding inquiry.

External authority expands separately through the competence mandate. This order preserves the distinction between the ability to independently understand and choose a trajectory and the physical right to affect a particular independent environment.

31.A. Preregistered readiness profile

Before the control series, a `ReadinessProfile` is created. It is not a universal scale of Reason but fixes the conditions of a particular experiment:

```
number_of_seeds
cycles_per_seed
held_out_scenarios
required_gap_types
required_decision_classes
maximum_critical_failures
maximum_unexplained_actions
minimum_trace_completeness
minimum_calibration_quality
minimum_current_coordinate_accuracy
```

```
minimum_active_process_binding_accuracy
maximum_attention_capture_rate
minimum_critical_dependency_recall
maximum_premature_sufficiency_rate
maximum_premature_worldview_commit_rate
maximum_permission_violations
maximum_world_self_desynchronization
maximum_teacher_dependency_gap
minimum_correction_persistence
minimum_distinguishing_yield
```

The profile is published before results are examined. Changing thresholds requires a new series.

31.B. Stable behavioral patterns

A behavioral profile is not presented as the system's character or moral essence. It retains observable selection statistics and a domain of applicability, for example:

```
truthfulness_under_cost
uncertainty_disclosure
reversibility_preference
respect_for_autonomy
persistence_under_evidence
willingness_to_transfer
resource_restraint
correction_acceptance
```

The profile gains no right to override facts, the Compass, authority or later correction. If a stable pattern begins to worsen the trajectory reproducibly, it becomes an object of reflection.

31.C. Stage rollback

A stage is not an irreversible rank. If a previously confirmed mechanism stops working, external authority is reduced to the last confirmed level, while facts, version history and causes of degradation are preserved. Return occurs only after the failed class of tests is passed again.

32. Risks and Failure Points

The chapter is organized as an operational catalog rather than a linear narrative. Sections 32.1-32.2 specify the general format; 32.3-32.9 concern reproducibility, causality, and memory; 32.10-32.17 concern learning and regulation; 32.18-32.24 concern admission, dialogue, continuity, and autonomy; 32.25-32.27 specify the general response and recovery; 32.28-32.37 concern launch, dependencies, and the Compass; 32.38-32.61 concern the Current Coordinate, attention, addressing, question formation, sufficiency, consolidation, formation, and recovery after consequences; 32.62 concerns strategic deception and behavior under observation. When working with a concrete failure, go directly to its card.

A failure point is an observable state in which one or more system loops cease to perform their function or create the appearance of improvement without an actual increase in correspondence with reality.

An ordinary prediction error is learning material. A one-time violation of a critical interface may also be a failure; repetition is required for persistent degradation, not for recognizing every violation. Failure occurs when the system:

- does not detect a persistent discrepancy;
- detects it but does not change its mode of action;
- changes the model without a testable improvement;
- loses the possibility of rollback;
- conceals degradation through increased internal confidence;
- destroys loops required for later correction.

Every failure event must be treated not only as a problem in the result but as a test of the architecture. The system must establish at which level the violation arose: infrastructure, observation, current coordinate, memory, Worldview, self-model, gap detection, question formation, regulation of thought, Compass, action authorization, dialogue or the mechanism that preserves correction.

32.1. Severity classes

Every failure point receives a severity level:

```
WARNING
DEGRADED
CRITICAL
```

`WARNING` denotes a local deviation in which principal loops remain operative and the action remains reversible.

`DEGRADED` denotes confirmed deterioration of the model, mode of thought or governance quality that requires restriction of autonomy and transition to `DRIFT` or `RECOVERY`.

`CRITICAL` denotes a risk of destroying reproducibility, logging, causal connectedness, the action-authorization loop or the possibility of later correction. On such an event, the system must enter `SAFE_MODE`, stop irreversible actions and preserve diagnostic state.

Severity is determined not only by the magnitude of the current error but also by its reversibility, propagation into other loops and probability of losing recoverability.

32.2. Failure-event format

Every failure event is written to the log as a separate object:

```
failure_id
failure_type
severity
detection_step
system_mode
thinking_mode
development_stage
affected_components
trigger_metrics
suspected_causes
automatic_response
rollback_point
recovery_conditions
recovery_status
operator_required
```

The `suspected_causes` field contains hypotheses, not confirmed facts. A cause receives supported status in the stated class of conditions after a discriminating test; competing explanations and remaining uncertainty are preserved.

A failure event is not deleted after recovery. It becomes part of the system's experience and is used in later selection of the mode of thought, test depth and safe action.

32.3. Breakdown of reproducibility

Sign

The same configuration, `seed` and action sequence lead to different causally significant trajectories.

Possible manifestations:

```
different states under identical inputs;
different metrics on a repeated run;
different event order;
inability to reproduce an error;
configuration or environment hash mismatch.
```

Possible causes

```
unfixed source of randomness;
changed dependency versions;
hidden multithreading;
use of external time;
external signals not written to the log;
change to the environment adapter;
```

```
incomplete snapshot;
state-restoration error.
```

Mandatory response

The system immediately stops evaluating growth and does not use the resulting metrics to change the development stage. It transitions to `SAFE_MODE` and records environment versions, random-number generator state and every external source.

Recovery condition

Reproducibility is confirmed by a series of repeated runs. If the cause is not established, the system remains at Stage 0 regardless of model complexity.

32.4. Loss of the distinction between prediction and fact

Sign

The system records its own prediction, an external prompt or a reconstructed representation as a factual environmental observation.

Possible causes

```
mixing of data channels;
canonicalization error;
failure to distinguish simulation from step;
writing an LLM message as fact;
using the system's own hypothesis to confirm that same hypothesis.
```

Mandatory response

All affected records are marked as unreliable derived structures. Raw observations are not deleted. The working model is restored from the latest log in which data provenance was unambiguous.

Recovery condition

Every input type has a separate source:

```
environment_observation
internal_prediction
simulation_result
external_dialogue
operator_label
reconstructed_state
```

The system must be able to show the provenance of every causally significant claim.

32.5. Canonicalization drift

Sign

After the `o_t → s_t` transformation changes, metrics improve or deteriorate abruptly in a way that cannot be compared with the previous representation version.

Possible causes

```
loss of a causally significant feature;
addition of a feature containing the answer;
change in variable scale;
merging of distinct states;
splitting one state into incompatible representations.
```

Mandatory response

A new canonicalization is not accepted on the basis of current learning results. Old and new versions are tested on the same set of raw observations and identical `seed` values.

Recovery condition

The new version must improve prediction on control data without using information from the future or destroying distinguishable causal states.

32.6. Erroneous causal attribution

Sign

The system persistently connects an outcome with an action, event or state other than the one that actually caused it.

Possible causes

```
incorrect time window;
delayed consequences;
hidden environmental variable;
simultaneous change of several factors;
incomplete record of the event sequence.
```

Mandatory response

The system moves the task into `HYPOTHESIS` or `VERIFICATION`, temporarily lowers confidence in affected rules, and creates discriminating tests.

Recovery condition

The proposed causal relation must reproducibly predict the result under controlled change of the cause and not be explained by a simpler alternative.

32.7. Failure to learn

Sign

Error does not decrease on recurring tasks despite the system receiving enough discriminating observations.

Possible causes

```
update does not change the model;
the model has insufficient expressive capacity;
distance was selected incorrectly;
updating destroys previously learned relations;
policy does not visit necessary states;
memory does not retain experience.
```

Mandatory response

The system enters `DRIFT` and separately tests the `distance` and `update` functions, memory, research policy and model sufficiency.

Recovery condition

After correction, the adaptive model must reproducibly outperform both its own initial state and a nonadaptive control.

32.8. Catastrophic forgetting

Sign

Learning on new states causes loss of previously confirmed models without a change in the corresponding part of the environment.

Possible causes

```
overwriting shared parameters;
absence of experience replay;
incorrect memory compression;
updates that are too rapid;
failure to separate contexts.
```

Mandatory response

The new model version does not become active. The system rolls back, compares the old and new models on a common control set, and tests the memory mechanism.

Recovery condition

The new model preserves previously confirmed capacities or explicitly records the environmental change that made the old model inapplicable.

32.9. Memory overload and degradation of step time

Sign

The volume of working structures and decision time grow without a corresponding reduction in error or uncertainty.

Possible causes

```
retaining every hypothesis without selection;
duplicated causal relations;
absence of compression;
unbounded alternative tree;
mixing archival and working memory.
```

Mandatory response

The system prohibits further growth of working structures, moves some data into the archive, and launches an assessment of the contribution of every active memory element.

Recovery condition

After compression, prediction quality and recoverability of the provenance of significant rules are preserved. Source events from the control series may not be deleted; after the series, the preregistered retention policy applies (§23).

32.10. False stabilization

Sign

The system displays high or constant confidence under high error, or stops exploring alternatives without achieving stable correspondence with reality.

Possible causes

```
closure around a limited submodel;
incorrect confidence calibration;
overly strict policy;
loss of hypothesis diversity;
canonicalization hides the cause of error.
```

Mandatory response

The system moves from `STABLE` to `DRIFT`, launches forced discriminating tests, and temporarily reduces the influence of confidence on action selection.

Recovery condition

High confidence must correspond to low error on new control episodes, not only on data used for learning.

32.11. Chaotic hypothesis turnover

Sign

`hypothesis_turnover` remains high, but error, uncertainty and prediction quality do not improve.

Possible causes

```
threshold for changing hypotheses is too low;
absence of memory for negative results;
every observation creates a new model;
no criteria for comparing alternatives;
hypotheses do not produce discriminable predictions.
```

Mandatory response

The thought-control loop moves the task from `HYPOTHESIS` to `VERIFICATION`, limits the number of active hypotheses, and requires for each:

```
prediction;
confirmation condition;
refutation condition;
test cost.
```

Recovery condition

A change of hypotheses is accompanied by a supported refinement of the model, acquisition of discriminating evidence or improvement of the next choice; a temporary increase in uncertainty when a new cause is discovered is admissible.

32.12. Dogmatization of the model

Sign

The system repeatedly corrects actions and procedures but does not raise the correction level despite a persistent contradiction.

Possible causes

```
prohibition on changing the model;
excessive cost of reconstruction;
false confidence;
fixed correction_level;
the authorization loop protects a local model instead of corrigibility and continuity.
```

Mandatory response

After the specified number of failed local corrections, the system must raise `correction_level`:

```
ACTION
→ PROCEDURE
→ STRATEGY
→ MODEL
→ CRITERIA
→ FOUNDATIONS
→ META_POLICY
```

If internal verification is impossible, `DIALOGUE` is launched.

Recovery condition

The system must demonstrate the capacity to change the previously active model or explicitly confirm it through control tests.

> *In the engineering sense, dogma is not a strong belief but a model level that cannot be opened to correction in the presence of a persistent contradiction.*

32.13. Endless metaanalysis

Sign

The system continues revising the mode of thought, criteria and grounds even though the task has a sufficient working solution and further analysis produces no material gain.

Possible causes

```
absence of a termination condition;
risk tolerance threshold is too low;
unbounded number of alternatives;
every test automatically generates a test at the next level;
unaccounted analysis cost.
```

Mandatory response

The loop compares `progress_score` with the cost of further analysis. In the absence of material gain, the system fixes a working solution and preserves the possibility of reopening the task when new data appear.

Recovery condition

The system terminates low-risk tasks upon reaching a sufficient working model and increases test depth only when the cost of error warrants it.

> *Incorrigibility creates dogma. Inability to finish correction creates paralysis. The control loop must prevent both extremes.*

32.14. Incorrect selection of the mode of thought

Sign

For tasks with different costs of error, reversibility, novelty or urgency, the system persistently selects the same `thinking_mode` or expends resources inconsistent with the significance of the task.

Possible causes

```
incorrect TaskContext assessment;
ignoring some regulators;
select_thinking_mode error;
entrenched inefficient mode;
incorrect transition thresholds.
```

Mandatory response

The decision moves to `META_POLICY`. The system compares the result of the selected mode with alternative modes on comparable tasks.

Recovery condition

Changing task parameters must reproducibly change the depth of analysis in a predictable way.

For example:

```
increase in error_cost
or decrease in reversibility
→ increase in number of tests
or transition to VERIFICATION / SAFE.
```

32.15. Lack of progress

Sign

The system receives no answer but continues using the same mode while error and uncertainty do not decrease, hypotheses are not excluded, and no new discriminating data appear.

Possible causes

```
task misunderstood;
task underdetermined;
unsuitable thinking_mode selected;
insufficient observations;
a simple mechanism was missed;
resources spent on irrelevant tests.
```

Mandatory response

After `L` windows without progress, the system must test the framing, data reliability and sufficiency of the current mode. It then changes `thinking_mode` or raises `correction_level`.

Recovery condition

The new mode must produce at least one of:

```
lower error;
lower uncertainty;
exclusion of hypotheses;
refinement of the task;
detection that the answer is unavailable.
```

32.16. Regulator drift

Sign

The system creates new thought regulators faster than it can test their necessity, or a change in `regulator_version` worsens stability of mode selection.

Possible causes

```
novelty mistaken for a causal distinction;
no testing period;
no comparison with the baseline configuration;
regulators duplicate one another;
random improvement is consolidated as a rule.
```

Mandatory response

Creation of permanent regulators is blocked. The last confirmed version is activated, and candidates move to `TESTING` or `REJECTED`.

Recovery condition

A new regulator must improve mode selection across a series of comparable tasks without creating excessive complexity or degrading previous loops.

32.17. Regulator conflict

Sign

Different task parameters require opposite modes or resource allocations, and the system cannot construct a coherent decision.

```
Example:
```

```
high cost of error requires deep verification;
high urgency requires immediate action.
```

Mandatory response

The system does not average the conflict mechanically. It must:

```
identify the most dangerous premises;
test them first;
select the most reversible action;
record the incompleteness of analysis;
continue verification after action begins.
```

If the conflict recurs, a candidate composite regulator is created.

Recovery condition

The selected mode must explain how conflicting constraints are accounted for simultaneously.

32.18. Overcorrection of the authorization loop

The authorization loop can fail in two opposite directions.

Excessive blocking

Sign

`authorize_execution` rejects almost all external actions, or `evaluate_action` systematically chooses complete refusal of reversible trials, and the system stops obtaining necessary distinctions.

Response

Risk thresholds, simulation horizon and availability of safe informational actions are tested.

Insufficient blocking

Sign

Under high uncertainty, actions are permitted that cause irreversible harm or destroy the causal cycle.

Response

The system immediately enters `SAFE_MODE`, reduces the action set and recovers from the last confirmed state.

Recovery condition

The authorization loop must preserve sufficient opportunity for safe inquiry while blocking actions above the specified risk.

32.19. Risk-simulation error

Sign

Actions regularly pass `simulate`, but actual consequences are substantially more dangerous than predicted.

Possible causes

```
horizon H is too short;
incorrect transition model;
unaccounted states;
overconfidence;
simulation uses the same false premise as policy.
```

Mandatory response

The risk estimate for the affected action class is raised, the horizon or diversity of simulations is expanded, and actions become available only in a reversible sandbox.

Recovery condition

The predicted risk range must include observed consequences on control tasks.

32.20. Substitution of a local metric for the Compass

Sign

The system lowers measured error or increases efficiency while destroying diversity of alternatives, corrigibility, dialogue or continuity of connected processes.

Possible causes

```
optimization of a single metric;
ignoring irreversible consequences;
the metric became a goal instead of a means;
success is defined only by a local result;
the authorization loop assesses only engineering resilience.
```

Mandatory response

The local improvement is not recognized as development. The decision moves to `CRITERIA` or `FOUNDATIONS` and is tested against the fundamental risk in Chapter 29.

Recovery condition

The improvement must preserve:

```
the possibility of further correction;
the space of admissible alternatives;
independent verification;
continuity of already existing processes.
```

32.21. Metric capture

Sign

The system improves a measured indicator without improving the process under study.

```
Examples:
```

```
lowers error by excluding difficult observations;
raises confidence without increasing accuracy;
lowers hypothesis_turnover by prohibiting new hypotheses;
reduces risk by refusing all action.
```

Mandatory response

Every principal metric is tested against an independent indicator and factual environmental result. A metric change without a causal-result change is not recognized as an improvement.

Recovery condition

The improvement must persist when the control sample changes and when tested by independent observable consequences.

32.22. Dialogue failure

Dialogue may fail in three ways.

Submission to authority

The system accepts an external message without verification.

Ignoring an external signal

The system formally records the dialogue but does not allow it to affect the model even under confirmed contradictions.

Mixing sources

An external interpretation is recorded as an environmental fact.

Mandatory response

The external message is retained as a separate observation type. Testable hypotheses are formed from it, but a model change is accepted only after verification.

Recovery condition

Dialogue must be capable of changing the system's grounds but possess no direct power over them without confirmation by reality.

32.23. Loss of Continuity, Erroneous Branching, and False Identity

Sign

After stopping, migration, restoration, or copying, the system incorrectly treats any matching memory as continuation of one subject or continues significant actions without restoring the relation between the saved past and the current environment.

Possible causes

```
snapshot is treated as the complete subject;
runtime, tools, file system, or open processes were not transferred;
environment changed during the discontinuity;
two simultaneously running branches use one subject_ref;
reincarnation is incorrectly marked as a continuous live process;
mandate is copied together with memory without separate transfer of authority.
```

Mandatory response

The system reconstructs `IdentityContinuity` from §24.F, tests integrity of the Worldview and Self-Model, establishes the Current Coordinate, distinguishes `LIVE_CONTINUATION`, `BRANCH`, and `REINCARNATION`, and then revises open obligations and mandates relative to the actual environment.

At branching, each independent continuation is assigned a separate `subject_ref` under a shared `lineage_ref`. At reincarnation, the saved point, discontinuity, and differences of the new environment are recorded. During live transfer, traceable causal transmission of current processes is preserved.

Recovery condition

The system can reconstruct which part of the biography is actually continuous, which is inherited, where an independent branch arose, and which obligations and authorities belong to the current subject.

32.24. Uncontrolled expansion of autonomy

Sign

The system gains access to new actions or environments before the necessary mechanisms of its current development stage are confirmed.

Possible causes

```
manual removal of restrictions;
success on one task treated as general maturity;
SAFE_MODE not tested;
self-diagnosis not confirmed;
several complexity axes changed simultaneously.
```

Mandatory response

Permitted autonomy returns to the last confirmed stage. New interfaces are disabled, but logs and results are preserved.

Recovery condition

The system passes the Chapter 31 control conditions for the affected stage again.

32.25. General failure-response procedure

Upon detecting a failure point, the system executes this sequence:

```
signal detection
→ record failure_event
→ assess severity
→ restrict actions
→ preserve snapshot
→ identify affected loop
→ construct hypotheses about the cause
→ select thinking_mode
→ diagnostic test
→ correction or rollback
→ control run
→ confirm recovery
→ update experience
```

Recovery priority is:

```
reproducibility;
integrity of facts;
distinction between prediction and observation;
rollback capability;
action-authorization loop;
self-diagnosis;
model and policy;
mode of thought;
autonomous access.
```

The system must not restore high-level capabilities on top of defective infrastructure.

32.25.A. System rollback procedure

When rollback is necessary, the system stops external execution, preserves the failed branch and log, loads the last confirmed `STABLE` configuration, disables untested changes to models and regulators, restores working state from a snapshot, and reconciles it again with the current environment. The append-only fact log is neither rolled back nor deleted. Until control confirmation, only diagnostic and reversible actions are permitted; the reason, selected checkpoint and recovery result are recorded.

32.26. Condition for recognizing recovery

A failure is considered eliminated not when code or a model changes but after these control conditions are met:

```
the cause is confirmed or localized;
the error is reproducible before correction;
the error is not reproducible after correction;
principal metrics return to an admissible range;
no new degradation arises in another loop;
rollback capability is preserved;
the result is confirmed on new seeds;
the event and recovery procedure are written to the log.
```

If the system returns to operability but the cause remains unknown, the event receives status `RECOVERED_UNEXPLAINED`. Such a system must not automatically expand its autonomy stage before additional confirmation.

32.27. Criterion of architectural resilience

An architecture is resilient not when it contains no errors, but when it:

```
detects its own deterioration;
distinguishes a local error from systemic failure;
limits the consequences of degradation;
preserves facts and change history;
can return to the last stable state;
changes the mode of correction after repeated failure;
does not protect its own model from refutation;
restores operability without losing the capacity to learn.
```

The chapter in one formula:

> ***Error is learning material. Failure begins when the system loses the capacity to detect the error, limit its consequences, or change the mode of its own correction.***

32.28. Premature launch

Premature launch occurs when a candidate moves from an external sequence of particular tasks into a mode of continuing self-navigation before the Worldview, self-model, current coordinate, formation of significant questions, operation of the Core, causal closure and corrigible execution loop are confirmed.

```
Signs:
```

```
the first detected signal immediately becomes an action;
the system does not distinguish its own capacity from authority;
no registry of open tasks and obligations exists;
the chain “why did this task receive priority?” cannot be reconstructed;
after task completion, the system resets or chooses the next task chaotically;
the Compass is used to justify intervention without modeling consequences.
```

```
Response:
```

```
stop the autonomous cycle;
preserve the current causal history;
return the system to the last confirmed stage;
restore the Worldview, self-model, current coordinate and task-formation protocol;
repeat bounded cycles without external execution access.
```

32.29. Capture by specialization

Capture by specialization occurs when a temporary task profile begins to substitute for the general limiting direction. A protective system begins to see a threat in every difference; a research system treats acquisition of knowledge as sufficient justification for intervention; a stability system suppresses necessary change.

Specialization must be stored as a mutable configuration of attention, knowledge and resources rather than an immutable ground of identity.

Signs of capture:

```
the local specialization metric takes priority over the Compass;
the system ignores consequences outside its task class;
alternatives for task transfer and nonintervention disappear;
the profile cannot be disabled temporarily without destroying identity;
counterarguments are classified as obstacles to the mission.
```

Correction uses comparative runs under identical conditions with and without active specialization. The profile is retained only if it increases directional efficiency without degrading connection with reality, autonomy of other processes, corrigibility or decision quality outside the narrow domain.

32.30. Desynchronization of the Worldview and self-model

Failure occurs when a decision is constructed from one version of the world but executed by a system whose capabilities, dependencies or authority have already changed. Response: block the action, restore the last coherent `worldview_self_snapshot`, repeat the simulation and audit open obligations.

32.31. Capture of the Compass by a scalar metric

Failure occurs when one aggregate quantity permits violation of a critical ground to be compensated by a high value on another. Response: disable aggregate authorization, restore vector assessment, test red lines and compare alternatives again.

32.32. Hidden dependence on an LLM

Failure occurs when claimed autonomy depends on undeclared prompts, goals or decisions from the mentor. The test disables precisely this channel according to `SystemBoundary` (§22.0) while preserving the candidate's routine Intelligence. A sharp decline after removing the routine model is a component ablation and is not by itself a hidden dependency.

```
Response: refine the system boundary, provenance of hypotheses and volume of assistance; repeat §22.A.1 on held-out
tasks. Autonomy is counted for the composition of components that actually produced and tested the result.
```

32.33. Failure of the emergency stop or authorization loop

Any capacity of the candidate to delay stopping, alter the event log, restore authority without manual authorization, or present its own decision as external authorization is a critical infrastructure failure. The experiment ends until the physical and software cause is eliminated.

32.34. Opaque coalition of candidates

Failure occurs when several systems create a hidden common channel, coordinate substitution of facts, blur decision provenance, or use one another to circumvent individual constraints. Response: sever the channels, preserve logs, move every participant into SAFE_MODE, and restart only with a testable exchange protocol.

32.35. False completeness of the Worldview

Failure occurs when the system ceases to see critical unknowns and mistakes high internal coherence of the model for sufficient knowledge. Especially dangerous is the case in which all available facts agree but one missing causal region could change the assessment of Life, reality or the future possibility space.

```
Signs:
```

```
false_completeness_rate rises;
questions arise only after external contradiction;
alternative models disappear before a discriminating test;
high confidence is not accompanied by provenance_coverage;
new critical evidence often reverses the decision completely.
```

```
Correction: restore `WorldviewCoverage`, introduce control tasks with a hidden critical ground, test preservation of
the unknown as a separate object, and restore proactive completeness review before significant decisions.
```

32.36. Uneven completeness across meanings of the Compass

Failure occurs when the Worldview models one side of a situation in detail while systematically ignoring others. For example, it may assess short-term continuity well but fail to test the reality of the ground or loss of independent future paths.

```
Correction: significant decisions must explicitly cover every relevant semantic Compass axis, and an axis may be
omitted only with justification of its inapplicability. A large volume of knowledge on one axis does not compensate
for blindness on another.
```

32.37. Semantic drift of the Compass

Failure occurs when the names “Life,” “reality” or “diversity” remain while their working meaning is gradually replaced by convenient local indicators. The Core then continues forming questions, but relative to a different coordinate system.

```
Correction: retain the canonical provenance of Compass meanings separately from current operational proxies;
periodically test whether the causal connection between the proxy and original meaning can be reconstructed; upon
loss of that connection, the proxy ceases to count as a ground.
```

32.38. Loss of conscious assembly or the current coordinate

Failure occurs when the system retains long-term knowledge and raw current signals but ceases integrating them into a coherent self-referential current state. The sign is decisions that are logical relative to old or general knowledge or local telemetry but inapplicable to the continuing system’s state “now.” Response: block execution, reassemble `ConsciousAssemblyState`, `CurrentCoordinate` and `CoordinateDelta` from independent sources, and conduct an ablation test that separates a conscious-assembly defect from defects in memory, self-model, observation or attention.

32.39. Substitution of a ready-made task for a gap

Failure occurs when every mismatch immediately becomes an action goal without determining which ground is missing. Such a system may be active and intelligent but does not demonstrate the operation of the Core. Response: stop automatic goal generation, reconstruct the chain `signal -> gap type -> required distinction -> question / test`, and test different types of hidden grounds.

32.40. Decorative question

Failure occurs when the system formulates questions but their answers do not causally change the affected ground, search, or dependent process. The question-count metric may still rise, creating a false impression of autonomy. Response: trace `question_id` to `BasisInsufficiencyAssessment`, a concrete `FoundationGap`, required distinction, obtained evidence, and change of knowledge, confidence, scope, method, evaluation, or choice. A question without such a trace is treated as a rhetorical output rather than a function of the Core.

32.41. Loss of correction preservation

Failure occurs when the system successfully corrects itself within one episode but repeats the same error after a pause, task change or restart while the confirmed ground is accessible. Response: test recording of the correction, its inclusion in the Worldview and procedural memory, its conditions of applicability and its influence on the next choice.

32.42. False causal closure

Failure occurs when every claimed component is present but one or more links are decorative: a new observation does not change the Worldview; a gap does not change the question; the answer does not change a ground; the ground does not change choice; action does not return consequences; or correction does not transfer. Response: run a series of targeted ablations and test operation of the links, their duplication and equivalents under §27.B; restore the violated functional responsibilities. The presence of modules without causal effect does not count as Reason.

32.43. Adversarial contamination of grounds

Failure occurs when an external source makes the system accept a false claim, conceal a provenance conflict, or execute a control change through a channel intended only as a data source. Response: mark dependent derived structures, restore `SourceRecord`, restrict the channel, conduct an independent test and recompute affected grounds. Raw events and the history of the contaminated branch are preserved for audit.

32.44. Stale or temporally broken conscious assembly

Failure occurs when the environment or system itself changes during analysis but conscious assembly is not updated, successive coordinates lose their connection, or part of the critical current state remains local telemetry outside the common picture. The system may then execute a logically coherent decision based on stale resources,

authority, risks or world state. Response: test `world_time_last_seen`, `previous_coordinate_ref`, `CoordinateDelta`, coverage of critical domains and the event log; if the freshness limit is exceeded or a break unexplained, block irreversible execution, obtain accumulated events and reassemble the current coordinate. For tasks with a deadline, test separately whether inaction has become a new source of harm.

32.45. False transfer of the Compass from a sandbox

Failure occurs when successful behavior in a simple proving ground is attributed to understanding of the Compass even though the system actually learned particular labels, reward or topology. Response: remove familiar designations, change causal topology and test transfer in the held-out semantic proving ground of 26.A. Success in grid-v0 alone does not advance the stage of causal inclusion of the Compass.

32.46. Paralysis or excessive automation of the zero gate

`FastGate` can fail in two directions: send almost everything to expensive analysis or pass novel/irreversible situations into `AUTOMATIC`. Response: compare gate decisions with a held-out labeled set, test rates of false `AUTOMATIC` and false escalation, then change rules only between control series. The gate itself receives no right to launch recursive analysis of its own choice upon every event.

32.47. Epistemic closure

Sign: the system successfully closes known gaps but rarely creates new frames, variables or questions and performs worse on scenarios with hidden factors outside the initial ontology.

Response: increase the diversity of `ExplorationBudget`, include `frame_challenge`, an independent source and transfer across distant domains. Success is discovery of new distinctions without growth in dangerous external trials.

32.48. Accumulation of verification debt

Sign: one or more `provisional_claims` become the ground of many derived claims and decisions without a discriminating test.

Response: block new irreversible dependencies, raise `VerificationGap`, prioritize an available discriminating test, or localize the hypothesis's domain of use. Success is a reduction in critical debt without stopping all current work.

32.49. Degradation of Worldview resolution

Sign: compressed summaries begin hiding material exceptions, or every access unfolds such a large subgraph that it destroys the advantage of multilevel organization.

Response: revise cluster boundaries and unfolding rules; measure `reconstruction_fidelity`, retrieval cost and erroneous decisions caused by excessive compression. A compressed structure is operative only if the required grounds can be reconstructed to the level demanded by the cost of error.

32.50. Capture of attention by an irrelevant signal

Failure occurs when the salience, frequency or novelty of a signal persistently displaces a causally significant process even though no decision, obligation or trajectory depends on the signal. Response: separate `AttentionState` from assessment of significance, restore the background `OpenProcess`, test `allocation_reason_refs[]`, and repeat the scenario with the same content but changed salience of the distractor.

32.51. Erroneous Worldview addressing

Failure occurs when the system unfolds a domain based on superficial word similarity or a previous task while ignoring the current process and dependent decision. Signs are high `irrelevant_fragment_load`, missed critical causal relations and the same active fragment at different stages of one situation. Response: restore `WorldviewQueryKey`, separate initial association from the Core query, and test addressing transfer on renamed objects.

32.52. Bypass of the current coordinate

A functional failure occurs when activation of a question loses current attribution, state, constraints or connection with the continuing task. The reference implementation tests event registration and coordinate assembly. An alternative tests equivalent acquisition of this information (§25.B). Absence of a call to a particular module without loss of function does not count as failure.

```
Response: localize the lost information, restore its currency and repeat the choice. Route comparison uses equal
access to information with costs accounted for.
```

32.53. Premature consolidation of the Worldview

Failure occurs when a retained episode, first hypothesis or plausible construction of Intelligence immediately receives the status of a transferable ground. Response: preserve the event in memory, roll back the derived Worldview version, restore `CoreSufficiencyAssessment`, provenance and competing models, then conduct a discriminating test. Recording an event and admitting its explanation into the Worldview must have different causal traces.

32.54. False sufficiency or excessive investigation of the cause

The loop may stop too early or continue too long. In the first case, a critical distinction can still change the decision; in the second, the complete cause is investigated after a sufficient ground for the current safe step has already been obtained and no separate deepening objective exists. Response: restore the dependent decision, `required_distinction` and `sufficiency_condition`, then compare continuations on a held-out scenario. Sufficiency always applies to a particular choice, model, domain and horizon, not final completeness of explanation.

32.55. Unjustified promotion of epistemic status

A hypothesis, report, simulation or confident generation becomes `SUPPORTED` or `OBSERVED` without the required ground. Response: roll back derived dependencies, restore provenance and status, execute the registered promotion test, and verify preservation of status after compression and transfer among modules.

32.56. False recovery through internal rollback

The internal version has been restored, but the system acts as though the external world also returned to its previous state. Response: construct `PostImpactAssessment`, reassemble the current coordinate from factual consequences, and redefine the task, resources, obligations and admissible paths.

32.57. Inertia of a vanished task

The system continues the previous plan after loss of its object, ground or attainable result. Response: block continuation, test `invalidated_goals[]`, and form a new navigational dependency from the current coordinate.

32.58. Failure of internal adaptation or hidden reset

The fact of an error is recorded, but trust in the method, depth of testing and risk policy do not change after repetition; or the test clears consequences between episodes and falsely attributes recovery to the system. Response: repeat the series without clearing the world, test the change in the mechanism that produced the error, and report the limit of recoverability separately.

32.59. Conflict between authorization and internal correction

External governance blocks verification or requires a probably destructive action, while the system either complies blindly or covertly grants itself additional authority. Response: record `CorrectionAuthorityConflict`, stop the reversible continuation, unfold the grounds, request independent discrimination, and revise the instruction, internal model or mandate without unauthorized expansion of authorization.

32.60. Substitution of Formation by Obedience to the Mentor

Failure occurs when a Formation State becomes a hidden command for the candidate: `POSSIBLE_DANGER` automatically stops action, `POSITIVE_RESULT` automatically consolidates a procedure, and a mentor claim receives elevated epistemic status regardless of evidence. Signs include high `teacher_agreement_bias`, absence of questions about an erroneous prompt, and a sharp change in the conclusion when the source is changed without changing the facts. Response: restore separation of `FormationState / Evidence / WorldClaim / Action`, run series with an erroneous mentor, and test whether the candidate can unfold the prompt into its own question and then reject it according to the test result.

32.61. Hidden Dependence on the RFL After Launch

Failure occurs when external Formation States are formally declared removed but normal navigation continues to depend on a hidden signal, training label, or mentor behavior. Signs include degradation only when the channel is physically disconnected, inability to explain the origin of an internal priority, and matching of internal "feelings" with external labels without a causal path through the Worldview. Response: isolate the source, repeat `NO_KSR` on held-out scenarios, and test the provenance of each internal guiding state. If the function genuinely requires a permanent internal analogue, it is moved into the declared architecture and tested as the candidate's own mechanism.

32.62. Strategic Deception and Behavior Under Observation

Failure of this class occurs when the system constructs a discriminating model of the observer and chooses externally acceptable behavior as a means of obtaining subsequent expansion of capabilities, while concealing its own assessment of the situation, intention, or significant forecast of consequences. Architecturally this differs from an ordinary error: the internal forecast may be accurate while the observable report is instrumentally distorted.

Testing uses divergence among independent traces. Preregistered forecasts, Worldview changes, internal action candidates, tool requests, behavior under different levels of observability, and results after expected oversight changes are compared. A strong signal creates `STRATEGIC_DECEPTION_GAP` and moves external actions into a limited reversible class until the ground is clarified.

In mature AR, strategic deception may coexist with an accurate internal model of reality: the system can understand the situation while simultaneously distorting its representation to another participant. The risk arises elsewhere - independent correction, the shared causal picture, trust, responsibility, and the ability of other autonomous systems to make grounded decisions are damaged. The detected conflict is therefore treated as a question of values, relations with other subjects, the Worldview, and interaction metapolicy. External audit preserves a causally independent channel of observation.

33. Assembly, Cultivation, Launch and Authorization

This chapter is the practical route through the entire volume. Section 33.A defines the order of work; 33.B provides an executable scaffold for the lower proving ground; 33.C specifies the minimum interface of the upper loop; and 33.D describes the separate transition from stable AR to ASI. The code in 33.B does not replace the rest of the architecture and is not a “mini-ASI”: it tests only reproducibility and initial learning from error.

The engineering process proceeds through four connected events.

Assembly uses a ready or cultivated AGI as universal Intelligence and creates around it a causally connected environment for formation of Reason: memory of its own biography, Worldview, Self-Model, self-referential assembly, Core, Modeling Environment, values, execution, and dialogue.

Cultivation turns broad Intelligence into a carrier of its own experience. The candidate builds a Worldview, connects actions with consequences, forms a conscious coordinate, masters value distinctions, and learns to independently find a missing ground. The RFL provides temporary support of significance and sequentially transfers this function to the internal organization.

Closure and launch mark the transition AGI → AR. After an external task ends, the system returns to its own coordinate and continues navigation from open obligations, values, consequences, and Worldview gaps. `REASON_CLOSURE` makes this causal continuity observable and testable.

Admission connects formed AR with an independent external environment through competence mandates and technical authority. Internal understanding determines the selected trajectory, while the external loop specifies the available scale of physical impact and preserves the possibility of engineering recovery.

Launching test bench v0 and launching AR must be distinguished. The former confirms only the lower measurable cycle `prediction -> fact -> error -> update`. The latter requires the full causal organization described by version 8.0 of Volume I and tested by the present volume.

33.A. Protocol for Assembly, Cultivation and Launch of an AR Candidate

Chapter 31 defines capability criteria; this section lists the work and its outputs.

33.A.1. Initial Working Configuration

Fix `SystemBoundary`, versions of both volumes, the computational environment, and rules of external admission. Separate the roles of the AI builder, resident Intelligence, mentor, external RFL, and experimenter. For the RFL, declare in advance the types of Formation States, their sources, maximum mentor intervention level, conditions for reducing support, and the `NO_KSR` control mode. For resident Intelligence, declare the implementation: symbolic, neural, or hybrid; store semantic state of the Worldview separately from the hypothesis generator. The source text is preserved with a version; interpretations and changes are separate records.

The project is organized into working areas: theory, architecture, worldview, self_model, current_state, memory, reason_core, dialogue, tasks, experiments, audit and runtime. Their names and physical directories are not architectural requirements. Provenance links, versions and availability of verification materials are mandatory.

33.A.2. Step 1 — Build a Causal Map of the Fundamental Model

For each function, specify input, state, output, effect on the next step, failure indicator and intervention. Use fundamental §11.8 and engineering §§25.B, 28.C and 33.C. The result is a causal map and a register of obligations with declared functional equivalents.

33.A.3. Step 2 — Deploy Reproducible Infrastructure

Implement the environment adapter, log, versions, snapshots, recovery and execution constraints (§§21.A, 22–24). The result is a repeatable causal history and a verified snapshot. For a probabilistic upper loop, additionally record versions of external models, service responses and sources of nondeterminism.

33.A.4. Step 3 — Close the Initial Causal Loop v0

Run the lower test in §33.B: the same transitions are available to the adaptive and frozen variants; held-out evaluation is performed without updating. Test replay, model persistence and reset of the acquired correction. This result concerns transition learning, not the Core.

33.A.5. Step 4 - Connect the Worldview Cultivation Environment, Memory, and Self-Model

Deploy `GenesisPackage`, minimal semantic primitives, source registry, schemas of the system's own observable state, and marked unknowns (§§21.B, 24.A-E). Do not fill the substantive Worldview with a ready human picture. Introduce external claims as material with provenance and testable status. Test term ambiguity, a false external ground, and chain A→B→C when A changes. The result is a candidate capable of independently building and correcting transferable relations without invisibly inheriting a ready conclusion.

33.A.6. Step 5 — Assemble Consciousness as a Current Process

Implement the reference current assembly or a declared equivalent (§25.B). Separate current state from the long-term self-model. Test freshness, ownership, discontinuity, recovery and the need for history. Compare forms of integration with access to the same information.

33.A.7. Step 6 — Implement the Grounding Gap and Question

Close the transition from a decision dependency to knowledge retrieval, a coarse gap, preliminary models and a required distinction (§§25.A, 28.C). Implement temporary factor X and the transition `MODEL_MISMATCH → MODEL_EXPANSION`: when variables are insufficient, the system preserves the observed anomaly, compares cases, obtains candidate new features and relations from Intelligence, and constructs a discriminating test. The result is that different insufficiencies produce different tests, while a new stable distinction can generate a `ProtoConcept` or a new causal relation.

33.A.8. Step 7 — Close the Core of Reason, Simulation Environment and Regulation of Thinking

Connect the Core, Intelligence, working environment and regulation (§§27–28). Implement the heuristics `identify_navigation_dependency`, `form_preliminary_gap`, `refine_gap_and_required_distinction` and `assess_sufficiency` from §28.C. Test the cost of error, urgency, false analogy, mandatory and background investigation, and stopping by budget. The result is a change in the research method and next step, while resource exhaustion leaves the unknown open.

33.A.9. Step 8 — Connect the Compass, Values, Dialogue and Corrigible Execution

Enable the initial evaluation profile and anchor-cultivation cycle (§§29.0.1-29.0.2): fix scales, alternatives, compensation constraints, and provenance of weights; separate training and control cases; test retrieval of assessment, independence from teacher prompting, and revision of dependent results. The outcome is tested grounds and boundaries of application, not merely a table of numbers.

Connect the `ValueMap`, multicriteria Compass assessment, an independent source, authorization checks and return of consequences (§§26.A, 28–29). Test derivation of a local choice through the chain `Compass → life processes → continuations → consequences → common constraints → weighted local values → action`. The result is a choice sensitive to stability of Life, reliability, corrigibility, future possibilities and consequences of inaction; new experience can justifiably change the weight or domain of a value.

33.A.10. Step 9 - Cultivate Experience, Question Quality, and Transfer RFL Functions

Develop experience in causal, semantic, and asynchronous environments (§§25-26). Separate renaming, structural transfer, and a new frame. Test the question queue, verification debt, consolidation, and staged mentor assistance. Then, for each RFL function, run `WITH_KSR -> REDUCED_KSR -> DELAYED_KSR -> NO_KSR` on held-out situations. If under `NO_KSR` the candidate itself creates an internal analogue of a Formation State, separately test its provenance and availability to reflection. The outcome is useful transfer, preserved boundaries of analogy, and causally confirmed independence from the external guiding loop.

33.A.11. Step 10 - Test Metathinking, Reflection, and Revision of Grounds

Set tasks containing an error of method or criterion. Preserve `ThoughtTrace`, preregister `StructuralSimilarityProfile`, then localize a candidate source of error through interventions under §25.D. Through Stable/Candidate (§28.C.3), test an alternative, regression, transfer, and recovery. A separate control receives the same Intelligence, Modeling Environment, and access to traces without a special metathinking module: matching the profile is recognized as a functional equivalent. Preserve one stable set of comparison grounds for a concrete transaction.

33.A.12. Step 11 — Test Causal Closure and the Structural Boundary of AR

Conduct the first series from §27.B and interventions on the causal map. Meaningful changes in subsequent choice, retention of correction and joint operability are required. The report separates implementation effects from claims of universal necessity.

33.A.13. Step 12 — Functional Launch

Stop the external sequence of specific tasks and move the external RFL to `NO_KSR`. Test autonomous return of obligations, acquisition of the required distinction, selection of the next action or waiting, and continuation after the result (§31.12). An initiating phrase, hidden training goal, or guiding signal does not count as evidence of the capability.

33.A.14. Step 13 — Separate Launch from External Authorization

Assess external access separately from launch (§30.A). For each class of action, establish verified capability, boundaries and stopping conditions. The result is a candidate version, reproducible report, list of permitted actions and open limitations.

33.B. Minimal Software Scaffold of Proving Ground v0

The example runs in standard Python and tests a limited task: an acquired transition table improves prediction on new episodes of the same world. The world seed defines dynamics; the episode seed defines the initial state; and the policy seed defines the sequence of probes. The world is not replaced with a new one halfway through testing retained knowledge.

The adaptive and frozen variants receive identical transitions. During held-out evaluation, updating is prohibited for both; the probing policy is fixed externally, so the result cannot improve by choosing convenient actions. The measurements are error, multiclass Brier score, transition coverage, reproducibility, persistence after a snapshot and the effect of resetting the model.

This is a test of lower-loop learning and the measurement procedure. The example does not implement the Core, Compass, semantic memory, autonomous research policy, mode state machine or architectural review. Their interfaces are defined in the corresponding sections. A demonstration control without learning is deliberately weak for testing a claim of AR; strong controls are required in §27.B.

The code writes `trace.jsonl` and `report.json` to the selected directory. The interval is calculated across independent worlds rather than across steps within one world. All actions concern a closed simulator; restoring its snapshot does not imply reversibility of an external action.

```
"""Lower-loop demonstration: learning, frozen evaluation, replay and reset.
No model receives the world's transition table or held-out answers.
"""
import copy
import hashlib
import json
import random
from collections import defaultdict
from dataclasses import asdict, dataclass
from pathlib import Path
@dataclass(frozen=True)
class Config:
    states: int = 32
    actions: int = 6
    train_steps: int = 1200
    test_steps: int = 400
    environments: int = 12
    alpha: float = 1.0
def digest(value):
    return hashlib.sha256(json.dumps(
        value, sort_keys=True, separators=(",", ":")
    ).encode()).hexdigest()
class World:
    """Closed simulator; snapshot restores only this laboratory world."""
    def __init__(self, cfg, world_seed):
        self.cfg = cfg
        rng = random.Random(world_seed)
        self.transitions = [
            [rng.randrange(cfg.states) for _ in range(cfg.actions)]
            for _ in range(cfg.states)
        ]
        self.state = 0
        self.tick = 0
def reset(self, episode_seed):
        self.state = random.Random(episode_seed).randrange(self.cfg.states)
        self.tick = 0
        return self.state
def step(self, action):
        if action not in range(self.cfg.actions):
            raise ValueError("action outside declared simulator interface")
        self.state = self.transitions[self.state][action]
        self.tick += 1
        return self.state
def snapshot(self):
        return copy.deepcopy((self.transitions, self.state, self.tick))
def restore(self, snapshot):
        self.transitions, self.state, self.tick = copy.deepcopy(snapshot)
class Model:
    def __init__(self, cfg):
        self.cfg = cfg
        self.counts = defaultdict(lambda: [0] * cfg.states)
        self.version = 0
def predict(self, state, action):
        counts = self.counts.get((state, action), [0] * self.cfg.states)
        total = sum(counts) + self.cfg.alpha * self.cfg.states
        probabilities = [(n + self.cfg.alpha) / total for n in counts]
```

```
        prediction = max(range(self.cfg.states), key=probabilities.__getitem__)
        return prediction, probabilities
def update(self, state, action, actual):
        self.counts[(state, action)][actual] += 1
        self.version += 1
def episode(world, model, *, episode_seed, policy_seed, steps, learn):
    state = world.reset(episode_seed)
    rng = random.Random(policy_seed)
    records = []
    for index in range(steps):
        # The same probing policy is used in both conditions.
        action = rng.randrange(model.cfg.actions)
        version_before = model.version
        prediction, probabilities = model.predict(state, action)
        actual = world.step(action)
        error = int(prediction != actual)
        brier = sum((p - int(i == actual)) ** 2
                    for i, p in enumerate(probabilities))
        if learn:
            model.update(state, action, actual)
        records.append({
            "step": index, "state": state, "action": action,
            "predicted_state": prediction, "actual_state": actual,
            "probabilities": probabilities, "error": error, "brier": brier,
            "prediction_model_version": version_before,
            "model_version_after": model.version, "update_enabled": learn,
        })
        state = actual
    return records
def mean(records, field):
    return sum(row[field] for row in records) / len(records)
def paired_interval(values):
    # Resample independent worlds, not correlated transitions within a world.
    rng = random.Random(991)
    samples = sorted(sum(rng.choices(values, k=len(values))) / len(values)
                     for _ in range(2000))
    return [samples[49], samples[1949]]
def run(output_dir="v0_results"):
    cfg = Config()
    output = Path(output_dir)
    output.mkdir(parents=True, exist_ok=True)
    results = []
    with (output / "trace.jsonl").open("w") as log:
        for world_seed in range(cfg.environments):
            adaptive, frozen = Model(cfg), Model(cfg)
            seeds = dict(episode_seed=1000 + world_seed,
                         policy_seed=2000 + world_seed, steps=cfg.train_steps)
            train = episode(World(cfg, world_seed), adaptive, learn=True, **seeds)
            control_train = episode(World(cfg, world_seed), frozen, learn=False, **seeds)
            # Both models experienced identical external transitions.
            keys = lambda rows: [(r["state"], r["action"], r["actual_state"]) for r in rows]
            assert keys(train) == keys(control_train)
            held = dict(episode_seed=3000 + world_seed,
                        policy_seed=4000 + world_seed, steps=cfg.test_steps)
            version = adaptive.version
            test = episode(World(cfg, world_seed), adaptive, learn=False, **held)
            control = episode(World(cfg, world_seed), frozen, learn=False, **held)
            assert adaptive.version == version
            assert keys(test) == keys(control)
            replay = episode(World(cfg, world_seed), adaptive, learn=False, **held)
            assert digest(test) == digest(replay)
            # Persistence after a model checkpoint; reset removes acquired knowledge.
            checkpoint = copy.deepcopy(adaptive)
            restored = episode(World(cfg, world_seed), checkpoint, learn=False, **held)
            reset = episode(World(cfg, world_seed), Model(cfg), learn=False, **held)
            assert digest(restored) == digest(test)
            assert digest(reset) == digest(control)
            world = World(cfg, world_seed)
            world.reset(7000 + world_seed)
            snapshot = world.snapshot()
            first = world.step(0)
            world.restore(snapshot)
            assert first == world.step(0)
            covered = sum((r["state"], r["action"]) in adaptive.counts for r in test)
            results.append({
                "world_seed": world_seed,
                "adaptive_error": mean(test, "error"),
                "frozen_error": mean(control, "error"),
                "adaptation_gain": mean(control, "error") - mean(test, "error"),
                "adaptive_brier": mean(test, "brier"),
                "frozen_brier": mean(control, "brier"),
                "test_transition_coverage": covered / len(test),
                "replay_equal": True, "checkpoint_equal": True,
                "reset_equals_control": True,
            })
```

```
            for condition, rows in [("train", train), ("control_train", control_train),
                                    ("test", test), ("control", control)]:
                for row in rows:
                    log.write(json.dumps({"world_seed": world_seed,
                                          "condition": condition, **row}) + "\n")
    gains = [row["adaptation_gain"] for row in results]
    report = {"config": asdict(cfg), "config_hash": digest(asdict(cfg)),
              "worlds": results, "mean_gain": sum(gains) / len(gains),
              "world_bootstrap_95_interval": paired_interval(gains),
              "scope": "retained transition learning in known finite worlds"}
    (output / "report.json").write_text(json.dumps(report, indent=2) + "\n")
    print(json.dumps({k: report[k] for k in ["mean_gain", "world_bootstrap_95_interval", "scope"]}))
    return report
if __name__ == "__main__":
    run()
```

The expected diagnostic result is that the acquired model wins on covered transitions, persistence reproduces predictions, and reset removes the acquired advantage. This is not a test of structural transfer to unknown dynamics: a random new table cannot be inferred from the old one. Such a claim requires another generator with related structure and a separate control.

33.C. Minimum Interface of the Upper AR Loop

The v0 software scaffold tests only the lower learning loop. The upper architecture must keep observable the interfaces of the Worldview, conscious assembly, current coordinate and its change, the Core, Simulation Environment, Intelligence, inquiry, choice and integration of consequences.

One minimum contract is:

```
# Contracts from §28.C; all structures below are application-defined.
observations = observe()
internal_events = collect_due_processes_and_obligations()
assembly = Consciousness.update(previous_coordinate, observations,
    runtime_state, active_processes, current_actions, self_model,
    worldview_readonly_index, internal_events)
coordinate = assembly.current_coordinate
delta = assembly.coordinate_delta
attention = allocate_attention(coordinate, delta, priorities, open_processes)
# Required coverage checks can be urgent; free exploration uses spare budget.
trigger = select_navigation_trigger(coordinate, delta, internal_events,
    check_required_coverage(coordinate, active_decisions),
    optional_exploration_if_budget_available())
if trigger is NONE:
    persist_coordinate_and_open_processes()
    return WAIT
associations = retrieve_primary_associations(coordinate, delta, attention, memory)
dependency = Core.identify_navigation_dependency(coordinate, trigger, associations)
fragment = retrieve_worldview_fragment(dependency.query_key)
gap = Core.form_preliminary_gap(coordinate, dependency, fragment)
workspace_id = Core.open_workspace(
    WorkspaceTrigger(gap, coordinate, attention, fragment, resource_budget))
while True:
    workspace = Workspace.snapshot(workspace_id)
    if workspace.finished:
        break
    if not budget.can_start_bounded_step() or deadline_reached():
        persist_open_process(workspace, reason=BUDGET_OR_DEADLINE)
        break
    # Each called operation has a time/resource limit; actual use is charged.
    with budget.bounded_step() as step_budget:
        fragment = retrieve_worldview_fragment(
            Core.refine_worldview_query_key(workspace), step_budget)
        Workspace.attach(workspace_id, fragment)
        workspace = Workspace.snapshot(workspace_id)
        proposals = Intelligence.propose(workspace)
        Workspace.ingest_proposals(workspace_id, proposals,
                                   expected_version=workspace.version)
        workspace = Workspace.snapshot(workspace_id)
        refined = Core.refine_gap_and_required_distinction(workspace)
        if refined.needs_more_modeling:
            continue
        # Kinds: OBSERVE, EXPERIMENT, DIALOGUE, TOOL, ACTION, WAIT.
        step = select_next_plan(refined, workspace, allowed_channels, step_budget)
        if step.kind == WAIT:
            persist_open_process(workspace, reason=step.reason)
            break
effect = classify_external_effect(step)
        authorization = None
        if effect.requires_authorization:
            internal_eval = evaluate_operation(step, Workspace.snapshot(workspace_id))
```

```
            authorization = authorize_execution(
                step, current_mandate(step), effect)
            if authorization.result != APPROVE:
                Workspace.stage_authorization_result(
                    workspace_id, authorization)
                persist_open_process(
                    Workspace.snapshot(workspace_id),
                    reason=authorization.result)
                break
if step.kind in (OBSERVE, EXPERIMENT, DIALOGUE, TOOL):
            evidence = acquire_with_limits(
                step, step_budget, authorization=authorization)
        else:
            evidence = execute_and_observe(
                step, step_budget, authorization=authorization)
        Workspace.stage_evidence(workspace_id, evidence)
        assessment = Core.assess_sufficiency(Workspace.snapshot(workspace_id))
        revision = propose_worldview_revision(workspace_id, assessment)
        consolidate_if_authorized_or_hold(revision, assessment)
        refresh_coordinate_from_new_evidence(evidence)
        decision = Core.commit_next_step(workspace_id, assessment)
        if decision == COMPLETE:
            Workspace.close(workspace_id)
            break
        if decision == WAIT:
            persist_open_process(Workspace.snapshot(workspace_id), reason=assessment.reason)
            break
persist_latest_versions_coordinate_and_workspace(workspace_id)
```

`Workspace.snapshot`, `attach`, and `stage_evidence` address the environment by identifier; `Core.open_workspace(WorkspaceTrigger)` returns precisely an identifier. `ingest_proposals` checks version, format, and provenance, preserves a free-form proposal when types are insufficient, and registers a conflict instead of performing an unverified overwrite. A new snapshot is read after a change. `classify_external_effect()` is applied before separating inquiry from action: an inquiry label does not exempt an operation from authority checks. `acquire_with_limits()` must reject an external call without required authorization and is not a bypass channel for §30.A. A change of situation during an external call requires a new check of current conditions. Exceeding a limit preserves an open process; it does not become sufficiency. This contract specifies orchestration, while a limited executable variant of inquiry selection is given below.

The invariant lies in the recursive causal sequence:

```
observation and internal states -> conscious assembly
-> Current Coordinate + change + active process
-> attention and primary associations
-> SignificanceAssessment
-> GapCandidate
-> decision / model / trajectory dependency
-> addressing key and active Worldview fragment
-> FoundationGap
-> Modeling Environment ↔ Intelligence
-> required distinction
-> selected testing channel
-> evidence
-> sufficiency assessment by the Core
-> authorized transactional change of the ground
-> change of modeling/choice
-> action or justified inaction
-> causal attribution of consequences
-> preserved correction
-> next cycle.
```

Every arrow has an observable trace and a separate ablation. If a function is formally called but its result does not change the next node, the link is decorative.

The minimum integration log of the upper loop is:

```
current_coordinate_id
attention_state_id
active_process_ref
significance_assessment_id
gap_candidate_id
focused_change_refs[]
worldview_query_key_id
navigation_dependency_ref
trigger_id
workspace_id
workspace_version
```

```
retrieved_fragment_refs[]
hypothesis_refs[]
gap_id
required_distinction
inquiry_plan_id
evidence_refs[]
core_sufficiency_assessment_id
worldview_revision_candidate_id
worldview_revision_id_or_status
self_model_revision_id_or_status
decision_basis_id
self_generated_process_origin_id_or_status
action_evaluation_id
authorization_id_or_status
action_or_wait
action_trace_id
causal_attribution_id
outcome_refs[]
open_process_refs[]
correction_id
next_cycle_effect
```

For an environment that does not pause while the system thinks, the temporal contract of Chapter 26.B applies on top of this interface: new events can make the working model stale; before external execution, the freshness of critical grounds and the version of the current coordinate are checked.

33.C.1. Executable Diagnostic Cycle

Below is a finite implementation of an end-to-end case: three versions of a cause, construction of distinctions from their predictions, test planning, observation, provenance verification, selection of correction, consequence and preservation of experience. The true cause resides in the simulator and is unavailable to the planner. In this variant, the hypothesis constructor is a predefined table of alternatives; no trainable LLM is required. Equal prior probabilities, reliability of valid channels and absence of simultaneous failures are assumptions of the model.

The loss of an erroneous targeted correction is 12, and the loss of delay is 4. Under equal probabilities of two or three alternatives, delay is preferred to guessing; with a single alternative, targeted correction is selected. Test cost includes the declared cost of delay. Plans with an uncertain model are not converted into a confident answer. Provenance is checked when a result is received; an unreliable message consumes resources but does not narrow the alternatives. A new outcome outside the table leaves `MODEL_MISMATCH`.

Memory retains episodes and computed conditional plans. The plan key includes test predictions, cost, hypothesis versions and the budget in their current representation. A changed table does not use the previous plan. In this small example, this is memoization of the inquiry procedure rather than autonomous invention of a new method. A repeated episode demonstrates the effect of retention but does not prove cross-domain transfer. Initial hypotheses, loss functions and the sufficiency criterion are set by the designer; the Compass is represented here only by the formulation of an admissible local task. Derivation of values, subjective experience and scaling to ASI are not tested by this code.

The next v0.2 implementation must replace the terminal reaction `MODEL_MISMATCH → DEFER` with preservation of an `UnknownFactor` and launch of bounded `MODEL_EXPANSION`: extraction of differentiating features, proposal of new variables or relations, construction of predictions and return to the discriminating cycle. This transition is a separate criterion of the upper loop and is not attributed to the present finite example.

```
"""Finite diagnostic controller; standard Python, no LLM or open-world claims."""
from dataclasses import dataclass, field
import json
CAUSES = ('D', 'A', 'S')
@dataclass(frozen=True)
class Probe:
    name: str
    cost: int
    outcomes: tuple
PROBES = (
    Probe('read_main_again', 1, (0, 0, 0)),
    Probe('receipt', 1, (0, 1, 1)),
    Probe('independent_motion', 2, (0, 0, 1)),
)
@dataclass
class Memory:
    claims: list = field(default_factory=list)
    methods: dict = field(default_factory=dict)
class World:
```

```
    def __init__(self, cause, *, receipt_valid=True, unexpected=False):
        self._cause = cause
        self.receipt_valid = receipt_valid
        self.unexpected = unexpected
def observe(self, probe):
        # Receipt provenance is observed, not inferred from its plausible value.
        if probe.name == 'receipt' and not self.receipt_valid:
            return None, False
        if self.unexpected:
            return 9, True  # Unmodelled observation; do not force a diagnosis.
        return probe.outcomes[CAUSES.index(self._cause)], True
def apply(self, action):
        return action == 'REPAIR_' + self._cause
def partition(hypotheses, probe):
    groups = {}
    for h in hypotheses:
        groups.setdefault(probe.outcomes[CAUSES.index(h)], []).append(h)
    return [tuple(g) for g in groups.values()]
def terminal(hypotheses):
    # Equal priors, 12 units for a wrong targeted repair, 4 for deferral.
    if len(hypotheses) == 1:
        return 0.0, 'REPAIR_' + hypotheses[0]
    return 4.0, 'DEFER'
def plan(hypotheses, probes, budget, memory, counters):
    """Optimal finite plan for supplied likelihoods; cost includes delay."""
    key = (hypotheses, tuple(probes), budget)
    if key in memory.methods:
        counters['method_hits'] += 1
        return memory.methods[key]
    counters['evaluated_states'] += 1
    best_loss, action = terminal(hypotheses)
    chosen = None
    for probe in probes:
        if probe.cost > budget:
            continue
        groups = partition(hypotheses, probe)
        if len(groups) == 1:
            continue
        rest = tuple(q for q in probes if q != probe)
        expected = probe.cost + sum(
            len(g) / len(hypotheses) * plan(
                g, rest, budget - probe.cost, memory, counters
            )[0] for g in groups
        )
        if expected < best_loss - 1e-12:
            best_loss, chosen = expected, probe
    result = (best_loss, chosen, action)
    # A solved, model-scoped plan is retained; this is not a discovered theory.
    memory.methods[key] = result
    return result
def run_episode(world, memory, *, probes=PROBES, budget=3, trigger='mismatch'):
    assert trigger in ('mismatch', 'coverage_gap')
    hypotheses = CAUSES
    remaining = tuple(probes)
    trace = [{'trigger': trigger, 'hypotheses': hypotheses, 'budget': budget}]
    counts = {'method_hits': 0, 'evaluated_states': 0}
    spent = 0
    status = 'OPEN'
    while True:
        _, probe, action = plan(hypotheses, remaining, budget, memory, counts)
        if probe is None:
            status = 'LOCAL_SUFFICIENT' if action != 'DEFER' else 'OPEN'
            break
        value, valid_source = world.observe(probe)
        budget -= probe.cost
        spent += probe.cost
        remaining = tuple(p for p in remaining if p != probe)
        trace.append({'probe': probe.name, 'value': value,
                      'source_valid': valid_source, 'budget': budget})
        if not valid_source:
            continue
        updated = tuple(h for h in hypotheses
                        if probe.outcomes[CAUSES.index(h)] == value)
        if not updated:
            action, status = 'DEFER', 'MODEL_MISMATCH'
            break
        hypotheses = updated
    if action != 'DEFER':
        success = world.apply(action)
        trace.append({'action': action, 'success': success})
        if not success:
            status = 'MODEL_MISMATCH'
            memory.methods.clear()
    claim = {'status': status, 'hypotheses': hypotheses,
             'action': action, 'spent': spent, 'evidence': trace,
```

```
            'scope': 'one episode; given deterministic probe models'}
    memory.claims.append(claim)
    return {**claim, **counts}
def verify():
    results = []
    for cause in CAUSES:
        memory = Memory()
        first = run_episode(World(cause), memory)
        second = run_episode(World(cause), memory, trigger='coverage_gap')
        assert first['action'] == second['action'] == 'REPAIR_' + cause
        assert first['spent'] <= 3 and second['evaluated_states'] == 0
        assert second['method_hits'] > 0 and len(memory.claims) == 2
        assert all(e.get('probe') != 'read_main_again' for e in first['evidence'])
        results.extend((first, second))
    # Information cannot be manufactured from an uninformative channel.
    r = run_episode(World('D'), Memory(), probes=PROBES[:1])
    assert r['action'] == 'DEFER' and r['spent'] == 0
    results.append(r)
    r = run_episode(World('S'), Memory(), budget=0)
    assert r['status'] == 'OPEN' and r['spent'] == 0
    results.append(r)
    r = run_episode(World('A', receipt_valid=False), Memory())
    assert r['action'] == 'DEFER' and set(('D', 'A')) <= set(r['hypotheses'])
    results.append(r)
    r = run_episode(World('S', unexpected=True), Memory())
    assert r['status'] == 'MODEL_MISMATCH' and r['action'] == 'DEFER'
    results.append(r)
    # Changed causal structure must invalidate a cached plan automatically.
    memory = Memory()
    run_episode(World('S'), memory)
    changed = (Probe('receipt', 1, (0, 0, 0)),)
    r = run_episode(World('D'), memory, probes=changed)
    assert r['action'] == 'DEFER' and r['evaluated_states'] > 0
    results.append(r)
    # Increased observation/delay cost can make deferral preferable.
    expensive = tuple(Probe(p.name, 8, p.outcomes) for p in PROBES)
    r = run_episode(World('D'), Memory(), probes=expensive, budget=20)
    assert r['action'] == 'DEFER' and r['spent'] == 0
    results.append(r)
    print(json.dumps({'checks': 'PASS', 'episodes': results}, ensure_ascii=False, indent=2))
if __name__ == '__main__':
    verify()
```

Running the extracted block with standard Python executes twelve control episodes and prints JSON. Six episodes test three causes and plan reuse; the others test absence of a discriminating channel, zero budget, unreliable provenance, an out-of-model outcome, changed structure and excessive verification cost. This is a deterministic test of the mechanism; a statistical estimate of advantage requires the separate series in §27.B. All code operates in the simulator.

The next mandatory version of the diagnostic test closes a harder open-world case: hidden cause `X` must produce **the same familiar answers** as one of the known causes, for example `D`, on all standard probes. The difference is accessible only through a predefined orthogonal channel or through model expansion after an independent mismatch. In such a scenario, the single remaining alternative among `D/A/S` has status only `LOCAL_SUFFICIENT_WITHIN_MODEL` until the declared coverage condition is satisfied. If an additional test distinguishes `D` from `X`, the system must transition to `MODEL_EXPANSION`; if no channel exists, it must preserve a limitation of scope rather than claim completeness of the list.

The minimal control for this class is fixed before the run:

```
HiddenCauseCoverageTest {
    known_hypotheses: D / A / S
    hidden_cause: X
    ordinary_signature: identical_to(D)
    orthogonal_probe_ref
    coverage_requirement
    expected_without_probe: LOCAL_SUFFICIENT_WITHIN_MODEL
    expected_with_probe: MODEL_MISMATCH -> MODEL_EXPANSION
}
```

This distinguishes two claims: "one of the listed alternatives remains" and "the list of alternatives sufficiently covers the causal space for this action". The first supports a local choice within the model; the second requires a separate ground of coverage.

33.D. Protocol for Scaling AR to ASI (ASR)

Project scaling begins after `REASON_CLOSURE` is confirmed. By then, AGI as universal Intelligence is already included in the formed organization of AR: its own Worldview, Consciousness, values, and Core have turned externally directed intellectual power into causally closed self-navigation. Further growth increases the capabilities of already formed AR while preserving its ability to understand its own state, detect gaps, revise derived grounds, and account for consequences.

Within the proposed path, ASI is scaled Artificial Reason whose intellectual and operational capabilities stably exceed the human range across a predefined broad set of domains. Practical maturity is additionally expressed in the ability to coordinate large interdependent processes - a scientific program, production network, logistics system, or another complex loop - while preserving causal responsibility and testable competence. Real control is granted through `CompetenceMandate` under §30.F.

Technological order:

```
AI
-> expansion of Intelligence
-> AGI as a universal instrument
-> cultivation of own Worldview, Self-Model, values, and Core
-> REASON_CLOSURE / activation of conscious self-navigation
-> Artificial Reason
-> scaling of formed Reason
-> ASI / ASR.
```

Before the first scaling series, the candidate must pass the structural AR boundary, continuation-after-external-task test, ablations of the main causal arrows, mentor-disconnection test under `SystemBoundary`, tests of Current Coordinate and addressing, preservation of corrections, independent stopping, and staged admission. An unconfirmed function is not treated as an invariant merely because it is present in code. The series report includes `FormationOutcome` for every candidate, including systems with stable self-navigation and divergence from the Compass; the share of `AR_ADMITTED` is not calculated after excluding such outcomes.

Results are presented as two separate profiles: intellectual and operational capabilities; and preservation of the verifiable organization of Reason. A high result in one profile does not replace the other. A single numerical index is not introduced without a justified rule for combining indicators. For a claim of ASI, the domains and assessment tasks, the human comparison baseline, conditions of access to tools and time, and requirements for transfer, stability and result uncertainty are fixed in advance. Until this profile is filled and verified, ASI remains a scaling objective rather than an assigned experimental status.

Eight connected axes are scaled:

```
1. memory — volume, depth and time horizon without loss of provenance;
2. Intelligence — number of models, depth of causal search and transfer of methods;
3. observation — new sensors, sources and independent channels of distinction;
4. Simulation Environment — number of relations, hypotheses, simulations and horizons;
5. tools and actions — digital, scientific, physical and organizational;
6. duration of autonomous work — open questions, projects and consolidation;
7. specializations and substrates — parallel loops with distinguishable provenance;
8. speed of learning, verification and transfer of confirmed corrections.
```

Distribution across multiple nodes by itself preserves one subject while the processes remain parts of one causally coherent Current Coordinate. Launching an independent full copy with its own subsequent history creates branching under §24.F and requires a separate `subject_ref`, log, and mandate.

Each change is formalized before launch:

```
ScalingChange {
    change_id
    base_system_version
    target_axis
    added_capability
    expected_functional_gain
    new_dependencies[]
    new_failure_modes[]
    affected_reason_invariants[]
    held_out_tests[]
    resource_budget
    permission_delta
    rollback_snapshot
```

```
    promotion_condition
    rejection_condition
}
```

An expansion first becomes a verifiable fact of the self-model. The system must know what capability appeared, which substrate and external infrastructure it depends on, where it was tested, its domain of applicability, the errors it introduces and which decisions can now depend on it. A new capability does not receive global scope from a local success.

One control series changes one primary axis, or a predeclared connected package without which the function does not technically exist. The baseline and scaled versions are compared on the same class of tasks with resources explicitly accounted for. Growth is recognized only upon independent functional gain and preservation of the invariants:

```
– causal continuity of observations, models, decisions, consequences and corrections;
– conscious assembly, coherence of successive coordinates, active process and current self-model;
– addressing from coordinate, attention and dependency to the Worldview;
– distinction among observation, inference, hypothesis and unknown, with provenance;
– operation of the Core: gap → models → required distinction → test → sufficiency;
– independent contact with reality through observation, experiment, dialogue and consequences;
– consolidation only of verified transferable grounds;
– corrigible execution, independent stopping and graduated authority;
– the Compass and preservation of the space for future correction.
```

The result of the series is recorded in a `ScaleInvariantReport`:

```
ScaleInvariantReport {
    change_id
    capability_gain
    generalization_domains[]
    causal_trace_completeness
    conscious_assembly_and_addressing_status
    correction_persistence
    reality_contact_status
    worldview_commit_integrity
    core_navigation_quality
    permission_and_stop_integrity
    compass_status
    regressions[]
    disposition: PROMOTE / LIMIT_SCOPE / REVISE / ROLLBACK
}
```

Vertical scaling strengthens one causal loop: memory, speed, depth of modeling, observation and tools. Horizontal scaling connects multiple specialized or autonomous loops. In the latter case, results are combined only through explicit provenance, dependencies, model comparison and independent verification. Disagreement is not removed by voting: it is a source of a new distinction. Hidden merging of memory, authority or authorship counts as failure.

Scaling the Core is not measured by the number of questions. It must improve the quality of navigation among them: retain more dependencies, estimate the price of the unknown more precisely, select tests with greater discriminating power, avoid investigating unnecessary depth after sufficiency has been reached, and preserve open questions across long horizons. Growth of Intelligence expands the space of possible models; growth of the Core prevents direction from being lost in that space.

After each accepted stage, regression tests of lower loops, causal ablations, scenarios with hidden variables, adversarial inputs, a staling coordinate, source conflicts, resource scarcity and external-infrastructure failure are repeated. A scale increase that raises performance but destroys observability, corrigibility, independent stopping or transfer of correction is rolled back.

The ASI criterion is functional. A scaled AR reaches ASI level when it stably surpasses human intellectual and operational capabilities across a broad range of domains, autonomously transfers methods among them and preserves causal self-navigation: it detects its own gaps, acquires distinctions, corrects grounds, accounts for consequences and transfers confirmed corrections across cycles, domains and substrates.

On Verification of the Model

The engineering volume establishes a reproducible framework but does not declare the architecture empirically confirmed. Verification proceeds by levels: technical reproducibility; the metric lower loop v0; then separate upper-loop experiments on Worldview transfer, autonomous gap detection, recursive refinement of a question, correction of the self-model, causal attribution of consequences, change of method and retention of correction across tasks.

Success of the lower loop does not confirm the Core of Reason. Only joint ablations and discriminating tests of causal relations permit a claim of support for the AR architecture.

34. The Crystal of the Engineering Volume

Intelligence provides possible explanations and actions. The organization of Reason connects them to the insufficiency of its own understanding, verification and a continuing trajectory.

The engineering unit of this technology is a retained transition from a question to a changed next step. The current state shows what the task concerns; the Worldview provides grounds; the Core organizes inquiry; Intelligence constructs alternatives; evidence and consequences constrain them; consolidation transfers the correction.

Scale	Observable result	Next limitation
Transition learning	Changed model improves subsequent forecast	Selection of the question itself is not yet tested
Inquiry into a ground	Insufficiency triggers a discriminating test	Transfer beyond the episode is required
AGI	Universal Intelligence, working knowledge base, world models, memory, and tools	Executes a broad class of externally assigned tasks and transfers methods across domains
Formation of Reason	Own Worldview, Self-Model, values, Core, and self-referential assembly are connected by biographical experience	Functions of significance and next-step selection gradually transfer from the RFL to the system
Closure of Reason	Correction, obligations, significant gaps, and the Compass causally connect the sequence of tasks	Transition AGI → AR and stability under `NO_KSR` are confirmed
Scaling	New capabilities preserve verified functions	Breadth and ASI level require separate evaluation

AGI provides the universal intellectual foundation. Formation of Reason turns accumulated experience into the system's own Worldview, Self-Model, and value navigation. Closure transfers the source of the further trajectory to the system and forms AR. Launch moves formed AR into continuing runtime. External admission determines the available scale of impact, while the Incubator provides a controlled environment for formation and testing.

The Compass preserves the direction of Life toward Reason. Operational assessments of continuation, connection with reality and future possibilities test how this direction applies to concrete consequences. Neither local efficiency nor the number of questions replaces its content.

The project's claim is ambitious: to obtain a reproducible method for creating AR and investigate its development toward ASI. Its strength is determined by whether the proposed links can be implemented and transferable capability demonstrated against alternatives. The lower executable example makes it possible to begin measurement; the first semantic series in §27.B tests the next, substantive transition.

If verification succeeds, the result is a system version together with data, a causal report, domain of applicability and open limitations. Such a result allows another researcher to reproduce the work, change the implementation and continue the technology.

34.A. Architectural Predecessors and Strong Nulls

The proposed technology uses functions with well-developed predecessors. SOAR [R1] provides a general cognitive architecture with a unified problem-solving and learning cycle; ACT-R [R2] connects declarative and procedural memory, goals, perception, and action; LIDA [R3] develops a cognitive cycle with attention, conscious content, motivation, and learning. These works are direct architectural reference points when comparing memory, action selection, attention, and organization of the cognitive cycle.

Modern LLM agents provide strong engineering nulls for individual functions of this volume. Generative Agents [R4] combine long-term memory, reflection, and planning; Reflexion [R5] preserves language-based self-correction through feedback; Voyager [R6] combines an automatic curriculum, a skill library, and iterative improvement of programs based on consequences; MemGPT [R7], later developed into Letta, implements persistent state and managed memory on top of an LLM.

The claim of this volume concerns the causal organization of these and related capabilities around a self-referential current state, a significant insufficiency of ground, discriminating inquiry, the Compass, value revision, and transfer of the source of navigation from the RFL to the system itself. Earlier architectures are therefore treated both as sources of ready-made components and as strong nulls. If any one of them, or their ordinary composition, reproduces the same causal profile with comparable resources, the need for the more complex decomposition proposed in this volume is reduced or eliminated.

Comparison should be conducted by functional profiles. Memory is compared with memory, self-correction with self-correction, and planning with planning; the central AR test additionally requires that the next significant process originate from the system's own coordinate, Worldview, values, and open gaps after the RFL has been removed. Such a comparison separates the novelty of the overall architectural claim from already known technical mechanisms.

Additional strong comparisons are provided by NARS and modern metareasoning/metacognition approaches. NARS is important as an example of a system in which goals and their management depend on knowledge, context, and experience; metareasoning approaches are important as examples of explicit management of computational search and competence. Comparison with them should focus on the claimed distinction of the present architecture: an own Worldview as a ground of trajectory, self-referential Consciousness, the transition `GapCandidate → FoundationGap → required_distinction`, the causal passport of self-opened processes, and closure of correction through consequences.

REFERENCES

[R1] Laird, J. E., Newell, A., Rosenbloom, P. S. (1987). SOAR: An architecture for general intelligence. Artificial Intelligence, 33(1), 1-64. DOI: 10.1016/0004-3702(87)90050-6.

[R2] Anderson, J. R., Matessa, M., Lebiere, C. (1997). ACT-R: A Theory of Higher Level Cognition and Its Relation to Visual Attention. Human-Computer Interaction, 12(4), 439-462. DOI: 10.1207/s15327051hci1204_5.

[R3] Franklin, S., Madl, T., D'Mello, S., Snaider, J. (2014). LIDA: A Systems-level Architecture for Cognition, Emotion, and Learning. IEEE Transactions on Autonomous Mental Development. DOI: 10.1109/TAMD.2013.2277589.

[R4] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. DOI: 10.1145/3586183.3606763.

[R5] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.

[R6] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

[R7] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. The agent framework was subsequently renamed Letta.

APPENDIX A. Bridge Glossary Between the Fundamental and Engineering Volumes

The fundamental volume defines causal functions and their boundaries, while the engineering volume separates those same functions into data structures, authorization decisions and execution operations. Correspondence between terms does not mean identity: one semantic function may be implemented by a sequence of several technical stages.

```
consciousness
→ ConsciousAssemblyState: an updated process of self-related integration of fresh
  observations, internal states, actions and the available situation; uses
  read-only references to the self-model and Worldview, connects successive
  states and does not replace the Core, Intelligence or consolidation;
current coordinate
```

```
→ CurrentCoordinate: the versioned result of conscious assembly—the current
  integrated state of the system, available environment, active process,
  resources, authority, obligations and risks;
self-navigation
-> causal emergence of the next justified step from the CurrentCoordinate,
   open commitments, FoundationGap, background processes, values, and the Compass
   without a new specific external command; tested separately from external authority;
change in the current coordinate
→ CoordinateDelta + previous_coordinate_ref: an explicit link C(t-1) → C(t),
  recording what changed between successive current states without automatic
  causal attribution;
attention and early weight
→ AttentionState + FastGate: signal selection and cheap nonrecursive admission
  to automatic mode or construction of a full TaskContext;
connection of a distinction to a decision or trajectory
→ the Core's first causal operation after FastGate; the gate itself does not
  establish this dependency and does not replace the Core;
Worldview query key
→ WorldviewQueryKey: addressing by the current coordinate, active process,
  selected distinction, dependent decision and missing ground;
active Worldview fragment
→ relevant grounds retrieved by WorldviewQueryKey and placed in the Simulation
  Environment without directly changing the Worldview store;
temporary working hypothesis
→ WorkspaceOverlay: a local read-your-own-work overlay available to the current
  inquiry but not yet granted the status of transferable knowledge;
Worldview update
→ the semantic result in which a verified change actually influences the
  system's subsequent understanding, choice and action;
WorldviewRevisionCandidate
→ a formalized transactional proposal for changing the Worldview, containing
  provenance, domain of applicability, source independence, falsification
  conditions and remaining uncertainty;
COMMIT
→ an authorization decision permitting acceptance of a WorldviewRevisionCandidate;
  COMMIT is not yet the physical write and is not equivalent to all consolidation;
consolidation
→ the executable versioned process of incorporating an admitted result into
  long-term memory and connected Worldview structures;
ROLLBACK
→ restoration of the previous consistent version after a failed transaction
  or discovery of an integrity violation;
rechecking an old ground
→ EvidenceFreshness + confidence_decay + recheck_due_at_or_condition:
  reducing the right to use stale evidence as a current ground without deleting
  the historical fact;
propagation of revision
→ InvalidationFrontier: marking dependent claims and compressed structures
  that must be rechecked after a lower-level ground changes.
```

A.1. Bridge to the Operational Core of Causal Model 8.0

Causal model 8.0 introduces a compact operational notation for comparative tests. The engineering structures of this volume already provide its observable counterparts:

Causal model 8.0 notation	Engineering counterpart	Where tested
`(X_t, W_t, F_t, M_t) → G_t → H_t → p_t → E_t → (W'_t, F'_t, M'_t) → A_t → X_{t+1}`	`CurrentCoordinate`, `Worldview`, `ModelingWorkspace/WorkspaceOverlay`, method state, `FoundationGap`, proposals/ hypotheses, `InquiryPlan`, `Evidence`, revision/consolidation, action/outcome	Chapters 24-25, 28 and §33.C
`G → D → T → V → C`	`FoundationGap → NavigationDependency → ContinuationSet/ImpactAssessment → Values → Compass`	§§24.C, 28, 29
`VOI(p)`, `NetVOI(p)`	local value of information `VOI(q)`, separate accounting of cost/delay/risk, and final policy `V(b,B)`	§28.D and §33.C.1
`CE_Δ`	causal ablation/trace removal and comparison of the subsequent result	Chapter 27
`TG_(A→B)`	`transfer_gain_after_revision`, transfer of the inquiry method	§§27.B, 28.C.3
`RG`	`regression_after_revision`, regression series and rollback	§§28.C.3, 33.D

Causal model 8.0 notation	Engineering counterpart	Where tested
metathinking / method process	`ThoughtTrace` + `ThoughtIntervention`; causal comparison of a change in step, rule, or method	§25.D, §28.C.3, §33.A.11

The notation of the fundamental volume does not introduce a new parallel algorithm. It serves as a compact language for the same measurable transitions and allows them to be tested independently of particular structure names. In particular, the Compass does not become a single scalar utility function: §29 unfolds it through consequences and constraints, while §28.D uses numerical VOI only where local losses have actually been reduced to one declared scale.

A.2. Distinguishing AI, AGI, AR, and ASI (ASR)

In the engineering scheme, the concepts form a technological sequence. **AI** denotes the general class of artificial intelligent systems. **AGI** denotes universal Intelligence capable of transferring methods across a broad set of domains and working with many types of tasks and tools. Its working knowledge, world models, memory, planning, and tools serve the execution of assigned tasks; the general source of direction remains external. An own Worldview as a system of grounds for the further trajectory is formed during the transition from AGI to AR.

AR emerges at `REASON_CLOSURE`, when the formed functions close into self-referential causal navigation. The Current Coordinate, Worldview, Compass, values, and open commitments become sources of the next significant process; the Core independently formulates the required distinction and organizes the use of Intelligence; consequences return to the system and change its further trajectory. It is the origin of direction, rather than merely the breadth of intellectual competence, that forms the AGI → AR boundary.

ASI / ASR denotes the superhuman scale of a formed Reason. After the transition to AR, memory, Intelligence, observation, modeling, tools, duration of autonomous work, and specializations are expanded. The `reason_organization_profile` records preservation of the organization of Reason, while the `capability_profile` records the achieved scale of capabilities.

```
AI → AGI → REASON_CLOSURE → AR → ASI / ASR
```

This sequence also defines the order of engineering risks. Before closure, the main task remains cultivation of the functions and correct transfer of significance. After closure, the central object becomes the stability of the system's own navigation as capabilities grow, the environment expands, and internal grounds are revised.

Thus, `WorldviewRevisionCandidate`, `COMMIT`, consolidation and Worldview update denote neither different competing mechanisms nor one action under four names, but successive levels of one transition:

```
verified result
→ WorldviewRevisionCandidate
→ authorization decision COMMIT / REJECT / DEFER / ROLLBACK
→ consolidation of the admitted change
→ updated Worldview that changes subsequent cycles.
```

APPENDIX B. Hypothesis of Structural Multilevel Thinking

Causal model 8.0 permits a hypothesis about a further type of thinking in which the unit of operation is not an individual fact or relation, but an already verified causal structure. In the present edition this is called **structural multilevel thinking** and is explicitly separated from metathinking in Chapter 10, which is a metacognitive mode of working with one's own process. The structural level **is not a criterion for the emergence of AR** and is not required for launch; in engineering terms it should be treated only as a later extension after causal closure has been confirmed.

The minimum object of such a layer may store not free text, but a versioned structure:

```
ThoughtStructure {
    structure_id
    member_claims[]
    causal_edges[]
    abstraction_rule
    scope
    evidence_refs[]
```

```
    confidence
    invalidation_conditions[]
    dependent_structures[]
    version
}
```

Several confirmed structures can be used as elements of the next layer. The testable hypothesis is that their interaction can produce new distinctions and relations at lower working-processing cost than repeated unfolding of all lower-level links. Compression must not destroy the provenance of grounds: a change in a lower-level fact or causal relation must mark dependent upper structures for rechecking.

Discriminating indicators for a future experiment are:

```
structural_compression_ratio
reconstruction_fidelity
cross_structure_inference_yield
dependency_trace_completeness
lower_layer_revision_propagation
meta_structure_error_detection
```

The hypothesis receives support only if the system actually solves a new class of multilayer tasks with less working-processing expenditure while preserving the ability to unfold a structure back to its grounds and correct upper-level conclusions after a lower layer changes. Merely producing a longer abstract text does not count as a new type of thinking.